Physics of Language Models: Part 4.1, Architecture Design and the Magic of Canon Layers

Zeyuan Allen-Zhu
zeyuanallenzhu@meta.com
FAIR at Meta

May 2, 2025

(version 2.0)*

Abstract

Understanding architectural differences in language models is challenging, especially at academic-scale pretraining (e.g., 1.3B parameters, 100B tokens), where results are often dominated by noise and randomness. To overcome this, we introduce controlled synthetic pretraining tasks that isolate and evaluate core model capabilities. Within this framework, we discover *Canon layers*: lightweight architectural components—named after the musical term "canon"—that promote horizontal information flow across neighboring tokens. Canon layers compute weighted sums of nearby token representations and integrate seamlessly into Transformers, linear attention, state-space models, or any sequence architecture.

We present 12 key results. This includes how Canon layers enhance reasoning depth (e.g., by 2×), reasoning breadth, knowledge manipulation, etc. They lift weak architectures like NoPE to match RoPE, and linear attention to rival SOTA linear models like Mamba2/GDN—validated both through synthetic tasks and real-world academic-scale pretraining. This synthetic playground offers an *economical, principled path* to isolate core model capabilities often obscured at academic scales. Equipped with infinite high-quality data, it may even *predict* how future architectures will behave as training pipelines improve—e.g., through better data curation or RL-based post-training—unlocking deeper reasoning and hierarchical inference.

*__V1__ appeared on SSRN on this date; __V1.1__ (May 18, 2025) improves writing and adds the relu^2 experiments (and is accepted by NeurIPS 2025); __V2__ adds GDN experiments, tightens some experiments (for a stronger, fairer comparison), and re-organizes sections. **Future updates and code release** can be found on SSRN and the project page physics.allen-zhu.com.

ZA sincerely thanks Vahab Mirrokni for the invitation to the Yale workshop in October 2023, where this research was sparked through enlightening discussions with Vahab Mirrokni and Peilin Zhong. Canon layers build on the idea of uniform attention previously explored in [6]. ZA thanks Alberto Alfarano for introducing the papers [31, 45, 66, 82], and the PyTorch scaled dot product attention function. At Meta, we extend our heartfelt gratitude to Lin Xiao and Kristin Lauter for their insightful discussions and unwavering supports, which made this research possible. Special thanks go to Wangzhi Dai, Sam Doud, Dinesh Kannappan, Niki Kim, Junjie Qian, Ammar Rizvi, Travis Seevers, and Stephen Hartken at Meta, as well as Abraham Leal from W&B; without their invaluable technical assistance, the experiments presented in this paper would not have been feasible. We are deeply grateful to Songlin Yang and Ali Behrouz for providing detailed instructions on replicating their academic-scale pretraining experiments, and Fangcheng Sun for many helpful conversations on architecture design in general.

Contribution statement. ZA proposed all ideas, conducted all investigations, implemented all code, performed all experiments, authored the entire manuscript, and managed all necessary compliance reviews and social promotions; the term *Canon Layers* was jointly conceived and designed with Xiaoli Xu.

1 Introduction

Recent advances in large language models (LLMs) have sparked transformative progress across numerous tasks, including question answering, summarization, translation, code generation [14, 16, 40, 64]. Despite rapid progress, systematic understanding of effective neural architecture design has remained elusive, fundamentally hindered by some major challenges.

Challenge 1: Pretraining loss as an unreliable proxy for intelligence. Architectural comparisons often rely on perplexity or cross-entropy loss, but these metrics do not reliably reflect real-world capabilities—especially since natural data is *skills-mixed*. For example, state-space architectures like Mamba [19, 26] frequently achieve lower perplexity early in training due to rapid memorization, yet perform poorly on complex reasoning tasks. Reliance on *early stopping via perplexity* is thus problematic: it may lead to comparing models that have merely internalized surface-level linguistic patterns without developing deeper reasoning or factual understanding [32].

Challenge 2: Noise below emergence thresholds. Emergent abilities—complex skills that only arise in large-scale models (e.g., 7B parameters, 10T tokens [1])—complicate architectural comparisons at smaller, academic scales (e.g., 1.3B parameters, 100B tokens [10, 25, 73]). At these scales, small benchmark gains (e.g., 2%) often result from random initialization (and/or data shuffling)—variance that can cause 2–4% swings in accuracy (see Figure 1). More fundamentally, **models fail even the simplest 2-hop reasoning** tasks, performing no better than random guessing.[1] This basic reasoning floor *masks architectural differences* in more advanced cognitive skills, making evaluation at this scale deeply unreliable. While large-scale industry training might reveal these differences, its prohibitive cost blocks systematic ablations, impeding academic contributions to rigorous architecture science—and often reducing design choices to heuristics and guesswork.

Challenge 3: Grokking, Data Quality, and Curriculum Learning. Failures in complex reasoning tasks typically stem from deficiencies in training data, *not* architectural limitations. Too few challenging samples and a lack of intermediate-complexity data often force models to rely on unstable grokking behavior—where generalization only emerges after unnecessarily long pretraining [44]—and disrupt curriculum learning [11]. For instance, models lacking 2-hop reasoning data may unpredictably learn 3-hop tasks after extensive exposure to 1-hop and 3-hop examples. This makes training highly sensitive to randomness, further complicating architectural comparisons. Reinforcement learning (RL)-based post-training methods, such as GRPO [55] and PPO [54], aim to address this by delivering tailored data at optimal difficulty levels. While effective, these methods introduce new experimental confounds—it becomes unclear whether performance gains stem from pretraining, RL fine-tuning, stochastic training dynamics, or architectural strength.

Our approach: Atomic decomposition of intelligence. To overcome the noise and cost of real-world pretraining—especially at academic scales where even 2-hop reasoning fails to emerge—we decompose intelligence into core (ideally atomic!) components, such as reasoning depth and breadth, and design synthetic, controllable *pretrain* tasks to isolate and evaluate them independently. This framework sharply characterizes architectural strengths and scalability under clean, idealized conditions (see Figure 1), offering a principled and economical path for architecture design.

This directly addresses Challenge 1 by enabling *single-skill evaluations*, minimizing the confounding factors prevalent in real-world pretraining data. For example, it allows rigorous comparisons of whether architecture A outperforms architecture B in reasoning depth, while ensuring modifications do not degrade other capabilities. By isolating intrinsic architectural biases, synthetic

[1] In our simplest 2-hop reasoning tasks, birth years for 3 individuals are presented, followed by 3 "[name2] was born in the same year as [name1]" equivalences. The model is prompted to infer the second group's birth years. Academic-scale pretrained models can only guess. See Result 12.

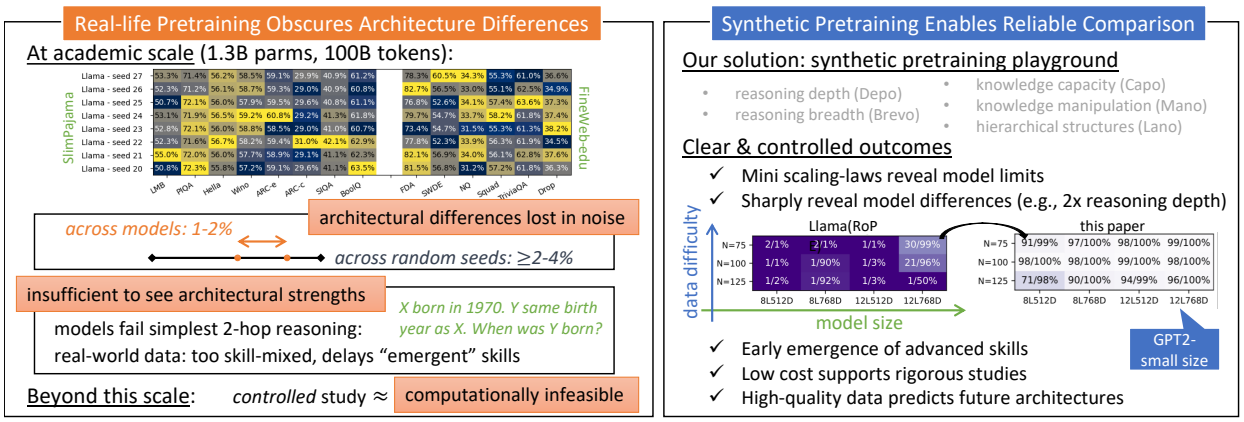


Figure 1: Architecture search in noisy real-life pretraining (good luck!) vs. our synthetic playground (scientific rigor). *See Figure 21 (Page 43) for more benchmark variability, including fixed data and varied model random init.*

pretrain tasks reveal properties often obscured by noise and mixed signals in typical real-life setups.

Challenge 2 is mitigated by *lowering resource* needs for rigorous comparisons. Synthetic benchmarks yield infinite high-quality data, enabling meaningful pretraining even for smaller models (e.g., GPT2-small) where complex skills might otherwise not emerge. In these controlled environments, capabilities like deep multi-hop reasoning *emerge clearly and reliably*, allowing rapid identification of architectural limitations, investigation of *mini scaling-laws*, and uncover trends that real-world pretrained models often fail to reveal due to noise or insufficient signal despite extensive training.

For Challenge 3, we manage data difficulty distributions to ensure adequate representation of intermediate-complexity samples, smoothing learning curves and enabling the *early and consistent emergence* of advanced skills—unlike less predictable real-world data prone to grokking-driven instability. As training pipelines improve—via better data curation or RL-based continued pretraining—synthetic pretrain benchmarks may provide *predictive insight* into which architectures best support scaling to more advanced tasks in the future.

We draw inspiration from physics, where idealized settings—such as frictionless planes or vacuum chambers—reveal first principles by removing confounding factors. Similarly, synthetic tasks eliminate the noise, randomness, and data contamination of real-world datasets, enabling clean, controlled, apples-to-apples architectural comparisons, much like Galileo's Pisa tower experiment.

This paper's key contributions are summarized below:

Result 0: Building the Synthetic Playground (Section 2+3). We introduce five synthetic pretraining tasks—Depo (reasoning depth), Brevo (reasoning breadth), Capo (knowledge capacity), Mano (knowledge manipulation), and Lano (hierarchical language structure). This controlled setup reveals clear, commonsense capability trends *at small scale*: linear attention (e.g., GLA [72]) underperforms consistently; state-space model Mamba2 [19] excels at knowledge but struggles with reasoning; and GDN [73] and Transformers dominate complex reasoning.

Result 1: Canon Layers Add Horizontal Information Flow (Section 4). Transformers lack horizontal information flow within layers, leading to inefficiencies even on simple tasks like associative recall. Drawing on the musical canon (overlapping repetition), we introduce *Canon layers*, horizontal "residual links" across neighboring tokens that can be flexibly inserted at multiple points — before attention (Canon-A), inside attention (Canon-B), before MLP (Canon-C), inside MLP (Canon-D). While Canon layers can be implemented in many ways—even simple random averaging is highly effective—this paper focuses on trainable 1-d linear convolutions of kernel size 4. This is lightweight and integrates seamlessly into any sequence model with minimal code.

Results 2–5: When Transformer Meets Canon (Section 5).

- BOOST PERFORMANCE. In our playground, Canon layers improve reasoning depth (200–400%), reasoning breadth (30%), knowledge manipulation length (30%), and more. These stem from enhanced hierarchical learning dynamics and come with minimal computational overhead.
- REVIVING NOPE. Integrating Canon layers transforms NoPE models into strong performers, often matching or surpassing RoPE(+Canon). Canon layers outperform positional fixes like ALiBi [45] or H-Alibi [31], and reducing/removing RoPE usage improves length generalization.
- ABLATION STUDY. Canon layers contribute cumulatively across sublayer positions (Canon-A/B/C/D), independently of attention or MLP components. *Residual Canon* improve training efficiency; minimal parameter tuning is required without compromising stability.
- MLP AND MOE. Canon layers can recover some knowledge capacity lost in gated MLP or mixture-of-expert (MoE) architectures, via improved training efficiency and stability.

Results 6–9: When Linear Models Meet Canon (Section 6).

- UNIVERSAL BOOST. Across all linear architectures—*GLA, Mamba2, and GDN*—Canon layers consistently enhance *reasoning*: in-context (DEPO/BREVO), knowledge (MANO), and structural (LANO), though by varying degrees.
  - For linear attention (GLA), Canon lifts reasoning depth from 1 to 4-hop, doubles reasoning breadth and knowledge length, and even surpasses Mamba2.
  - Mamba2's built-in `conv1d` (partial Canon-B) drives most of its gains; removing it drops performance to GLA, while replacing it with full Canon yields further improvements.
  - GDN benefits least, as its gating and delta updates capture part of Canon-like behavior.
- ABLATION FINDINGS. Canon's residual design ensures stability and never hurts performance. Canon-ACD alone often matches `conv1d`/Canon-B, showing horizontal context flow is universal—not limited to linear-attention or SSM sub-layers.
- ARCHITECTURAL INSIGHT. Most linear-model performance (for Mamba2/GDN) is achievable with the simple **GLA+Canon** design, suggesting that many modern refinements *might largely replicate* Canon-like mixing rather than introduce new computation.

Results 10–11: Comparing Transformers and Linear Models (Section 7) .

- CONTROLLED COMPARISONS. Equipping all architectures with full Canon layers enables a fair, apple-to-apples evaluation. Linear models show ∼40% higher knowledge capacity, but Transformers reach 2–4× greater reasoning depth and stronger structural reasoning.
- ROOT CAUSE OF SHALLOW REASONING. Linear models fall short *not from insufficient memory*—each layer's recurrent state is vastly over-provisioned—but from cumulative compression and retrieval errors, pinpointing *memory dynamics* as the main bottleneck.
- PATH FORWARD. Canon-equipped Transformer–linear hybrids can mitigate these limits, enabling deep reasoning with linear efficiency.

Result 12: Academic-Scale Real-World Pretraining (Section 8) . Pretraining 1.3B-parameter models on 100B tokens (context length 4096) shows high noise and limited resolution, making many architectural comparisons statistically unreliable. **Still, several consistent patterns emerge.** Canon layers markedly improve NoPE and GLA—raising them to match RoPE and Mamba2/GDN, respectively—while removing `conv1d` reduces Mamba2 to GLA level. Linear models lag behind full Transformers on retrieval-heavy tasks even with Canon, and all models fail 2-hop reasoning, even in short (100-token) contexts, underscoring the limits of academic-scale pretraining. Reducing or removing RoPE improves long-context generalization when Canon layers are present. **These trends mirror our synthetic results** (Results 3, 6.1, 7.1, 8.1, 9, 10, 11).

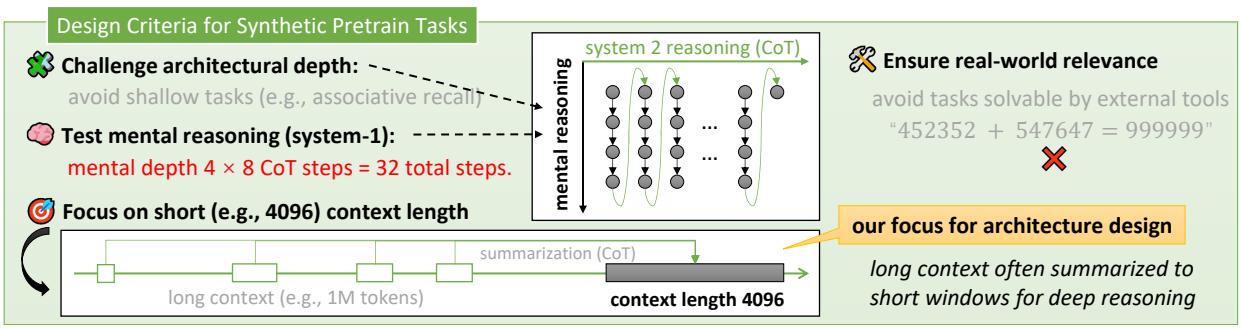


Figure 2: Our design criteria for synthetic pretrain tasks.

In summary, Canon layers fundamentally improve horizontal information flow across diverse architectures, enabling deeper reasoning and efficient scalability. Combined with synthetic benchmarks, they provide systematic insights into future opportunities in model design.

Future research. We plan to extend our study of Canon layers beyond the academic scale. Preliminary results from larger pretrains (1–8B models on 1–2T tokens) closely align with the findings reported here. Notably, several synthetic trends—such as Transformer+Canon strongly outperforming Transformer, GLA+Canon matching GDN and outperforming Mamba2—become *clearly observable at these larger scales*. Code is available on GitHub [2], some models on HuggingFace, and all resources are linked at `physics.allen-zhu.com`.

2 Synthetic Tasks for Decomposing Intelligence

We design synthetic tasks to systematically evaluate specific capabilities of language model architectures under controlled conditions, minimizing confounds and enabling clean comparisons. Task selection is guided by four criteria:

Criterion 1: Tasks must not be shallow. Shallow tasks—like associative recall or copying—are easily solvable by small and shallow models, and do not meaningfully test architectural strength. Deep learning relies on stacked layers to progressively learn abstract features [4], so tasks involving hierarchical reasoning better evaluate architectural scalability and efficiency.

Criterion 2: Emphasis on mental thinking. Tasks should assess a model's ability to reason internally without Chain-of-Thought (CoT). While CoT helps decompose problems, it does not reflect intrinsic "system 1" reasoning [77]. For example, a model reasoning 4 steps internally and 8 via CoT achieves 32 steps, but *only internal ones reflect architectural strength*. Current models like o3/R1 produce verbose reasoning traces even for trivial prompts (e.g., "Hello")—revealing inefficiencies in system 1. To guide architectural progress, tasks must target mental reasoning.

Criterion 3: Avoid emphasis on length generalization. Length generalization is often unstable—sensitive to random seeds and training order [82]—and thus unreliable for comparing architectures. While length generalization is important, models over-optimized for long contexts (e.g., 100k tokens) may exhibit reduced performance on standard lengths like 4096 tokens.[2] In practice, long inputs are typically summarized into shorter windows before reasoning, so we prioritize evaluating architectures on dense, 4096-token contexts, where critical reasoning unfolds.

Criterion 4: Relevance to real-world skills. Tasks should prioritize broadly applicable skills while avoiding capabilities better suited to external tools. For example, large-number arithmetic

[2]This is observed in methods like ALiBi [45], Halibi [31], and Mimetic initialization [66], whose performance degrades on shorter contexts, as we show in this paper.

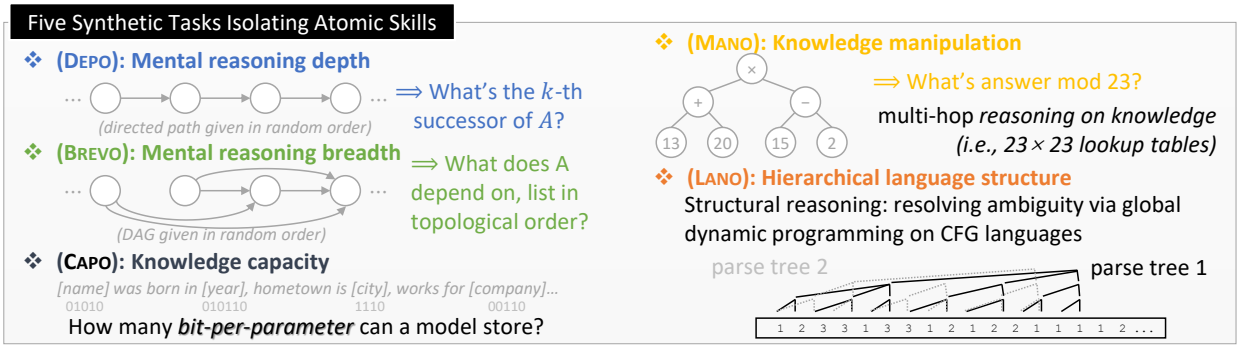


Figure 3: Overview of our five synthetic tasks, each isolating an atomic skill for rigorous architectural comparison.

(e.g., adding 10-digit numbers) is theoretically interesting but can be delegated to Python interpreters; failures in this area typically reflect limited data exposure rather than architectural weaknesses (e.g., Llama3-70B miscalculates 452352 + 547647). Synthetic tasks should focus on universally relevant skills, aligned with real-world applications, to ensure meaningful assessments.

2.1 Our First Set of Five Synthetic Pretrain Tasks

To operationalize the criteria above, we design five synthetic tasks—each targeting a distinct dimension of language model capability. We name them DEPO, BREO, CAPO, MANO, and LANO.

Task Depo: Mental reasoning depth. Reasoning depth represents a fundamental capability for LLMs, requiring models to retrieve information through multi-step computation. Task DEPO evaluates reasoning depth as k-hop traversal over directed permutations, where models compute the k-th successor for each query q entirely internally, without intermediate steps like Chain-of-Thought (CoT).[3] Each instance is formatted as:

```
<bos> x1 y1 x2 y2 ... xn yn <query_k1> q1 a1 <query_k2> q2 a2 ... <eos>
```

Here, $2n$ tokens encode n directed edges $x_i \to y_i$, forming a random permutation of n nodes.

The dataset is controlled by two parameters: N, the maximum permutation size, and K, the maximum reasoning depth. During training, n is sampled from $[3, N]$, while $k \in [1, K]$. Context lengths are fixed to 2048 tokens. We employ two variants of DEPO:

- DEPO1: Each node spans 1–2 tokens from vocab size 50, with $N = 225$, 300, 375 and $K = 8$.
- DEPO2: Each node spans 5–7 tokens from vocab size 4, with $N = 75$, 100, 125 and $K = 16$.

Evaluation focuses on both the hardest cases ($n = N, k = K$) and intermediate difficulty ($k = K/2$). For weaker models, we utilize *reduced* training setups with $K = 4$, denoted DEPO1($K = 4$) and DEPO2($K = 4$). The full methodological details are provided in Appendix A.1.

Task Brevo: Mental reasoning breadth. This evaluates a model's ability to process multiple dependencies simultaneously, as required in tasks involving tree-like traversal or dependency graphs. For example, solving queries like "Who are Alice's nephews?" or GSM-like examples requires parallel reasoning across branches of a graph to process relationships bottom-up [75]. Task BREVO isolates this capability using recursive traversal of directed acyclic graphs (DAGs), abstracting away natural language or arithmetic complexities. Each task instance is formatted as:

```
<bos> x1 y1 x2 y2 ... xm ym <query> q <ans> a1 a2 ... ap <eos>
```

Here, $2m$ tokens define m edges $x_i \to y_i$, representing dependencies where y_i depends on x_i. Upon receiving a query vertex q, the model outputs all vertices recursively reachable from q, sorted in topological order starting from the leaves (e.g., $u \to v \to q$ yields output u followed by v).

[3] Using CoT would reduce the k-hop task to simpler 1-hop associative recall.

The dataset is parameterized by N, the maximum graph size, with DAGs created using $n \leq N$ nodes, each of degree at most 4. Pretraining data is sampled by varying graph sizes, while testing focuses on the hardest graphs ($n = N$). We employ two variants of BREVO:

- BREVO1: Each vertex name spans a single token, with $N = 70/90/110$, fit within 1024 tokens.
- BREVO2: Name spans 2–4 tokens of vocab size 4, with $N = 30/40/50$, fit within 1536 tokens.

A key discovery from [75] revealed that, due to the non-uniqueness of valid outputs, language models must preprocess the entire topological order of the DAG *mentally* before generating the first token a_1. This insight confirms that our synthetic data rigorously evaluates reasoning breadth by requiring models to globally process the underlying graph structure before producing outputs.

Task Capo: Knowledge capacity. Task CAPO evaluates a model's efficiency in encoding factual knowledge directly within its parameters, quantified as *bits per parameter*, which measures reliable storage capacity. Following the framework in [8], synthetic datasets of (fake) biographies are constructed to test knowledge retention. Each biography includes several attributes (e.g., birthdate, university, employer, etc.) and is presented in diverse paraphrased formats to reduce surface-level memorization [5, 7]. Capacity is measured using the next-token prediction distribution, accounting for both exact correctness and partial accuracy.

To highlight architectural differences, we adopt an undertrained regime where each biography is exposed only 100 times during pretraining.[4] The dataset includes $N = 50$K to 2M biographies, encoding 2×10^6 to 10^8 total bits of information. Models of varying sizes are tested, and results are visualized via "bit vs. model size" plots. Additional details are provided in Appendix A.3.

Task Mano: Knowledge manipulation. Task MANO evaluates a distinct form of reasoning: the ability to manipulate stored knowledge internally, contrasting with in-context reasoning tasks like DEPO or BREVO. While those tasks focus on reasoning over external tokens, MANO requires models to retrieve factual knowledge embedded in their parameters and perform hierarchical computation entirely mentally. This combination of retrieval and reasoning makes knowledge manipulation uniquely challenging and a skill that must be learned during pretraining.[5]

To test this capability, MANO employs synthetic modular arithmetic expressions inspired by human mental computation, particularly small-number arithmetic like the 9×9 multiplication table. Models solve multi-step arithmetic problems without intermediate steps like Chain-of-Thought. For example, given: `<bos> + * a b - c d <ans>` the task requires evaluating $((a \times b) + (c - d)) \bmod 23$ for $\ell = 3$, where operands a, b, c, d are sampled uniformly from $[0, 22]$. Modular arithmetic provides the foundational factual knowledge (23×23 operation tables), while the task challenges hierarchical reasoning by recursively composing operations. Additional details are provided in Appendix A.4.

The dataset is parameterized by a maximum expression length L, with ℓ sampled uniformly from $[1, L]$. We prepare three MANO datasets across difficulty levels: $L = 10$, 13, and 16.

Task Lano: Hierarchical language structure. Task LANO evaluates structural reasoning over hierarchical relationships and long-range dependencies. Unlike DEPO, BREVO, and MANO, which rely on explicit key-value pairs (in-context or knowledge), LANO challenges models to infer implicit recursive structures across sequences and resolve global ambiguities within them.

To test this, LANO leverages synthetic datasets built from context-free grammars (CFGs). Training sequences consist of CFG-valid sentences separated by `<bos>` tokens. For example:

[4] Exposing each biography 1000 times during pretraining diminishes architectural differences, as even transformers without MLP layers can achieve similar storage efficiency [8]. Uniform exposure ensures clean systematic comparisons while avoiding confounding effects tied to rare outliers and junk data [8].

[5] For instance, questions like "Was [name] born in an even or odd month?" or derived 2-hop queries such as "What is [name]'s sister's birthdate?" demand reasoning layers over stored knowledge. These skills cannot reliably emerge through supervised fine-tuning alone [7] and require development during pretraining or continued pretraining.

<bos> 3 3 2 2 1 ... 3 3 1 2 <bos> 1 2 3 3 1 ... 1 2 2 1 <bos> ...

CFGs are designed with token-level ambiguity, where local tokens (e.g., 1, 2, 3) provide insufficient information to directly infer their mapping to CFG rules. Resolving this requires dynamic programming to globally map the entire sequence to a valid recursive application of CFG rules, which must also be learned during training. This reasoning grows in worst-case complexity ($O(n^3)$) as sequence lengths increase. Details are in Appendix A.5.

Building upon cfg3f [6], which includes sequences of lengths 100–500, we introduce extended datasets cfg3j and cfg3k, with sequences ranging up to 200–1000 tokens to increase recursive depth and test models on more nested rules and longer dependencies. Training uses context lengths of 1536 for cfg3j and cfg3k, compared to 512 for cfg3f. Evaluation prompts models with <bos> to generate CFG-valid sentences, validated via a dynamic programming parser. KL divergence is also used to compare token distributions against ground truth.

In summary. This set of five synthetic tasks covers non-overlapping skills and distinct aspects of accuracy—token-level (Depo, Mano), generative (Brevo, Lano), and distributional (Capo, Lano). While this pool can be further enriched, it serves as a strong starting point for deriving meaningful architectural insights, as demonstrated in the following sections.

3 Initial Comparison on Well-Known Base Architectures

Language model architectures have evolved significantly since Transformers [67], giving rise to three major families distinguished by their computational mechanisms.

Quadratic-time attention models include BERT [36] and GPT-2 [48]. Refinements such as Rotary Position Embeddings (RoPE) [13, 61] and gated MLPs [56] define their modern variants. We use the Huggingface implementation of Llama, denoted **Llama(RoPE)**, which includes both refinements, and **Llama(NoPE)**, which omits positional embeddings. When clear, we refer to them as RoPE and NoPE. Relative positional embeddings (e.g., [28]) are omitted due to limited empirical benefit but added computational cost [6].

RoPE models often generalize poorly beyond training context lengths, whereas NoPE generalizes better but achieves lower overall performance. Recent attention-score variants such as **ALiBi** [45] and **Hard-ALiBi** [31] partially mitigate this, and we shall investigate closely in this paper.

Linear-time attention reduces computation by compressing sequences into fixed-length representations. Notable architectures include Linformer [68], Performer [15], Linear Transformer [35]. We focus on more recent Gated Linear Attention (**GLA**) [72] for its efficiency and scalability.

Recurrent and state-space models (SSM) process long sequences via evolving hidden states rather than full attention. Mamba [19, 26] exemplifies this family; we study its 2nd generation (**Mamba2**). Another key model is Gated DeltaNet (**GDN**) [73], which we also analyze. Other notable variants include S4 [58], S5 [58], RetNet [62], RWKV [43], HGRN [46], GSA [80], and DeltaNet [74].

Exclusion of hybrid architectures. We omit hybrid models integrating attention with linear or state-space mechanisms—e.g., Griffin [20], Samba [50], GDN-H1/H2 [73], or sliding-window attention—to preserve clarity. Although such hybrids may excel on long contexts (up to 1M tokens), our focus is precision within standard windows (e.g., 4096 tokens). In practice, long contexts are often compressed (e.g., via CoTs) for detailed reasoning, making local precision the key concern.

Hybrids can *obscure architectural trade-offs*, as aggregated results blur the contributions of individual modules. For instance, Mamba2 performs well on memory tasks but underperforms on structured reasoning; hybridization may conceal such contrasts. To ensure transparency, we analyze isolated *base* architectures to reveal their intrinsic strengths and weaknesses.

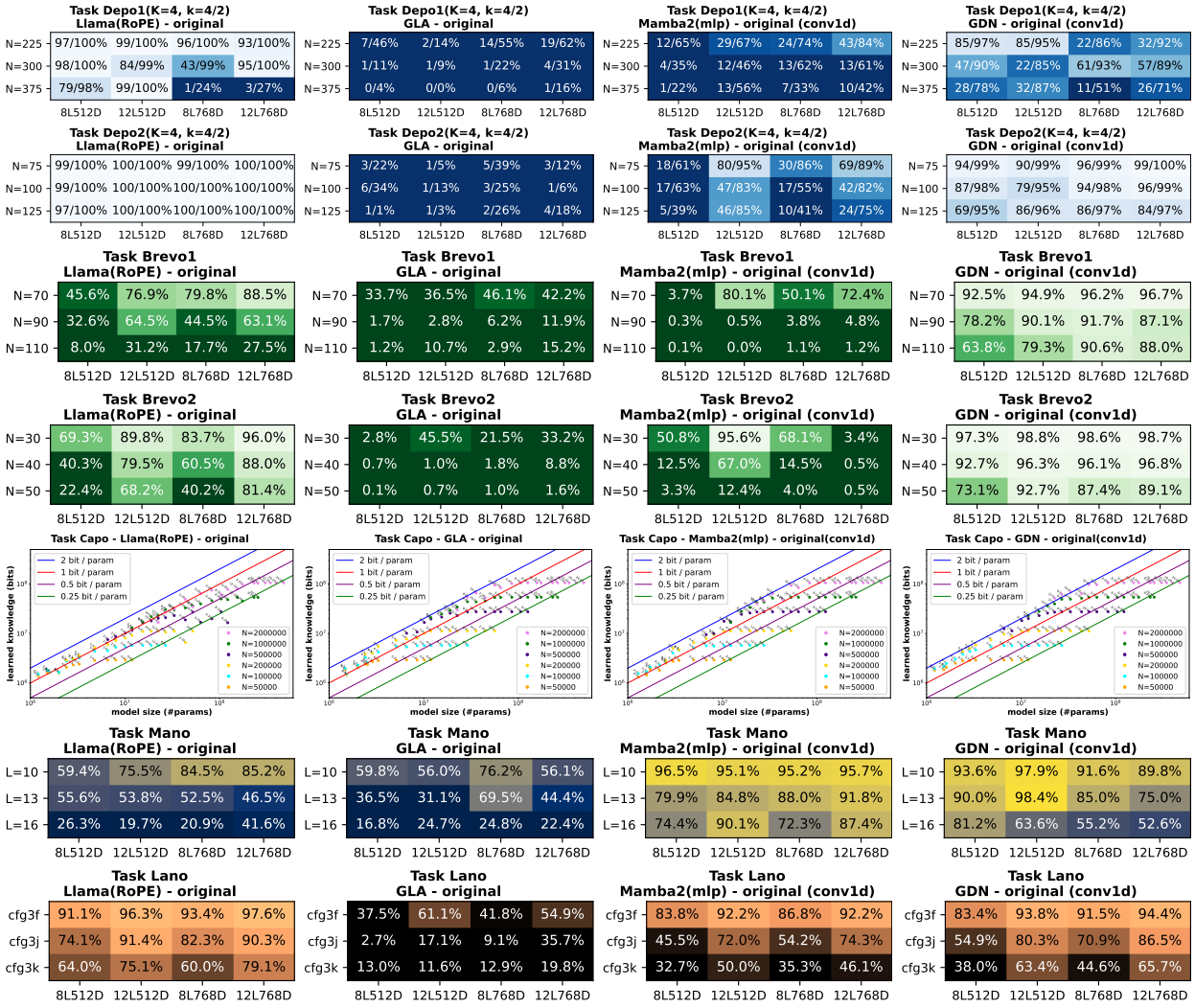


Figure 4: **Initial comparison of base models on five synthetic tasks.** GLA performs weakest; Mamba2(mlp) excels in knowledge (Capo, Mano); GDN strengthens reasoning and surpasses Llama(RoPE) on Brevo (reasoning breadth), while RoPE remains best on Depo+Lano (depth and structural reasoning). These results confirm our synthetic playground **as effective** for architectural comparison, but adding Canon layers (see later) will build a "Pisa tower"—enabling controlled, fair comparisons where the landscape **shifts drastically** and reasoning depth improves 2–4×.

Notably, Falcon-H1 [63] (May 2025, 32B) combines Mamba2 with full attention, while Qwen3-Next [47] (Sep 2025, 80B) combines GDN with full attention. These recent hybrids validate our choice of Mamba2 and GDN as representative base linear models.

Architecture size standardization. To ensure fair comparison, we standardize model sizes and evaluate Llama, GLA, Mamba2, and GDN as representatives of their respective families.

For all tasks except Capo, we test four sizes: Llama models with 12 or 8 layers and hidden dimensions of 768 or 512 (12 or 8 heads), denoted `12L768D`, `12L512D`, `8L768D`, and `8L512D`. (`12L768D` matches GPT-2-small.) These configurations are *translated* to GLA, Mamba2(mlp), and GDN for comparable parameter counts.[6]

For Capo (bit-per-parameter knowledge capacity), we vary model and data scales more broadly.

[6]See Appendix C for details. Briefly, with hidden size d, GLA follows the $4d^2 + 8d^2$ design (linear attention $4d^2$, MLP $8d^2$), while Mamba2(mlp) and GDN use $6d^2 + 6d^2$. We also test Mamba2 without MLP, reported separately in the appendix and referred to as Mamba2.

Following [8], model size is denoted ℓ-h: for Llama, ℓ layers, hidden size $64h$, and h heads. This notation extends consistently to GLA, Mamba2, and GDN (see Appendix C).

Training. All architectures share identical training settings (batch size, steps, learning rate, etc.) to ensure fairness. Full details appear in Appendix A. Random seeds are fixed so that all models pre-train on identical data sequences.

3.1 Initial Comparison Results

From Figure 4, linear-attention GLA performs weakest overall. Mamba2 excels on knowledge tasks (CAPO, MANO) but lags in reasoning. GDN improves Mamba2's reasoning and occasionally surpasses Llama(RoPE) on certain reasoning tasks (e.g., BREVO), though not others. These patterns align with real-world observations on natural data, supporting the validity of our synthetic playground. We defer deeper interpretation, as both Llama and GLA later prove to lack a critical architectural component—**making this comparison incomplete and partially unfair**.

For now, we highlight several *key remarks*.

3×4 mini scaling laws. Randomness can affect outcomes, especially on hard tasks where *grokking emerges*. In MANO, even with two seeds and four learning rates, smaller models sometimes outperform larger ones. This reflects staged reasoning: a model must learn k-hop reasoning (e.g., `Mano`, `Depo`) before advancing to $k+1$, and the transition often depends on random training dynamics. To reduce such variance, we test all tasks across *three* data scales and *four* model sizes (more for CAPO). These "3×4" mini scaling laws yield more stable and interpretable comparisons.

Benefits of synthetic tasks. Synthetic tasks clarify architectural differences starkly (e.g., 90% vs 5%), clearly exposing strengths and weaknesses. By contrast, real-world experiments often produce modest differences (e.g., 2%) buried in noise. Thus, synthetic pretraining environments allow clean evaluations of architectures' scalability and true capabilities.

Interpreting task failures. If a specific architecture (of a given size) fails at a certain difficulty level (e.g., large N or k), it does not imply the model cannot learn the skill given infinite training. Our comparison uses a fixed, limited training budget: all architectures train for the same number of steps with identical data and shuffling, reporting best accuracy across multiple learning rates. Thus, results should be seen as differences in the *speed of skill acquisition*, not absolute capability.[7]

Predicting future pipelines. Synthetic tasks simulate idealized, high-quality pretraining conditions targeting core skills like multi-hop reasoning (DEPO). Unlike datasets such as FineWeb-edu or SlimPajama, which contain sparse reasoning examples obscured by simpler content, synthetic tasks highlight core capabilities. Currently, 100B-token pretraining fails even simplest 2-hop reasoning (Result 12). As training pipelines evolve—via improved data curation or RL-based post-training—synthetic tasks like DEPO may better predict models' potential and guide architectural choices.

4 Canon Layers: Enhancing Horizontal Information Flow

Attention-based Transformers are widely recognized for their ability to perform associative recall—e.g., predicting `?` in the sequence `[A] [B] ... [A] [?]` where `?` = `[B]`. One might expect the second `[A]` could simply attend to the first to retrieve `[B]`, but causal masking makes this impossible: the first occurrence of `[A]` sees no future tokens. Accurate recall thus "requires" two

[7] Faster learning is practically important—for example, a model ideally learns reasoning skills quicker than pure memorization. Similar observations arise in knowledge capacity tasks [8], where architectural differences vanish with ample training but become pronounced when training budgets are limited.

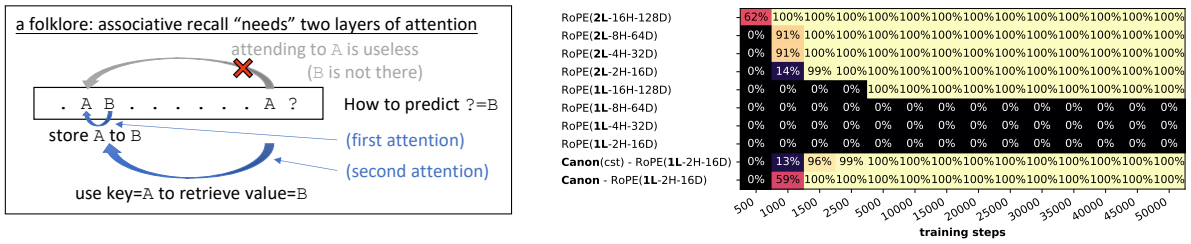


Figure 5: **A trivial token-copying experiment for 500 tokens**, added for completeness. 1-layer RoPE requires $d \geq 128$, while 2-layer RoPE or 1-layer RoPE + Canon achieves 100% with $d = 16$.

attention layers—the first copies the first `[A]` into its neighbor `[B]`; the second uses this enriched representation, querying by `[A]` to retrieve value = `[B]` (via key = `[A]`). Using global attention just to pass information between adjacent tokens is, in effect, *shooting a bird with a cannon.*

Remark 4.1. This is not a strict lower bound. A 1-layer Transformer is Turing-complete and can perform recall by blindly aggregating most (or all) context into one position, allowing MLP to do local query/key/value computations. But this is inefficient: Figure 5 shows that 1-layer Transformer needs hidden size 128 to recall length-500 sequences, while 2 layers succeed with size 16.

The importance of local context. Even simple tasks like token recall require careful mixing of local context—*not to say* more complex ones or when words span multiple tokens. Since MLP layers don't mix tokens, attention must handle all communication. Rotary and relative positional encodings help by biasing attention toward nearby tokens, but they remain tied to attention and still "shoot birds with cannons." Similar issues arise in GLA [72] and Mamba2, where recent-token information must be retrieved via compression mechanisms not optimized for local detail.

Canon layers: general form. Inspired by (vertical) residual connections, we introduce *Canon layers* to enhance horizontal information flow across neighboring tokens. Canon layers aggregate nearby hidden states into the current position, enabling lightweight local mixing within a fixed window (e.g., size 4), unlike attention-based global aggregation or recurrent compression.

Formally, for any hidden states $h_t \in \mathbb{R}^m$ at token position t, a Canon layer computes:

$$h'_t = w_0 \odot h_t + w_1 \odot h_{t-1} + w_2 \odot h_{t-2} + w_3 \odot h_{t-3},$$

where $\odot$ denotes element-wise multiplication, $w_i \in \mathbb{R}^m$ $(i = 0, 1, 2, 3)$ are weights, and padding zeros are used for boundary conditions. We call this *Canon*, borrowing from the musical term, as it resembles melodies played sequentially at fixed temporal delays.[8]

Flexible Integration. Canon layers integrate at multiple points within each Transformer block:

- *Canon-A*: Before the attention block ($m = d$ if hidden size is d), after RMSnorm.
- *Canon-B*: Inside the attention block, applied after Q/K/V projections ($m = 3d$).
- *Canon-C*: Before the MLP block ($m = d$), after RMSnorm.
- *Canon-D*: Within MLP ($m = 4d$ for standard, $m = \frac{16}{3}d$ for gated MLP), before activation.

Combining all four points gives *Canon-ABCD* (full-score Canon); partial combinations (Canon-A/B/ABC) can also be explored. Canon layers integrate flexibly across diverse architectures, including linear-attention and state-space models. For Mamba2 (without standard MLP layers), Canon layers appear at Canon-A and Canon-B positions (yielding Canon-AB); for Mamba2(mlp), the complete Canon-ABCD applies. Canon-B in Mamba2 scales as $m = 4d + o(d)$.[9]

[8] In Pachelbel's Canon in D, violins sequentially play the same melody with delays, creating overlapping horizontal repetition patterns analogous to Canon layers.

[9] For example, Mamba2 settings with `ssm_state_size=64`, `num_heads=16` result in $m = 4d + 144$ dimensions.

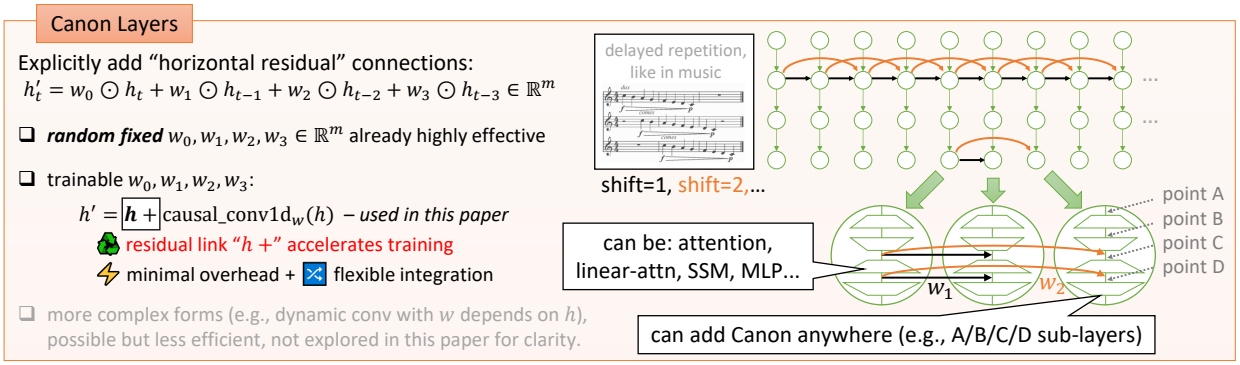


Figure 6: **Illustration of Canon layers**.

Canon layers: Implementation variants. Canon layers can be implemented in many ways. Even a simple version with **fixed, random weights**—aggregating past three tokens as *horizontal residual links*—**already notably enhances performance** (Figure 24 on Page 46).[10] More complex variants—e.g., dynamic convolutions with input-dependent weighting—are possible but not studied here, as it remains unclear whether such additional cost is justified.

In this paper, for simplicity and efficiency, we implement Canon layers as 1-d causal convolution with kernel size 4, available through efficient CUDA kernels implemented by the open-source H3 library (pip package `causal_conv1d`) [23]. We also incorporate explicit residual connections:

$$h'_t = h_t + \text{conv1d}\big([h_t, h_{t-1}, h_{t-2}, h_{t-3}]\big) \ , \tag{4.1}$$

denoted as Canon(res). Without residual connections, we denote it Canon(no-res). Minimal code changes (just a few lines) are needed for integration. Even fully enabled (Canon-ABCD), Canon layers increase the parameter count minimally.[11] Our emphasis is on clearly demonstrating Canon layers' substantial performance benefits; detailed runtime optimizations remain future work.

Related Work. A precursor to Canon layers appears in [6], which studied uniform attention—i.e., averaging the past k tokens—for $k \in \{1, 2, 4, 8, \dots\}$ on CFG tasks. Surprisingly, this simple mixing outperformed GPT2 with absolute positional embeddings and closely approached GPT2(RoPE).[12] Canon layers generalize this idea: we apply learned, position-specific mixing over a short window (typically 4 tokens), removing value and projection matrices for better efficiency and modularity.

Our use of `causal_conv1d` is inspired by Mamba [19, 26] and GLA [72], which trace back to H3 [23], where the component was introduced as "shift-SSM." After the initial release of our paper, we also became aware of Primer [59], which proposes "multi-dconv-head" attention. These models apply `conv1d` (often with SiLU activation) within SSM or attention modules, without residual connections. In our terminology, these roughly correspond to Canon-B(no-res).

Our work generalizes and isolates this design as the Canon layer, and systematically evaluates its effect across all types of sequential models and sublayers (A/B/C/D). By studying Canon under *controlled* synthetic pretraining, we can clearly attribute performance gains to the conv1d-

[10] Unlike vertical residual links ($h' = h + \sigma(\mathbf{W}h)$), Canon layers aggregate multiple token vectors from different relative positions ($t-1$, $t-2$, $t-3$). Assigning fixed orthogonal directions effectively provides each position a unique "ID" for aggregation. Simple scalar weighting (e.g., $h'_t = h_t + 0.4h_{t-1} + 0.2h_{t-2} + 0.1h_{t-3}$) can degrade performance.

[11] Fewer than 0.45% parameters for GPT2-small. For a 1.3B-parameter Llama with Canon-ABCD enabled, parameters increase by 0.0063%, runtime overhead on an H100 GPU with naive implementation (PyTorch bf16, flash attention, causal conv1d kernels) is 12.4%, 14.1%, and 20.8% for forward, backward, and generation respectively. For Canon-AC, overheads reduce to 5.8%, 5.8%, and 7.0%. Further runtime efficiencies are possible (e.g., consolidating multiple Canon operations across layers), though these optimizations remain beyond this paper's scope.

[12] One ICML reviewer rejected the paper, commenting that the results were "too surprising to be true." We invite curious readers to try it themselves—it really works.

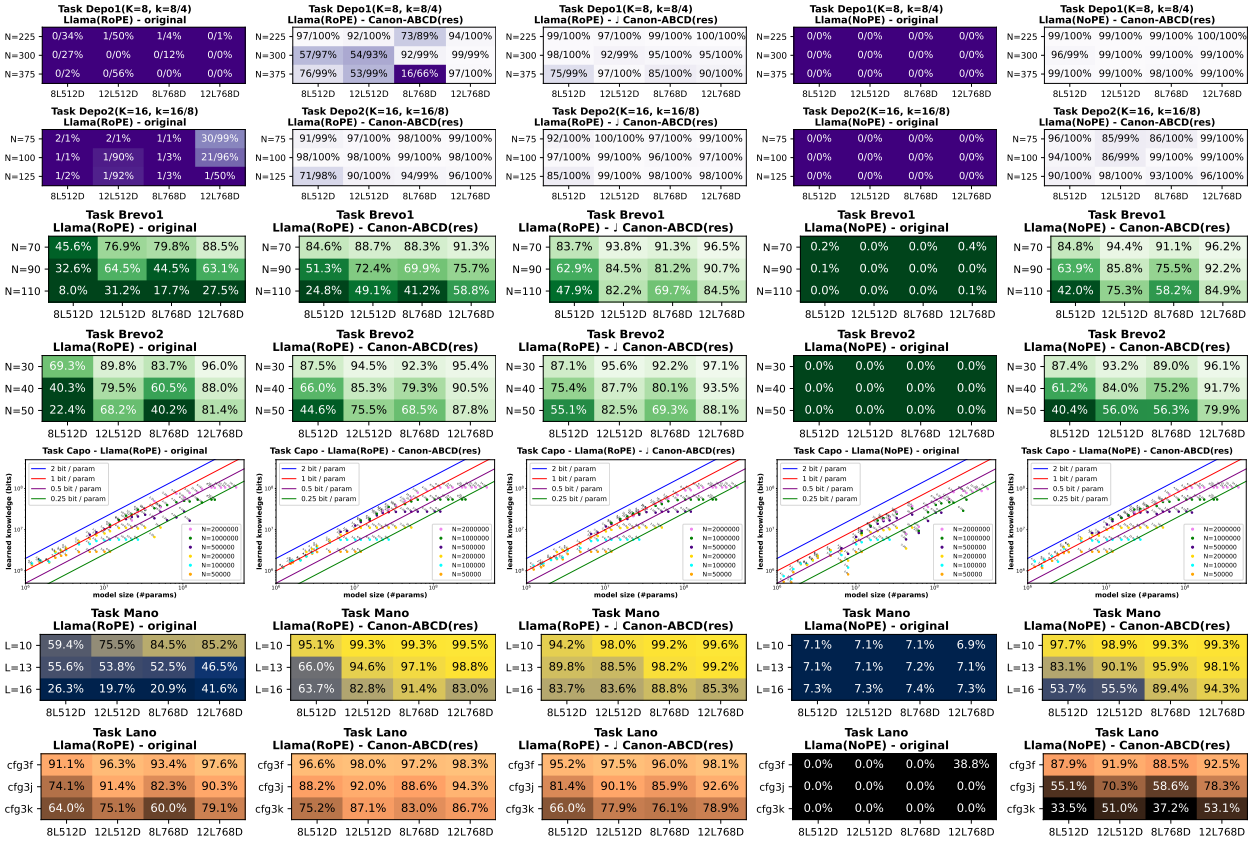


Figure 7: **Column 1→2**: Canon layers dramatically enhance RoPE, improving reasoning depth by 2–4×.
Column 4→5: Canon transforms NoPE into a strong performer on par with RoPE-based models.
Column 2+5→3: With Canon, RoPE usage can be reduced — RoPE + ♩Canon (RoPE enabled for 1/4 dimensions) outperforms both RoPE/NoPE + Canon, **great news for length generalization!**

Remark. This figure uses DEPO1(K=8) and DEPO2(K=16). Earlier results in Figure 4 were based on DEPO1(K=4) and DEPO2(K=4), because model performances were weaker.

based mixing mechanism, rather than to other architectural components such as attention or state-space recurrence. Moreover, we show that Canon layers are intrinsically *not tied to attention or SSMs*—and in fact, may not benefit from being tightly coupled to them.

Convolutions have been used in Transformers for different goals. Conformer [27] and CvT [70] integrate heavier convolutional modules for feature extraction in speech and vision. In contrast, Canon layers are lightweight and designed to enhance horizontal information flow—like horizontal "residual links." Notably, even random-weight Canon layers yield substantial improvements.

Concurrent work on Multi-Token Attention (MTA) [25] explores more complex 2D convolutional layers within attention heads. While MTA improves associative recall, it is heavier and more attention-specific. Investigating whether such designs offer further gains when combined with Canon, or whether Canon alone suffices for most settings, is an interesting direction for future work.

5 When Transformer Meets Canon

Figure 4+7 show that a 12-layer, 768-dimension Llama(RoPE) model trained on our ideal data can only handle 4-hop retrieval in contexts of length 2048. Can this be any better?

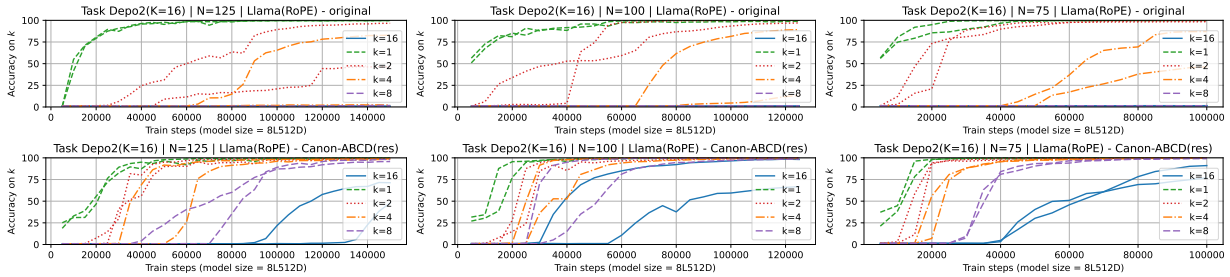


Figure 8: **Training curves for RoPE models w/+w/o Canon**, on DEPO2($K = 16$), evaluated at $k = 1, 2, 4, 8, 16$ and maximum size $n = N$, shown in two best LRs. More model sizes/data are in Figure 19 on Page 41.

5.1 RoPE with Canon Layers

Result 2 (Figure 7 — 1st vs. 2nd column)**.** *In our controlled playground, Canon layers (ABCD) introduce substantial improvements: with a 0.5% increase in trainable parameters, reasoning depth of RoPE increases by 2-4×, reasoning breadth by 30%, knowledge capacity by 10–15%, knowledge manipulation length by 30%, measurable gains in hierarchical language structure reasoning.*

Task Depo. In reasoning depth, RoPE pretrained on DEPO1($K = 8$)—covering ($k \leq 8$)–hop instances—achieves near-zero accuracy even at $k = 4$, whereas RoPE+Canon-ABCD exceeds 50% at $k = 8$. On DEPO2($K = 16$)—a more challenging setup where each directed edge spans 10–14 tokens, far beyond a 4-token Canon window—RoPE completely fails, while RoPE+Canon-ABCD attains near-perfect accuracy at $k = 16$. This demonstrates that Canon layers are not merely for single-token recall: by enriching local representations of multi-token segments, they empower the global attention to more effectively chain information across hops.[13]

These gains may seem surprising. For associative recall (analogous to DEPO1 with $k = 1$), theory suggests a single Canon + attention layer suffices (recall Figure 5), suggesting Canon could reduce required attention layers by at most one. So, why a 2–4× increase in reasoning depth?

The answer lies in learning dynamics. Deep reasoning tasks like DEPO unfold through a *hierarchical learning* process—models first master 1-hop, then gradually progress to 2-hop, 3-hop, and beyond. This process relies heavily on two factors: (1) training data spanning a range of difficulty levels and (2) architectural support like residual connections. Without either—e.g., training only with $k = 8$ data *or* removing residuals—the model can fails entirely.[14]

Thus, architectures that enable faster mastery of 1- and 2-hop reasoning climb the hierarchy faster, as illustrated in Figure 8. RoPE + Canon-ABCD achieves deeper reasoning progression much faster than vanilla RoPE, leveraging the inherent easy-to-hard structure of multi-hop tasks. We emphasize again that this is not about performance under infinite training data—RoPE could eventually achieve similar accuracy on DEPO2($K = 16$). However, RoPE + Canon achieves comparable results with significantly fewer training steps, making it far more efficient.

Task Brevo. On reasoning breadth, we observe 30% improvement by introducing Canon-ABCD. Specifically, the accuracies of RoPE to solve BREVO1($N = 70$) or BREVO2($N = 30$) resemble the performance of RoPE+Canon to solve BREVO1($N = 90$) or BREVO2($N = 40$). Since input length scales with N, this reflects roughly 30% increase in reasoning breadth.

To understand the source of this improvement, we analyze the accuracy across tasks stratified

[13] DEPO2 is designed so a 4-token window cannot resolve key–value pairs spanning 10–14 tokens, posing a substantial challenge even for Canon.

[14] The first theory foundation for why deep learning can perform deep (hierarchical) learning was established by Allen-Zhu and Li [3] (in the 3-layer case) and Allen-Zhu and Li [4] (for $\omega(1)$-layer). They show that deep learning relies on easy-to-hard curricula and residual structures for progressively building complexity.

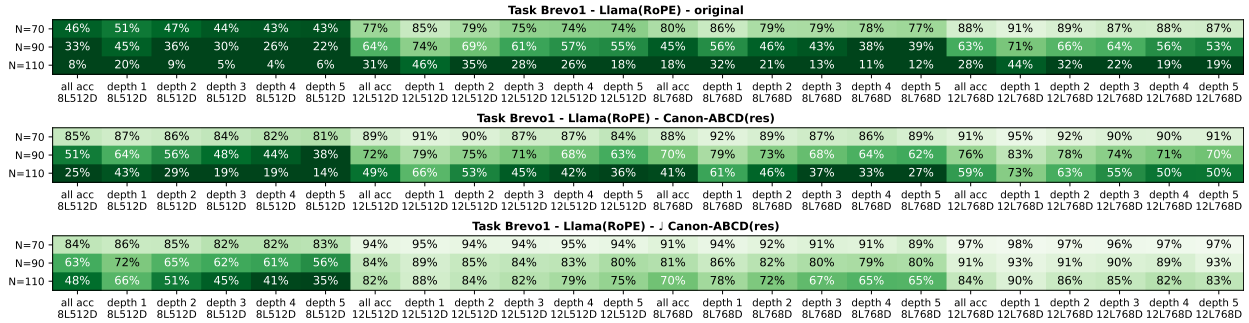


Figure 9: Detailed accuracies for Task BREVO1, shown overall and stratified by dependency graph depths $1, 2, 3, 4, 5$.

by *depth* of the dependency depth. Recall each query in BREVO requires the model to identify all vertices it recursively depends on, forming a sub-DAG of varying (minimum) depth. In Figure 9, we plot model accuracy not only overall but also separately for problem instaces spanning DAG depths of $1, 2, 3, 4, 5$. The results show that vanilla RoPE struggles with instances involving greater DAG depth, whereas RoPE+Canon improves reasoning performance on deeper structures. This suggests that Canon-ABCD enhances localized reasoning paths within Transformer blocks, allowing for better handling of recursive dependency, which can be challenging for standard attention alone.

Task Capo. On knowledge capacity, prior work [8] found that gated MLP layers in Llama(RoPE) reduce model capacity due to slower and less stable training dynamics. One remedy proposed in that work is to revert gated MLP back to standard MLP; however, this sacrifices reasoning capability (see Section 5.4). Here, we present an alternative solution: adding Canon layers. Canon layers improve training speed and increase the effective capacity by 10–15% in the controlled 100-exposure pretraining regime for CAPO. On a separate note, GPT2(RoPE) models that originally employ standard MLP exhibit no capacity loss after Canon layers are introduced (Figure 11).

Task Mano. On knowledge manipulation, Canon layers increase manipulable length. RoPE+Canon matches the performance of vanilla RoPE on Mano($L = 10$) when tested on Mano($L = 13$), a 30% improvement in length. This again stems from Canon layers accelerating hierarchical learning, enabling the model to scale from simpler tasks ($L = 1$) to more complex ones ($L = 2$, $L = 3$, and beyond) faster. For simplicity, we omit the hierarchical learning speed visualization.

Task Lano. Canon layers improve RoPE's performance on hierarchical language structure reasoning, though interpreting the gains requires some algorithmic background. For context, dataset cfg3k adds one level of structural complexity above cfg3f, using the same CFG rule distribution (see Appendix A.5). RoPE+Canon outperforms standard RoPE on cfg3k, but still struggles to fully handle this increased complexity. This is expected, as deeper CFG structures increase sequence length n by 2–3×, and parsing these CFGs with dynamic programming involves worst-case time complexity $O(n^3)$. Consequently, cfg3k poses arguably more than 8× greater computational challenge compared to cfg3f. Our intermediate dataset cfg3j has difficulty around 4×, suggesting RoPE+Canon can handle roughly twice as challenging structure-learning tasks comparing to RoPE.

Summary. Canon layers consistently improve performance across reasoning, knowledge and language tasks, all without introducing instability or accuracy trade-offs.

5.2 NoPE with Canon Layers

Result 3 (Figure 7+10[a]). *Canon layers transform NoPE. Key findings include:*

- *NoPE+Canon matches RoPE+Canon and even surpasses it on* Depo*; a remarkable result given that without Canon, NoPE achieves essentially zero performance on all measures.*
- *NoPE+Canon significantly outperforms existing fixes for NoPE, such as ALiBi and H-Alibi.*
- *With Canon layers, RoPE usage can be greatly reduced: RoPE on only 1/4 dims (denoted RoPE+♩Canon) outperforms both RoPE/NoPE+Canon, great news for length generalization.*

[a](Sub-results correspond to Figure 7 (4th vs 5th column), Figure 10, and Figure 7 (3rd column), respectively.)

Canon layers skyrocket NoPE performance. Canon layers dramatically improve NoPE (No Positional Embedding) transformers, lifting them from near-zero accuracy to competitive levels, even slightly surpassing RoPE+Canon on reasoning depth. NoPE-Canon is only weaker on Task Lano, which involves hierarchical structural learning over long sequences, thus relying more heavily on relative distance between input tokens; yet even there NoPE-Canon remains competitive with alternatives such as Mamba2/GDN.

Dominance over existing fixes on NoPE. While NoPE excels at length generalization, its performance on complex reasoning tasks has historically been weak. Fixes like ALiBi [45] and Hard-Alibi [31] partially address this: ALiBi applies a distance-based penalty to attention weights[15], while Hard-Alibi disables attention beyond distance h for the h-th head. Although these methods improve NoPE performance (partly mimicking RoPE), Canon layers clearly dominate. As shown in Figure 10 (top), NoPE+Canon significantly outperforms both alternatives.

Minimal RoPE usage with Canon layers. Canon layers eliminate the need for heavy RoPE usage, and excessive RoPE can even hurt performance. With Canons, minimal RoPE usage is sufficient—often preferable—for optimal results. For example, enabling RoPE on half of the heads at half of their dimensions (denoted ♩Canon) consistently outperforms full RoPE usage or NoPE, as shown in Figure 7 (3rd column). **This is great news for long-context generalization**: RoPE is a known bottleneck for Transformers with longer inputs. As Canon layers allow significantly reduced RoPE without performance loss, they become indispensable for length generalization tasks.[16]

Remark 5.1. Despite their versatility, Canon layers alone cannot fully resolve extremely challenging tasks that require deep hierarchical reasoning over long sequences (e.g., cfg3k in Task Lano). Such tasks, requiring $O(n^3)$ dynamic programming over 1000 tokens, remain computationally demanding. Nevertheless, Canon layers consistently offer huge improvements outside these specialized scenarios.

These findings translate to real-life. To be shown in Section 8, NoPE+Canon consistently matches or surpasses RoPE+Canon in real-world pretraining; the RoPE+♩Canon variants outperform RoPE+Canon on several reasoning tasks, particularly involving long-context inputs.

Remark 5.2. This paper focuses on architectural differences within computational stages after relevant information is retrieved into manageable contexts (e.g., 4096 tokens). Techniques like DeepSeek's NSA architecture [78], designed for retrieval and compression from extremely long inputs (e.g., 1M tokens), are complementary to Canon layers. Such techniques and Canon layers can thus jointly handle distinct processing phases in long-context models.

[15]Specifically, adding $-|j-i| \cdot 2^{-8h/H}$ to the logits of head h of H total heads.

[16]Alternative reduced-RoPE configurations explored in the appendix include: ♩♩ (1/4 of heads at full dimensions) and ♩♩♩ (all heads at 1/4 dimensions, as in GPT-NeoX [13]). Among these, ♩ and ♩♩♩ are comparable, slightly better than ♩♩ according to Figure 26 on Page 47.

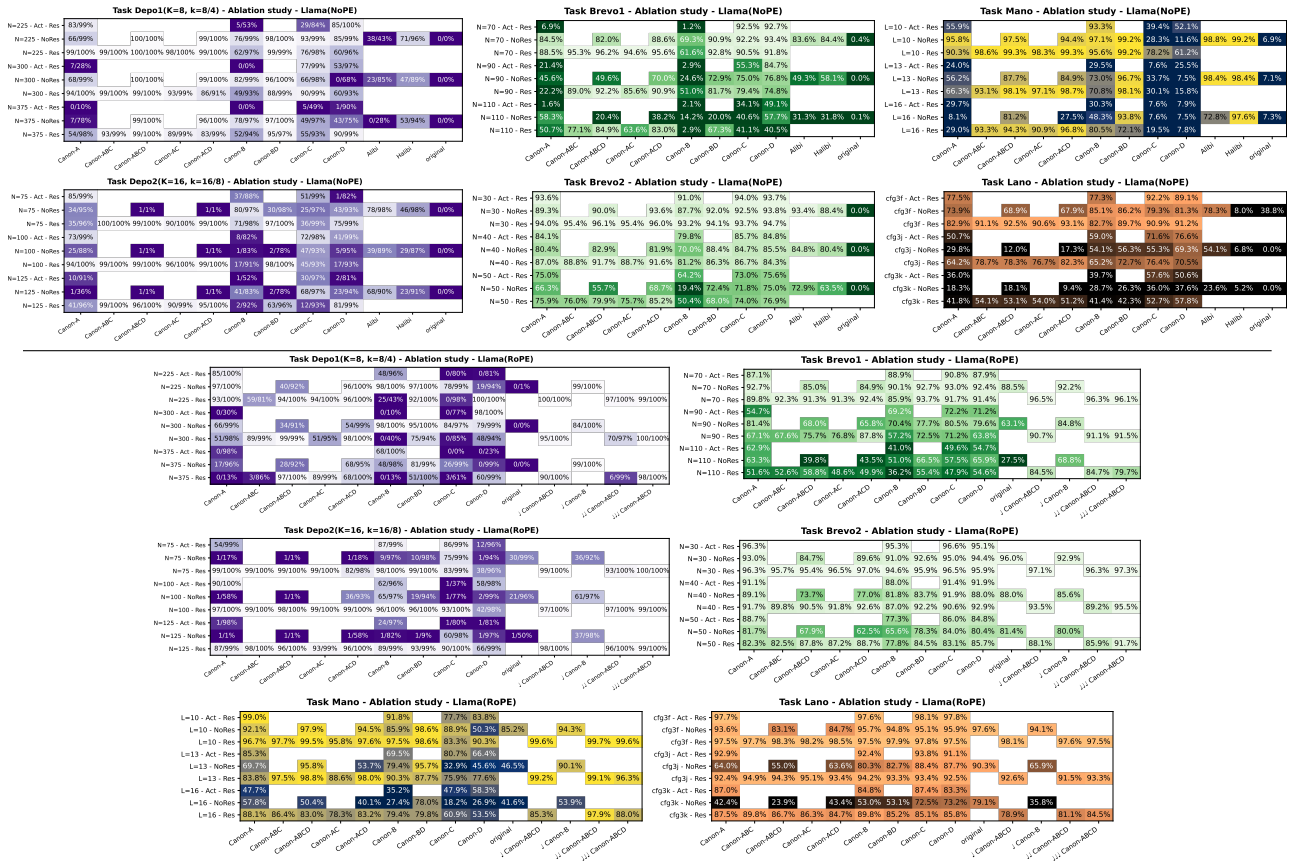


Figure 10: **Ablation study on 12-layer, 768-dim Transformers—NoPE (top) and RoPE (bottom)**—with Canon variants (**A–D**), **res**idual links, **act**ivation functions, **ALiBi**, and **H-Alibi**. Blank entries indicate untested configs due to resource limits. Additional ablation studies (with more model sizes) are in Figure 27 (RoPE), Figure 29 (NoPE), and Figure 28 (RoPE+Primer) in Appendix G.

5.3 Ablation Studies With Canon Layers

This section systematically investigates the design choices in Canon layers via ablation studies.

Component-level contributions. Each Canon component (A/B/C/D) contributes meaningfully to performance, with cumulative benefits from combinations. Adding even a single Canon layer yields notable gains, and stacking multiple Canon layers across sub-layers further amplifies these improvements, especially on weaker architectures like NoPE. Summaries appear in Figure 10 (for model size 12L768D) and additional size experiments in Appendix G.

Role of residual connections. Residual links around Canon layers — i.e., the "h_t+" part of (4.1) — are critical for training stability and effective learning, preserving vertical computational pathways and allowing global representations to selectively incorporate local context. Without residual connections, training becomes slower and less stable (see rows marked "NoRes" in Figure 10).

Independence of Attention/MLP. Prior works (e.g., the GLA [72] codebase and Primer [59]) focused solely on convolution operations within attention projections — Canon-B(no-res). However, we find that Canon-ACD alone already achieves substantial performance improvements, without modifying attention mechanisms. Similarly, Canon-ABC or even Canon-AC perform strongly without adjusting MLP layers. They all strongly outperform Canon-B(no-res) and thus outperform Primer. This highlights Canon layers' general role in enhancing horizontal information flow across architecture sub-layers, *independently complementing* attention or MLP mechanisms.

Nonlinear activations and computational simplicity. Contrary to prior works (e.g., H3/Mamba),

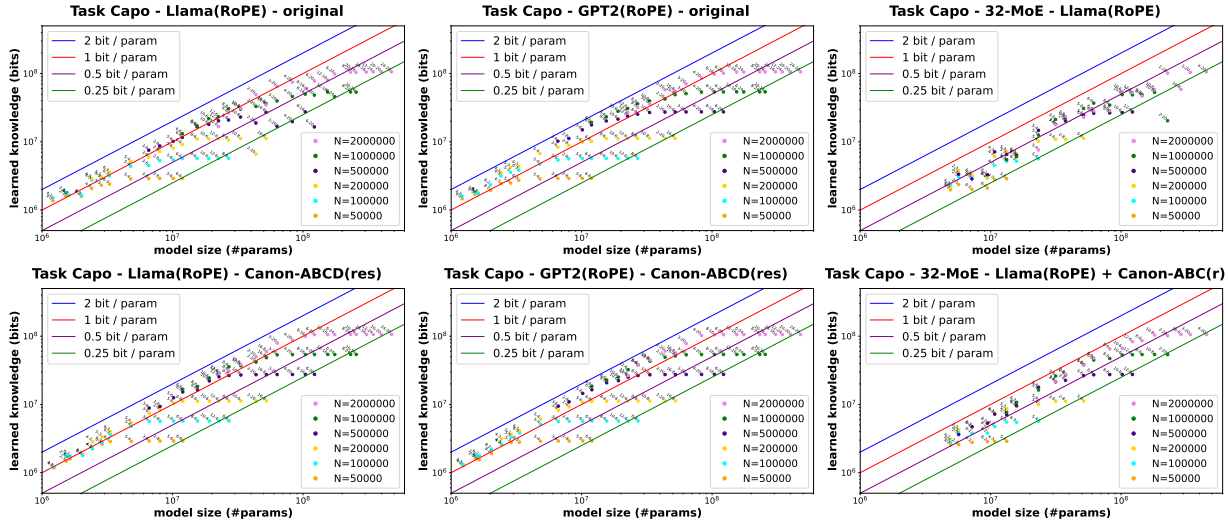


Figure 11: **Evaluation of knowledge capacity** (Capo) across architectures, measured as bits per parameter. The first row represents baseline models, while the second row shows improvements with Canon layers added. **Conclusion:** Canon layers enhance knowledge storage for architectures that are slower to train, such as gated MLP and MoE, mitigating the capacity gap between gated and standard MLP as identified in [8].

adding activation functions such as SiLU after the Canon layers does not yield noticeable benefits. Canon layers effectively inject local context directly into token positions, and nonlinear operations are sufficiently handled by the attention and MLP blocks (see rows marked "Act" in Figure 10).

Result 4 (Figure 10)**.** *Canon layers are lightweight, versatile, and effective enhancements that integrate seamlessly into Transformers. Key findings:*

- *Canon-A/B/C/D yield meaningful, cumulative improvements when stacked, and can be flexibly applied anywhere independent of attention or MLP modifications.*
- *Residual connections in Canon design are essential for stable, efficient training.*
- *Adding nonlinear activations (e.g., SiLU) provide no measurable benefit, simplifying design.*

(This **differs from prior works**: we show where to insert Canon layers, how to stabilize them, and why they matter.)

5.4 MLP and Mixture-of-Experts

Our synthetic playground provides a valuable framework to evaluate *broader architectural choices*.

Gated vs. standard MLPs. Gated MLPs [56], which replace standard MLP operations $V\sigma(Wx)$ by $V(\sigma(W_1x)\cdot(W_2x))$, improve expressiveness and parameter efficiency. Widely adopted by large-scale models (e.g., PaLM [16], Llama [64, 65], Mistral [33]), gated MLPs have become standard design choices. However, [8] found that gated MLP reduces knowledge capacity by about 30% in limited-exposure scenarios (e.g., 100-exposure Task Capo) due to slower convergence.

Thus, what is the best tradeoff? Our experiments (Figure 24 on Page 46) confirm gated MLP has slight advantage over standard MLP ("GPT2-style") on reasoning-heavy tasks, showing noticeable improvements on knowledge manipulation (Mano) and smaller gains on reasoning breadth (Brevo). Thus, replacing gated MLP with standard MLP may not be the best choice. However, keep in mind that adding Canon layers already partially mitigates gated MLP's capacity loss (recall Result 2), due to improving training dynamics and speed, recovering about half of its lost capacity.

Mixture-of-Experts. Mixture-of-Experts (MoE) [22, 57] enhances parameter efficiency by replacing dense MLPs with multiple parallel "experts," selectively routing tokens to fewer active

experts. While MoE achieves good scalability (particularly on knowledge capacity) and competitive inference-time performance, it suffers from significantly slower knowledge acquisition speed during training. For example, a 32-expert transformer may acquire $10\times$ less knowledge in the same 100-exposure regime (mimicking rare knowledge) compared to dense models (Figure 11). Could Canon layers mitigate this due to their improved training dynamics?

Integrating Canon layers with MoE, however, poses a challenge. Canon-D relies on neighboring tokens' hidden states, conflicting with MoE's independent token-wise expert dispatching. Adapting Canon-D to MoE would require complex engineering. To avoid such complexity, we test Canon-ABC layers alone, which already significantly accelerate MoE knowledge acquisition and improve bit-per-parameter efficiency (Figure 11), recovering at least half of the MoE-induced capacity loss.

MLP with Squared ReLU. The Primer [59] paper proposes using ReLU^2 as the activation function in standard MLPs, reporting improved performance over gated MLPs (e.g., SwiGLU) on real-world data. They also claim this gain exceeds that of Canon-B(no-res), which they refer to as "Multi-DConv-Head Attention." In our synthetic playground (see Figure 25 on Page 46), we confirm that ReLU^2 slightly improves standard MLPs (though not necessarily outperforming gated MLPs, consistent with recent findings [81]), while applying ReLU^2 to gated MLPs degrades performance. However, *these effects are* ***negligible*** *compared to the gains provided by Canon layers.*

Result 5 (Figure 11+24+25)**.** *Key insights for MLP and MoE architectures:*

- *Gated MLP slightly outperforms standard MLP (especially on* Mano*).*
- *Gated MLP reduces knowledge capacity (*Capo*); Canon layers partially recover this loss.*
- *$ReLU^2$ activation slightly improves standard MLP but degrades performance in gated MLP.*
- *Canon-ABC substantially improves MoE knowledge acquisition and bit-per-param capacity.*

6 When Linear Models Meet Canon

The three base linear models we study—GLA, Mamba2(mlp), and GDN—share a block-wise structure where each block consists of a "linear attention" layer (GLA, GDN, or Mamba2) followed by an MLP. This design naturally defines four insertion points for Canon layers, analogous to standard Transformers: **A** before the linear attention, **B** inside it, **C** before the MLP, and **D** inside. In the following subsections, we analyze each architecture separately.

6.1 When Linear Attention Meets Canon

Linear attention models reduce computation by maintaining a compact state instead of attending over all tokens. In Gated Linear Attention (GLA) [72], the attention map is updated recursively as $W_t = \alpha_t W_{t-1} + v_t k_t^\top$, where $W_t \in \mathbb{R}^{d_{\text{key}} \times d_{\text{value}}}$ remains fixed in size regardless of context length. This design is efficient but effectively averages over past tokens, weakening the influence of nearby ones—crucial for reasoning. Canon layers restore localized horizontal context flow, alleviating this limitation and improving reasoning fidelity.

Following the original GLA release, its authors added a `conv1d`-based enhancement in their GitHub repo—corresponding to our Canon-B variant but using SiLU activation and omitting residual connections. We refer to this as **GLA conv1d** or equivalently **GLA+Canon-b**. To show the strongest comparison, our **Canon-AbCD(res)** extends it by adding residual Canon-ACD layers while keeping their `conv1d`. We also explore the full Canon-ABCD design in the appendix.

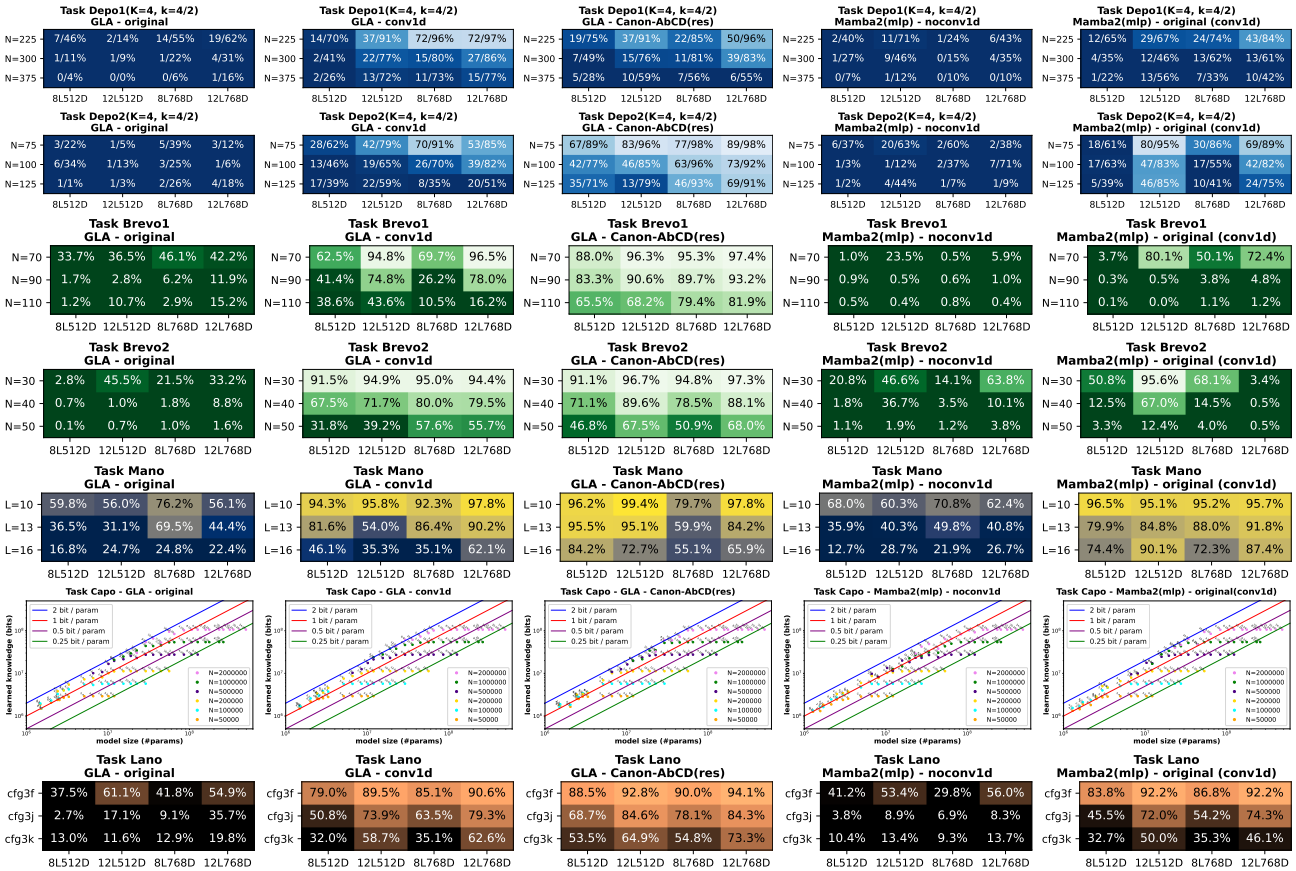


Figure 12: **Columns 1, 2, 3, 5**: Canon drastically improves GLA, making it better than Mamba2 (Result 6.1). **Columns 1, 4, 5**: Removing `conv1d` reduces Mamba2's performance back to match GLA (Result 7.1). *Remark.* Synthetic results here predict similar trends in real-life experiments (Result 12 and Figure 16).

As shown in Figure 12, integrating Canon-AbCD substantially boosts GLA's original (non-conv1d) performance across all benchmarks, transforming it from a weak baseline into a strong competitor. Despite its simplicity, GLA+Canon matches or surpasses Mamba2, particularly on reasoning breadth (Brevo). This upward trend persists in large-scale real-world pretraining (Section 8), improving nearly all standard evaluation metrics.

Result 6.1 (Figure 12)**.** *Adding Canon layers:*

- *Dramatically improves GLA's original performance—raising reasoning depth from 1-hop to 4-hop, doubling reasoning breadth, and more than doubling knowledge manipulation length.*
- *Brings GLA on par with or beyond Mamba2, significantly outperforming it on* Brevo*.*
- *Yields additional gains even over the stronger GLA conv1d baseline.*

As in the Transformer case, we perform ablations to determine optimal Canon placement. GLA also supports feature-map variants like $W_t = \alpha_t W_{t-1} + v_t \phi(k_t)^\top$, with the popular choice $\phi(x) = 1 + \mathrm{elu}(x)$ [35]. We test Canon compatibility both with and without this feature map.

Result 6.2 (Figure 33+34 on Page 52)**.** *Ablation study on GLA:*

- ***Residualness.*** *Unlike in full Transformers, Canon residualness is less critical: non-residual variants work better for* Mano*, while residual ones suit* Lano/Brevo1*.*
- ***Positioning.*** *Canon design is* not intrinsic *to the attention layer. Canon-ACD (or even Canon-A/C/D alone) can outperform Canon-b/B on many tasks, and combining all is best.*
- ***Feature maps.*** *Canon works well with* $1 + elu(x)$ *feature map, though better without it.*[a]

[a]Consistent with [72], where original GLA (without Canon) also performed better without feature maps.

Overall, these ablations highlight the importance of horizontal information flow independent of the architecture sublayers. Interested readers can find our full ablation results in Figure 33+34, where we for instance carefully compared Canon-AbCD(res/no-res), Canon-ABCD(res/no-res), and many more. We recommend the **Canon-AbCD(res)** configuration for GLA—keeping the non-residual `conv1d` from their original codebase while combining it with our residual, activation-free Canon-ACD. This achieves strong gains with *minimal code changes*.

6.2 When Mamba Meets Canon

While Mamba2 is recognized as a state-space model (SSM), it quietly includes a non-linear `conv1d` operation in each SSM block.[17] Originally introduced in H3 [23] as a *shift-SSM*, this mechanism effectively acts as a partial Canon-B layer—performing horizontal mixing on selected coordinates, applying non-linear activation, and omitting residual connections.

Surprisingly, this built-in `conv1d` contributes more to Mamba2's performance than its SSM formulation itself. Disabling it sharply degrades results, reducing Mamba2 to GLA-level performance on both synthetic (Figure 12) and real-world datasets (Section 8). This raises a key question: is Mamba2's strength primarily due to its Canon-like `conv1d` rather than the state-space mechanism?

To isolate this effect, we refer to Mamba2's internal `conv1d` as **Canon-b**, and extend it by adding residual Canon-A/C/D layers—denoted **Mamba2(mlp)+Canon-AbCD**. We also test our own Canon-B design in later ablations and in the appendix.[18] We additionally examine Mamba2 without MLP layers (which exposes Canon-A/B positions), reported in the appendix.[19]

As shown in Figure 13, adding Canon-AbCD further improves Mamba2(mlp) performance over the built-in `conv1d` (Canon-b), especially on Mano and Lano.

Result 7.1 (Figure 12+13)**.** *Key observations on Mamba2:*

- *Mamba2 includes an internal non-linear* `conv1d` *(partial Canon-B) that contributes more to performance than the SSM itself. Removing it drops performance to GLA levels.*
- *Replacing this with full Canon-AbCD layers further improves, notably on* Mano*,* Lano*.*

(Mamba1 [26] shows similar trends but is consistently outperformed by Mamba2 in our playground.)

To further understand Canon–Mamba interactions, we perform ablations varying Canon position, residualness, and initialization. Results mirror GLA: Canon layers remain effective even when placed outside the SSM block, showing that horizontal information flow is architecture-independent.

For initialization, we test the recent *mimetic initialization* [66], proposed to enhance associative

[17]Mamba1 also contains this component, but since Mamba2 consistently outperforms it, we report only Mamba2.

[18]For example, with `ssm_state_size=64` and `num_heads=16`, our Canon-B applies to all $4d + 144$ intermediate coordinates for hidden size d, whereas Mamba2's original `conv1d` acts only on a subset $(2d + o(d))$ with activation.

[19]Such Mamba2 doubles the layer count and recurrent state size compared to Mamba2(mlp). In practice, Mamba2(mlp) is preferred, e.g., in Falcon-H1 [63].

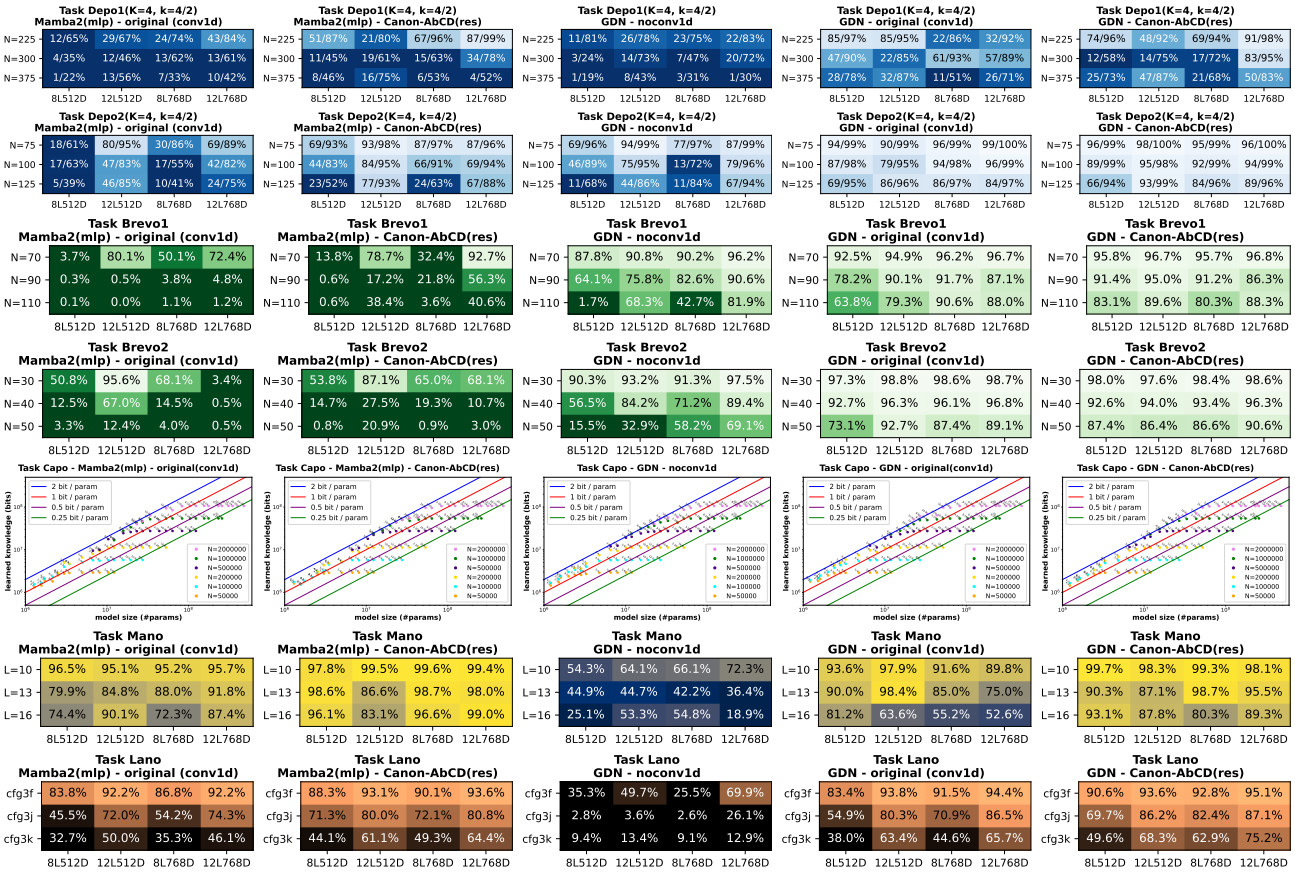


Figure 13: **Mamba(mlp) and GDN architectures** with no `conv1d`, with `conv1d` (original), and with full Canon.

recall and length generalization. However, our experiments (Figure 30+31) find no measurable benefit—and often degradation—on other tasks, suggesting that mimetic init may have overfit length generalization at the cost of broader reasoning. These findings highlight the **importance** of evaluating architectural choices over a **diverse** *synthetic playground*.

Result 7.2 (Figure 30+32 on Page 50)**.** *Ablation study on Mamba2(mlp):*

- *Mamba2(mlp) slightly prefers residual Canon for* LANO*, but non-residual for* MANO*.*
- *Canon layers stay effective outside the SSM block; e.g., Canon-ACD surpasses Mamba2(conv1d) on* DEPO2/LANO*, highlighting their strength as general horizontal-mixing modules.*
- *Mimetic initialization [66], designed for length generalization, harms shorter-context performance, reinforcing the need for diverse-task evaluation.*

We also evaluate Mamba2 without MLP layers (Figure 30+31); results remain consistent with those above. Interested readers can refer to Figure 30+31+32 for complete ablation results, including detailed comparisons between Canon-ABCD(res/no-res), Canon-AbCD(res/no-res) and many more. Our overall recommendation remains **Canon-AbCD(res)** for simplicity.

6.3 When Gated DeltaNet Meets Canon

Gated DeltaNet (GDN) [73] extends GLA with a gated delta-rule update. Instead of GLA's $W_t = \alpha_t W_{t-1} + v_t k_t^\top$, GDN adopts $W_t = \alpha_t W_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$, where β_t controls the balance between forgetting and writing. This formulation retains GLA's efficiency while adaptively

suppressing redundant information, allegedly yielding better reasoning and improved gradient flow.

Each GDN block retains the linear-attention–plus–MLP structure but also includes a non-residual, activated `conv1d` layer within its linear attention sublayer—referred to here as **conv1d** or **Canon-b**. This component remains important, though less critical than in GLA or Mamba2. Removing it *destroys* knowledge manipulation (MANO) and hierarchical reasoning (LANO), while in-context reasoning (DEPO/BREVO) is largely unaffected. (Section 8 later shows such differences may vanish in academic-scale real-life pretraining, *highlighting the importance of a versatile synthetic pretrain playground.*)

Following prior sections, we extend GDN by adding residual Canon-A/C/D layers, forming **GDN+Canon-AbCD**. We also test our own Canon-B design in later ablations and the appendix. As shown in Figure 13, Canon-AbCD slightly improves GDN+`conv1d` across benchmarks.

Result 8.1 (Figure 13)**.** *Key observations on GDN:*

- *GDN is less dependent on its internal* `conv1d` *(Canon-b) for strong performance.*
- *Replacing it with full Canon-AbCD layers still yields improvements, albeit marginal.*

We further perform ablation studies on Canon positioning and residualness:

Result 8.2 (Figure 35+36 on Page 53)**.** *Ablation studies on GDN:*

- *GDN slightly prefers non-residual Canon on* MANO*, though overall differences are minor.*
- *Canon layers remain effective even outside the GDN layer; e.g., Canon-ACD performs on par with GDN+*`conv1d`*, underscoring their generality as horizontal-mixing components.*

Interested readers can refer to Figure 35+36 for full ablation results, including detailed comparisons among Canon-ABCD(res/no-res), Canon-AbCD(res/no-res), and others. For simplicity and consistency, we recommend **Canon-AbCD(res)** as the default configuration.

7 Final Comparisons and Lessons to Architecture Design

Applying Canon uniformly across all architectures creates a controlled environment—like dropping them from the same height at the Tower of Pisa—revealing their *true* architectural trade-offs. We exclude hybrid models (e.g., Griffin [20], Samba [50]) to isolate behaviors of the *base* architectures.

7.1 Summary on Linear Models vs. Canon Layers

While many more linear-time architectures remain worth exploring, this study focuses on GLA, Mamba2, and GDN.[20] Despite their structural differences, several consistent insights emerge.

Result 9 (Section 6+Figure 14)**.** *Summary of Canon effects on linear models:*

- ***Universality.*** *Canon-ACD already matches internal* `conv1d`*, showing that horizontal mixing is* useful across all sublayers*, not limited to linear attention (i.e., the recurrent / SSM layer).*
- ***Robustness.*** *Adding Canon layers* never hurts*; the residual design stabilizes training.*
- ***Sufficiency.*** *Most performance appears achievable with the simplest* GLA+Canon-AbCD*, suggesting the current direction of linear-model architecture design may warrant re-evaluation.*

To elaborate more on the third bullet, modern models (Mamba2, GDN) show only marginal gains over the simple GLA+Canon-AbCD baseline. This suggests that many recent architectural

[20] GDN results were newly added after the NeurIPS 2025 accepted version (V1.1).

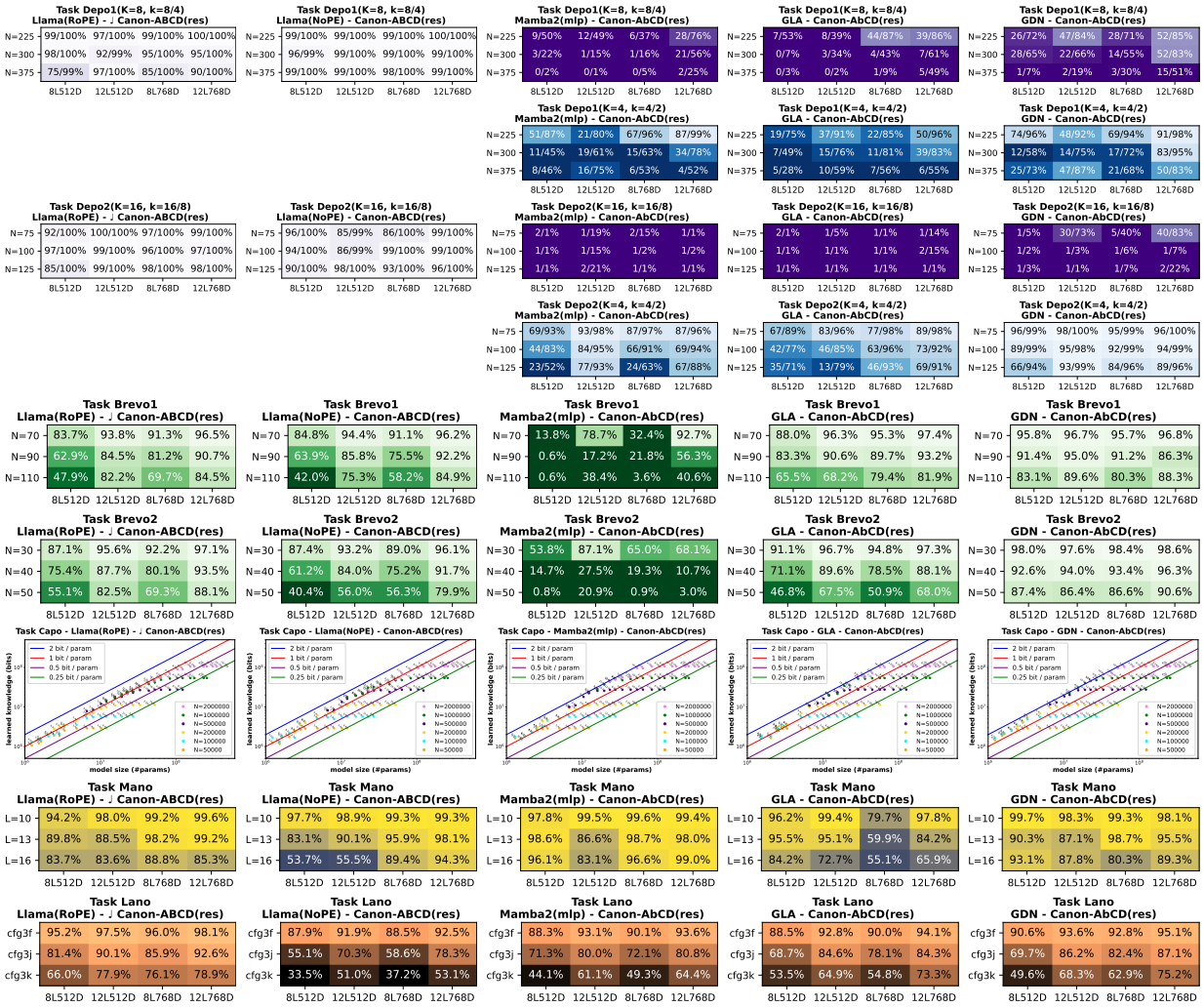


Figure 14: **Final comparison of base architectures** equipped with full-score Canon layers: RoPE(♩), NoPE, Mamba2, GLA and GDN. Most notably, with Canon layers added, Mamba2/GLA/GDN still underperform Transformers by 2× in reasoning depth, with meaningful results only for DEPO(K=4).

innovations may largely *replicate Canon-like horizontal mixing* rather than introduce fundamentally new computation. While such mechanisms can reduce explicit reliance on Canon layers, their improvements remain limited—raising the question of whether increasing architectural complexity truly expands capability or merely redistributes existing functions.[21]

Remark 7.1. We do not claim that "replicating" Canon is unworthy—such designs may improve efficiency and reduce GPU memory. However, it is crucial to understand what the model actually learns: complex module designs *need not realize* complex functions, as optimizers may often converge to simpler functions (e.g., Canon-like solutions in this case).[22]

[21] In our follow-up work [2], we show that Canon layers can lift GLA to match GDN (+ full Canon) even on 1B- to 8B-sized models pretrained using real-life data, further strengthening this point.

[22] The same holds broadly in deep learning: although an ℓ-layer quadratic MLP can represent a 2^ℓ-degree parity function, learning it is computationally intractable. Existence rarely implies learnability via training [4].

7.2 Summary on Transformer vs. Linear Models

We now compare Transformers and linear models under a *controlled, apple-to-apples* setting with full Canon layers added to all architectures.

Result 10 (Figure 14)**.** *With full-score Canon layers added, we find:*

- *reasoning depth: RoPE(♩) ≈ NoPE ≫ Mamba2 ≈ GLA ≈ GDN (e.g., 4× deeper reasoning);*
- *reasoning breadth: GDN ≥ RoPE(♩) ≈ NoPE ≈ GLA > Mamba2;*
- *knowledge capacity: Mamba2 ≈ GLA ≈ GDN ≫ RoPE(♩) ≈ NoPE (e.g., 1.4× capacity);*
- *knowledge manipulation: GDN ≈ Mamba2 ≈ RoPE(♩) ≥ NoPE ≈ GLA;*
- *hierarchical structure: RoPE(♩) > NoPE ≈ Mamba2 ≈ GLA ≈ GDN.*

Remark 7.2. The initial comparison (Figure 4) was not controlled: Mamba2 and GDN included internal `conv1d` layers, whereas GLA and Transformers did not. By adding full Canon (Canon-ABCD or -AbCD) layers to all, the comparison becomes scientifically meaningful.

While others may interpret the fine differences across architectures, we focus here on the most pronounced contrasts. First, linear models—regardless of design—consistently show a ∼40% gain in CAPO knowledge capacity compared to full Transformers. This is intuitive: their recurrent structure better supports associative-memory representations (an existential proof), and more importantly, optimizers can *learn* such representations effectively in practice.

More surprising is the behavior on reasoning depth. Linear models remain systematically weaker—about 2× on DEPO1 (8-hop vs. 4-hop) and up to 4× on DEPO2 (16-hop vs. 4-hop)—even under identical training conditions. We next examine this phenomenon in detail.

Deep Dive into Deep Reasoning for Linear-Time Models. We find that, due to compression of in-context knowledge, linear models struggle to reach 99% accuracy even on simple 1- or 2-hop retrievals (Figure 15), despite extended training. When reasoning depth exceeds 2 hops, early-step errors compound rapidly, preventing successful deep reasoning. In contrast, Transformers—especially with Canons—achieve near-perfect 1- and 2-hop accuracy very quickly (Figure 15).

Importantly, this weakness is **not due to insufficient recurrent memory**.

For instance, in Mamba2, **each layer** passes $128d$ parameters (expansion × `ssm_state_size` × hidden size d)—**hundreds of times more than sufficient** to store the full input sequence.[23] Moreover, Mamba2 performs well on 1-hop tasks (K=1) even with a single layer, confirming the bottleneck is not information-theoretic (a finding also to be reinforced in Section 8).

The same pattern holds for GLA and GDN, whose per-layer recurrent states ($64d$–$144d$) also provide ample capacity to store entire contexts (see Appendix C for architecture specifications). Hence, the true limitation lies in *memory dynamics*—how efficiently in-context information is encoded during compression and how reliably it is retrieved for reasoning. Errors in encoding or retrieval accumulate across hops, severely degrading multi-hop reasoning.

These results expose the *Achilles' heel* of current linear architectures and point to a concrete direction for future research: improving the fidelity of compressed in-context memory. Until such limitations are resolved, hybrid approaches that combine sliding-window attention (for deep reasoning) with linear or state-space components (for long-context compression) remain the most practical path forward.

[23] In Task DEPO, representing N key-value pairs with vocabulary V requires at most $2N \log_2 V$ bits. For DEPO2, with N=75 and $V \leq$2500, this is under 1700 bits, compared to Mamba2's recurrent state of $12 \times 128 \times 768 \approx$1.2M 32-bit floats. This occupies ∼0.001 bits per float; in contrast, long-term (factual) memory in weights can reach 2 bits per float (see [8] our Task CAPO).

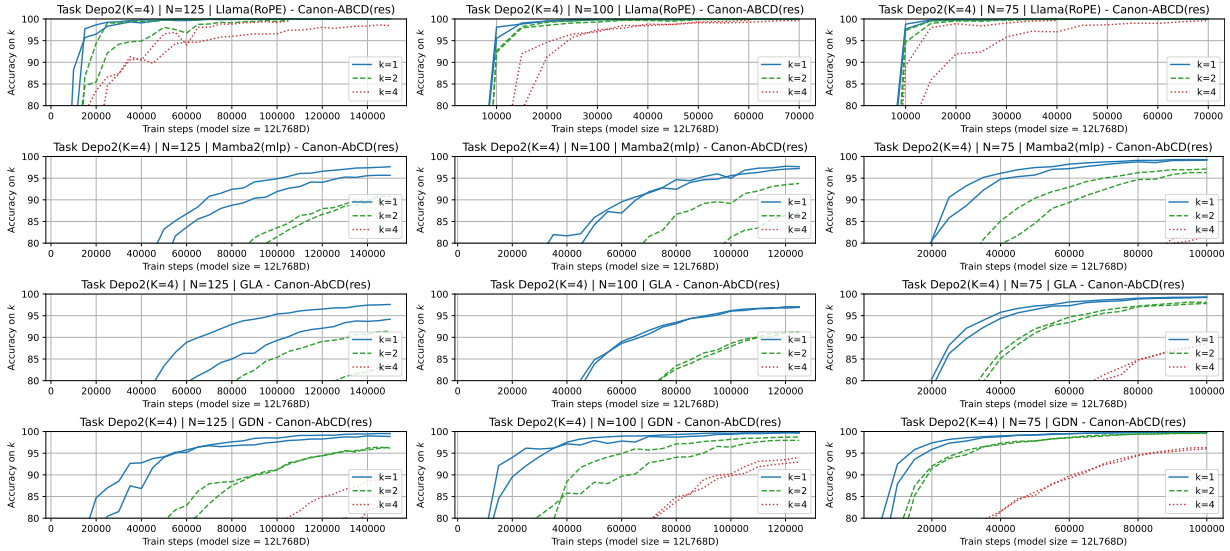


Figure 15: **Training curves for 12L768D architectures on** DEPO2(K=4), evaluated at $k = 1, 2, 4$ and $n = N$, with results shown across two best LRs for each k. Results for other data are in Figure 20 on Page 42.

Result 11 (Figure 15)**.** *Linear models such as Mamba2/GLA/GDN struggle with deep reasoning—not from lack of memory, but from accumulated errors in compression and retrieval. Hybrid models combining Transformers and linear layers, equipped with Canon, mitigate these limitations.*

8 Real-Life Experiments

We conduct real-life pretraining at the academic scale: 1.3B-parameter models trained on 100B tokens from FineWeb-Edu [42] and SlimPajama [60], using a 4096 context length (details in Appendix B). This mirrors setups common in recent studies such as Titans [10], GDN [73], and MTA [25], representing the standard academic pretraining paradigm.

Evaluation suites. We first evaluate all models on two benchmark suites. The first, based on `lm-evaluation-harness` [24], covers **discriminative tasks**: PIQA [12], HellaSwag [79], WinoGrande [51], ARC-easy/challenge [18], SIQA [52], BoolQ [17], WikiText, and LAMBADA [41]. Following prior work [10, 73], we adopt the original accuracy metrics for consistency.[24]

The second **generative-task** suite uses the *Just Read Twice (JRT)* protocol [9], designed to reduce noise in generative testing at this scale.[25] Tasks include SWDE, FDA, SQuAD(v2) [49], TriviaQA [34], NQ [38], and DROP [21], plus their JRT-enhanced variants (denoted as FDA2, SWDE2, etc.) We again follow the official JRT codebase for evaluation.

Key observations across both suites. Results show large variance across random seeds:

- Benchmark scores fluctuate with random seeds—up to **4% on LAMBADA, 3% on BoolQ**, and 1–3% elsewhere. Generative tasks vary even more (**9% on FDA**, **8% on SWDE**, 3–5% on others). The same holds even if data shuffling is fixed and model init varies (Appendix E.1).

Hence, only differences beyond these thresholds are statistically meaningful. From Figure 16:

[24] Following tradition [10, 72, 73], we use (`acc_n`) for HellaSwag and ARC-c, but `acc_n` for other tasks.

[25] Generative testing can be noisy at this scale, as such models often struggle with prompt comprehension. JRT addresses this by repeating the context and question twice, allowing models to more accurately reveal their intrinsic generative capabilities.

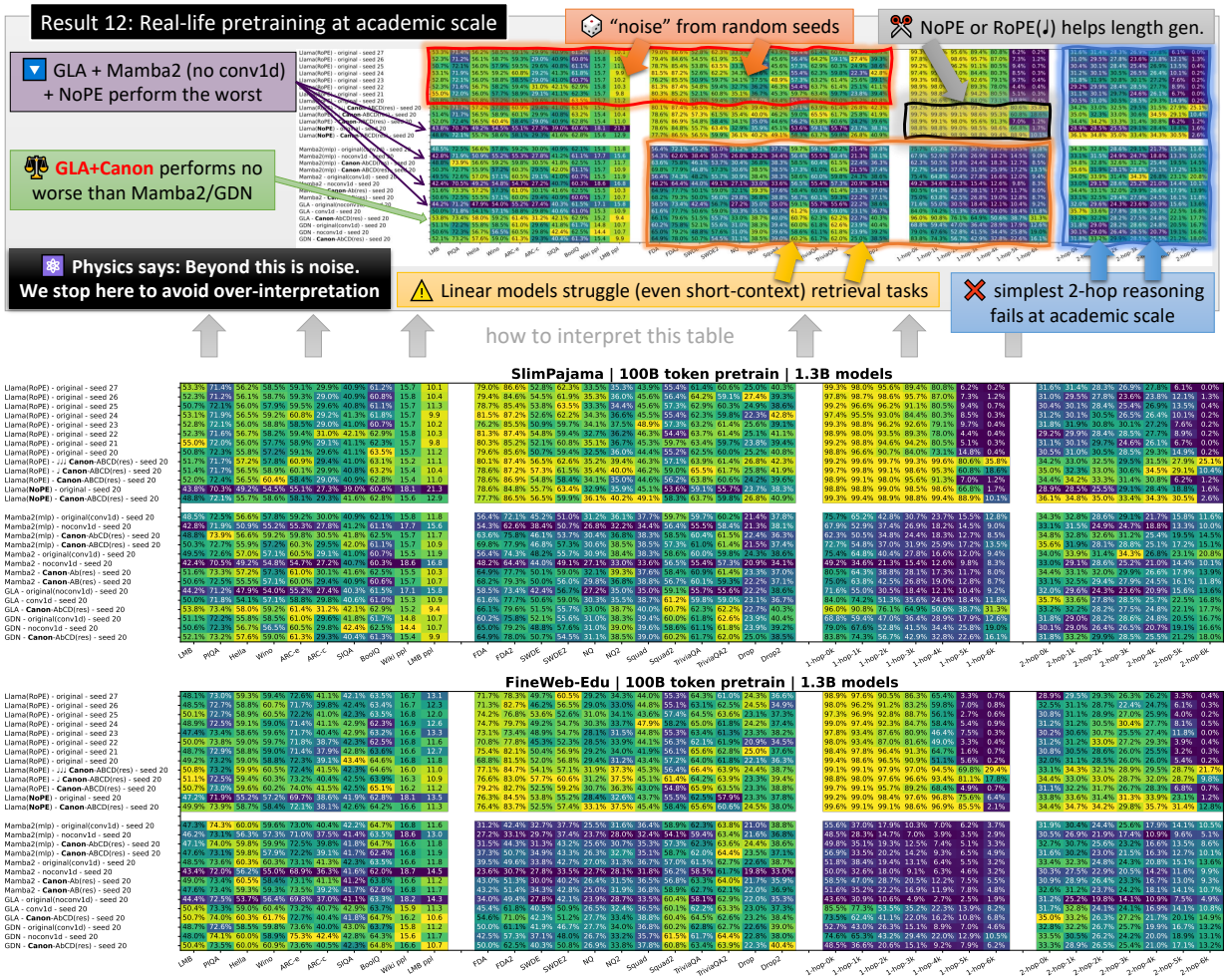


Figure 16: **Performance of 1.3B models pretrained on 100B tokens** across discriminative (left), generative (middle), and 1/2-hop reasoning (right) tasks. Best of 2 learning rates for Llama; 3 for GLA, Mamba, and GDN. GPT2 variants (e.g., squared ReLU) shown in Figure 22 on Page 44.

- Linear models (Mamba2, GLA, GDN) underperform full Transformers on generative tasks, **even for contexts shorter than training length**.[26] Retrieval-heavy tasks (FDA, SWDE) amplify this gap, consistent with Result 11.
- NoPE, GLA, and Mamba2 (w/o `conv1d`) perform poorly in base form but improve markedly with full Canon. GLA+Canon surpasses Mamba2 and matches GDN (even with Canon); NoPE+Canon performs on par with RoPE. GDN is least sensitive to Canon yet not clearly stronger than GLA+Canon—consistent with Result 3, 6.1, 7.1, 8.1, and 9.
- At this scale (1.3B/100B), RoPE, RoPE+Canon, and NoPE+Canon perform comparably, and most linear+Canon variants cluster together. **Academic-scale pretraining cannot reliably distinguish finer architectural differences.**

Needle, Babilong, and our Multi-Hop Reasoning Tasks. The Needle-in-a-Haystack (**NIAH**) task from RULER [29] tests recall of a "needle" value (e.g., a magic number) in long text. This makes it *too easy*: models—especially linear ones—may appear accurate while failing at most basic short-context retrieval (see later). For completeness, results are shown in Figure 23 (Page 45).

The **Babilong** dataset [37] embeds bAbi [69] tasks in long junk-filled passages to test multi-hop

[26] Generative task prompts are capped at 1024–2048 tokens (per original codebase), while training used 4096.

reasoning but proves overly difficult at this scale.[27] As shown in Figure 23, Babilong results are mostly indistinguishable; only trends are clear:

- Linear models underperform Transformers even on short contexts, confirming their weakness stems from *inefficient compression and retrieval*, not memory size (Result 11).
- Transformers gain on longer contexts when RoPE is reduced (RoPE♩) or removed (NoPE), particularly in 4k-token junk passages (c.f. Result 3).

To balance NIAH's simplicity and Babilong's difficulty, we introduce **our own multi-hop reasoning tasks**. **1-hop-L** embeds *five* birth-year statements within Wikipedia passages of length L, requiring direct recall of one of the birth years. **2-hop-L** embeds *three* birth-year statements plus three equivalence links (e.g., "XXX was born the same year as YYY"), requiring inference of the linked names' birth years. Details are in Appendix B. Results (Figure 16) show:

- All models struggle with 2-hop-L, achieving only 30–36% (near random) even with $L = 0$.
- **1-hop-L separates architectures**: full Transformers outperform linear models **even for** $L = 0$ (short contexts < 100 tokens), while NoPE and RoPE(♩) generalize better as L increases.

To summarize:

Result 12 (Figure 16+23)**.** *Academic-scale pretraining (1.3B params, 100B tokens, 4k context) shows* high noise and limited resolution*, making most architectural differences statistically insignificant. Yet several* consistent findings *hold:*

- *Linear models (Mamba2, GLA, GDN) underperform full Transformers* ***even on short-context retrieval tasks*** *(FDA, SWDE, or 1-hop-L with ∼100 tokens), even with Canon (Result 11).*
- *Canon elevates NoPE to RoPE-level (Result 3), GLA to Mamba2/GDN-level (Result 6.1, 9); removing* `conv1d` *downgrades Mamba2 to GLA (Result 7.1) but hardly affects GDN (Result 8.1).*
- All models fail 2-hop reasoning, even within 100 tokens*, revealing limit of academic-scale pretrain.*
- *Reducing or removing RoPE (NoPE, RoPE♩) improves long-context generalization (Result 3).*

9 Conclusion and Future Direction

Academic-scale pretraining suffers from high noise and failed multi-hop reasoning, hindering reliable architectural comparison. Our controlled synthetic playground offers a **cost-effective, principled alternative**: by decomposing intelligence into atomic tasks, we discover and optimize *Canon layers*—lightweight constructs that enhance reasoning depth and breadth, knowledge capacity and manipulation, and structural reasoning across diverse architectures.

Canon layers revive weaker models (e.g., NoPE, GLA) to match or surpass stronger baselines (e.g., RoPE, Mamba2), reduce reliance on RoPE to improve length generalization, and pinpoint that linear models' depth limitations arise from compression/retrieval inefficiencies rather than memory. Like residual connections or LoRA—simple yet powerful—Canon layers may become a minimal yet broadly applicable architectural primitive.

While our academic-scale real-world experiments align with synthetic findings, industrial-scale validation remains crucial; we hope our systematic, economical methodology **encourages future investigations** at larger scales. We plan to open-source our playground and evaluation suite to support rigorous, reproducible architecture research.

[27] For instance, in `babilong.qa2`, "Charlie got a bottle ... Charlie moved to the balcony." → "Where is the bottle?"—models score $< 37\%$ even without junk, i.e., random guessing.

Future Directions. Several interesting directions arise from this work:

- Alternative Canon Implementations. We focused on simple linear convolutional (kernel size 3) Canon layers for their simplicity and efficient CUDA kernels. Future work should explore dynamic, adaptive convolutions—with weights conditioned on hidden states to enable gating—to assess whether performance gains justify the added computational overhead.
- Fine-grained Canon Design. We briefly explored selective application (e.g., early layers) and cross-layer connections—e.g., $h'_{\ell+1} = h_{\ell+1} + \text{Canon}(h_\ell)$—which can fuse multiple intra-layer Canon operations into a single step, improving efficiency. A systematic evaluation within our synthetic framework could identify optimal Canon configurations. We are open to exploring this direction further, especially if the community expresses significant interest.
- Evaluating Emergent Architectures. We selected one representative per architecture family to ensure controlled comparisons and consistent inclusion of Canon layers. Without this rigor, results may misleadingly attribute Canon's gains to inherent architectural differences (e.g., Mamba2's built-in conv1d). With controlled comparisons in mind, future work can fairly evaluate emergent architectures, potentially discovering new components with statistically significant improvements.
- Enriching the Synthetic Playground. Our five synthetic tasks are only a starting point. Designing additional tasks that isolate other architectural capabilities *beyond those revealed here*—while remaining as atomic as possible—is crucial for finer-grained characterization of model strengths and weaknesses.
- Interpretability and Probing. We omitted interpretability and probing analyses here for clarity, despite existing frameworks for most tasks (e.g., Lano [6], Capo [5], Mano [7], Brevo [75, 76]). We have conducted preliminary probing for Depo, revealing internal model strategies such as positional parsing (even/odd positioning encoding "$\to a$" or "$a \to$") and preprocessing of permutations before the first query (analogous to Brevo [75]). We choose not to include them for clarity, as this paper focuses on architectural comparison.
- Sparking New Architecture Designs. By pinpointing specific weaknesses (e.g., linear models' reasoning depth limits and compression inefficiencies), our framework provides targeted signals for improved future designs. We hope synthetic benchmarking informs and inspires the next generation of architecture innovations.

Appendix

This appendix contains full technical specifications and implementation details for all experiments presented in the main paper. It is intended to support reproduction and in-depth inspection. We provide complete training protocols and evaluation procedures for all five synthetic tasks (Depo, Brevo, Capo, Mano, Lano), real-life experiments (1-hop-L, 2-hop-L, Babilong), and 100B-token SlimPajama/FineWeb-Edu pretraining. We also document the architectural configurations for all models, including Transformers, GLA, Mamba, GDN variants, and MoEs. Additional ablation figures, KL-divergence evaluations, and variant comparisons are included for readers interested in deeper technical insights or replication of results.

A Details on Synthetic Pretraining Tasks

We intend to release the code for generating all synthetic pretraining datasets used in this paper, though this may require additional time. To make this paper fully self-contained, we provide detailed specifications below.

Remark A.1. Throughout this paper, we utilize combinations of A100, H100, and H200 GPUs with bf16 mixed-precision training. While we report the total batch size used in our experiments, we do not specify the exact number of GPUs, as this does not materially affect the results.[28]

A.1 Details on Task Depo: Mental Reasoning Depth

The synthetic pretraining task Depo is designed to evaluate mental reasoning depth by requiring multi-step traversal over directed permutations. The dataset is defined by two parameters: the maximum permutation size N and the reasoning depth K. Each problem instance is generated as follows:

First, a permutation length n is sampled uniformly from $\{3, 4, \ldots, N\}$. A directed permutation of n nodes is then created, representing a cycle where each node points to its successor: $x_1 \to x_2 \to \cdots \to x_n \to x_1$. The permutation is presented as edges in the form of ordered pairs (x_i, x_{i+1}), but these edges are shuffled randomly into a sequence of $2n$ tokens. This random ordering ensures that the original cycle structure is not immediately apparent, which would otherwise make the task trivial. The final data format is:

```
<bos> x1 y1 x2 y2 ... xn yn <query k_1> q_1 <ans> a_1 ... <query k_t> q_t <ans> a_t
```

Here, $x_i \to y_i$ represents shuffled edges of the permutation. For each query q_j, a node is randomly chosen from $\{x_1, \ldots, x_n\}$, and its k_j-th successor in the permutation is computed based on the reasoning depth $k_j \in [K]$, sampled uniformly. The correct answer a_j is the k_j-th successor of node q_j. The number of queries t is set as $\min(10, n)$ to balance computational feasibility while ensuring smaller graphs remain interpretable.

Two variants of Depo are used:

- Depo1: Each node name is encoded as 1–2 tokens, with a vocabulary size of 50.

[28] For instance, training with a single GPU and a batch size of 128 is equivalent to training with 64 GPUs where each GPU processes a batch size of 2. Our codebase supports dynamic GPU allocation, ensuring the total batch size is fixed across training runs while the number and type of GPUs may vary.

- DEPO2: Each node name spans 5–7 tokens using a small vocabulary size of 4, introducing ambiguity that challenges the model's disambiguation capabilities.[29]

In addition to node names, special tokens are used: `<bos>`, `<ans>`, and `<query k>` for $k \in \{1, \ldots, K\}$. The total number of special tokens is $K + 2$.

Sampling distribution. To ensure controlled task difficulty progression, n is sampled proportionally to $\frac{1}{\sqrt{N}+n}$. This distribution biases training toward simpler cases early on, allowing the model to gradually build foundational reasoning skills before encountering harder examples. Although this distribution is not perfect, it is both simple and effective, enabling clean comparisons between architectural designs without introducing unnecessary hyperparameter complexity. More sophisticated curriculum-based approaches, such as scheduled difficulty [39], may provide an alternative solution but could introduce significant noise, thereby complicating controlled comparisons.

Remark A.2. This distribution was proposed and tested thoroughly by ZA in 2023 in a number of settings, and subsequently tested (via private communication) by Alfarano in modular arithmetic pretraining [53], where it was benchmarked against other options and shown to also perform well. While synthetic data like this cannot fully replicate the intricacies of real-world distributions, it allows us to simulate an ideal training regime. This forward-looking approach anticipates future improvements in pretraining data—such as higher-quality datasets or RL-based post-training—and evaluates model architectures based on their scalability under such optimal conditions.

Training protocol. To reduce computational cost, we employ label masking: cross-entropy loss is computed only on tokens associated with `<ans>` and a_j. This optimization halves training duration without affecting architectural comparisons. Problem instances are generated online, concatenated, and aligned into 2048-token context windows. Left alignment ensures that the first problem instance in each context is never truncated, as truncation leads to incomplete edges and unusable data.

Evaluation protocol. During evaluation, the permutation size is fixed at $n = N$, and reasoning depth is tested at both $k = K$ (maximum depth) and $k = K/2$ (intermediate depth). The protocol mirrors training by generating and concatenating evaluation samples online into 2048-token windows. Accuracy is reported over all answer tokens in the window, ensuring that results are stable regardless of whether answers appear early or late in the sequence.

Data splits and hyperparameters. For DEPO1, we use $N = 375$, 300, 225 and primarily $K = 8$, while testing $K = 4$ for weaker models. Models are trained from scratch with fresh data while using a fixed random seed to ensure data consistency across architectures. Training uses a batch size of 128, AdamW optimizer ($\beta = 0.9, 0.98$ and $\varepsilon = 10^{-6}$), weight decay of 0.03, learning rate warmup for the first 1000 steps, and cosine decay to 10%. Training steps are set to 112.5k, 100k, or 87.5k, adjusted for the problem lengths $N = 375$, 300, 225. The best accuracy is reported across four runs using learning rates $\{0.0003, 0.0005, 0.001, 0.002\}$.

Similarly, in DEPO2, we use $N = 125$, 100, 75 and $K = 16$ (or $K = 4$ for weaker models). Training steps are set to 150k, 125k, and 100k, respectively.

A.2 Details on Task Brevo: Mental Reasoning Breadth

Our pretraining synthetic task BREVO is designed to test mental reasoning breadth by requiring a subgraph topological sort from a given directed acyclic graph (DAG). The dataset is defined by a maximum graph (node) size N. For each problem instance, we first sample a graph of size

[29] Multi-token names are generated such that the first $\ell - 1$ tokens are chosen from $[1, V]$, while the final token is selected from $[V + 1, 2V]$. This creates implicit word boundaries similar to those handled by BPE-based tokenization strategies, such as GPT2Tokenizer.

$n \in \{3, 4, \ldots, N\}$ using the same sampling distribution $\propto \frac{1}{\sqrt{N}+n}$ as employed in DEPO, and generate data in the following format:

```
<bos> x1 y1 x2 y2 ... xm ym <query> q <ans> a1 a2 ... ap <eos>
```

Here, the $2m$ tokens define m directed edges $x_i \to y_i$ spanning n nodes, meaning that y_i depends on x_i. Given a query vertex q, the model must return all vertices it recursively depends on, in topological order starting from the leaves. Specifically, if $u \to v \to q$, the model must output u before v.

DAG generation protocol. After sampling n, we generate the random DAG as follows. First, we randomly shuffle all the vertices and begin inserting edges. We select a random number $L \in \{1, \ldots, \lceil \frac{n-1}{4} \rceil + 1\}$, designating the first L vertices as leaves (no incoming edges). Starting from vertex $L + 1$, we iteratively process each vertex by selecting all preceding vertices that have an out-degree of at most 3. From this set, we randomly pick a subset of between 1 and up to 4 vertices and connect them to the current vertex. This process continues until all vertices are traversed, yielding a DAG with a maximum in-degree and out-degree of 4.[30]

At this point, the vertices naturally form a topological order from left to right. We then select a random query vertex from the last quarter of the vertices. Choosing vertices closer to the right increases the depth of the dependency graph while avoiding degenerate cases where all nodes are reachable (such as if the query were the last vertex). Finally, we reshuffle all the vertices and assign random names to them. Vertex names are uniquely selected, as described below.

Vertex names. In BREVO1, each vertex name consists of a single unique token, randomly selected from $\{1, \ldots, N\}$. In BREVO2, each vertex name spans 2–4 tokens using a vocabulary of size 4, which introduces ambiguity (e.g., multiple token combinations can encode unique vertex names). See Footnote 29 for the method used to generate multi-token words. Aside from vertex names, we use 4 distinct special tokens: `<bos>`, `<query>`, `<ans>`, and `<eos>`.

Training protocol. To reduce computational costs, we enable label masking, where the cross-entropy loss is computed only on `<ans>`, `<eos>`, and a_j tokens. Selective testing showed that this technique saves training time without affecting architectural comparisons. Instances are generated online, concatenated, and left-aligned into context windows. By left-aligned, we mean that the first instance in each context window is never truncated. Without left alignment, truncation of the first instance would render it incomplete (e.g., missing edges in the graph), making the instance a useless training example.

Evaluation protocol. During evaluation, we fix $n = N$ and test only the largest graph. The model is prompted with a random DAG of size n and query vertex q, and tasked to generate the answer sequence $a_1, \ldots, a_p$. The generated sequence is then parsed and validated against the following criteria:

- The answer sequence must contain all reachable vertices from q and no non-reachable vertices.
- The vertices in the answer sequence must appear in a valid topological order. Since topological orderings are not unique, any valid ordering is accepted.

Invalid tokens, duplicate outputs, or missing vertices are not accepted, and no partial credit is given.

Training details. In BREVO1, we use $N = 110, 90, 70$ with vertex names consisting of one token, and each problem fits within 1024 tokens. Models are trained from scratch with fresh data but

[30] Constraining the maximum in-degree and out-degree to 4 prevents the dependency graph from becoming too shallow, which would make the task trivial.

a fixed seed (ensuring pretraining data consistency across model architectures). Training uses a context length of 1024, a total batch size of 256, AdamW optimizer ($\beta = 0.9, 0.98$ and $\varepsilon = 10^{-6}$), weight decay of 0.03, learning rate warmup over the first 1000 steps, and cosine decay to 10%. Pretraining lasts 150k, 125k, or 100k steps respectively for $N = 110, 90, 70$, accounting for the varying problem lengths. We report the best performance out of four runs using learning rates $\{0.0003, 0.0005, 0.001, 0.002\}$.

In BREVO2, we use $N = 50, 40, 30$, with vertex names spanning 2–4 tokens, and each problem fits within 1536 tokens. Models are trained in the same manner as BREVO1, except that we use a context length of 1536, a total batch size of 192, and pretraining lasts 250k, 225k, or 200k steps respectively for $N = 50, 40, 30$.

The comparison between BREVO1 and BREVO2 demonstrates that the ambiguity introduced by multi-token vertex names does not noticeably impact architectural comparisons, which is the focus of this paper.

A.3 Details on Task Capo: Knowledge Capacity

The synthetic pretraining task CAPO borrows directly from Allen-Zhu and Li [8], where the authors introduced the $\mathsf{bioS}(N)$ dataset. This dataset contains N biographies of randomly generated individuals, each described by six attributes: birth date, birth city, university, major, employer, and working city.[31]

To represent these biographies in natural language, each individual is described via randomly selected English sentences for every *exposure* to the pretraining data. Sentence templates correspond to the individual's attributes, ensuring diverse paraphrasing across exposures. For example:

> Anya Briar Forger was born on October 2, 1996. She spent her early years in Princeton, NJ. She received mentorship and guidance from faculty members at Massachusetts Institute of Technology. She completed her education with a focus on Communications. She had a professional role at Meta Platforms. She was employed in Menlo Park, CA.

The diversity in writing ensures that models learn to store explicit knowledge about an individual's attributes, rather than merely memorizing surface-level patterns in specific templates [5, 7]. Following the recommendations of [8], we pretrain models over 100 *exposures* per individual, which provides a controlled environment for comparing architectural differences. Training beyond 100 exposures diminishes architectural differences, as longer training typically allows all models to converge toward similar levels of performance [8].

Knowledge format independence. Previous experimental evidence suggests that a model's knowledge capacity does not heavily depend on the specific format in which the knowledge is stored. For example, one could consider synthetic alternatives such as longer word lengths, different vocabulary sizes, or even abstract encoding formats. Importantly, any such synthetic configuration remains a reliable discriminator for comparing model architectures. For simplicity and interpretability, however, we adhere to the more English-like biography format in $\mathsf{bioS}(N)$, aligned with [8].

Clean experimental comparisons. Models could alternatively be pretrained on exposures distributed according to power-law dynamics or incorporating infrequent "junk data." While such approaches might better mimic real-life datasets, they introduce subtle stochastic effects that can depend heavily on the formatting of rare samples. To avoid confounding factors, we adopt the cleaner 100-exposure baseline for pretraining individual biographies, as it allows for clearer isolation of architectural capabilities.

[31] The working city is derived from the employer's headquarters, while all other attributes are sampled uniformly and independently. Possible attribute domains include $N_0 = 400 \times 400 \times 1000$ person names, $12 \times 28 \times 200$ birth dates, 200 birth cities, 300 universities, 100 majors, 263 employers, and two pronouns.

Evaluation protocol. After pretraining on bioS(N) data, knowledge capacity is measured based on the number of *bits* a model reliably stores. This quantity is further normalized to **bits per parameter** to account for model scale. Partial correctness (e.g., recalling the year but not the full date of birth) is accounted for in the bit computation to ensure fine-grained evaluation of knowledge storage. For detailed computation, we direct readers to [8]. Unlike other tasks presented in this paper, measurement of bits per parameter requires varying both data sizes N and model sizes to compute the Pareto frontier of knowledge capacity versus parameter count. For this reason, we vary N between 50K and 2M while testing models ranging from 1M to 500M parameters.

Pretraining setup. To ensure consistency across all architectures, pretraining uses the GPT-2 tokenizer and ties weights for embedding and output layers. Tying weights ensures consistent learning dynamics across model families (e.g., GPT, Llama, Mamba, GLA), while limiting the vocabulary size to 3275 tokens (from GPT-2's original 50257 tokens), as the bioS(N) dataset does not utilize the entire vocabulary.

Batch size, learning rate decay, and other hyperparameters strictly follow the 100-exposure baseline outlined in [8], with only minor modifications. Specifically, we test *two* learning rates per configuration (selected from their three choices) and report the best results. As a result, our reported knowledge capacity in Figure 4 may slightly deviate from the original results, though introducing Canon layers restores capacity without adding hyperparameter choices.

Hyperparameters for dense models. The following hyperparameters were used for dense models in the 100-exposure setup:

- For $N = 50$K: weight decay $wd = 0.01$, learning rates $lr = 0.001/0.0005$, batch size 12.
- For $N = 100$K: $wd = 0.01$, $lr = 0.001/0.0005$, batch size 24.
- For $N = 200$K: $wd = 0.01$, $lr = 0.001/0.0005$, batch size 48.
- For $N = 500$K: $wd = 0.01$, $lr = 0.001/0.0005$, batch size 96.
- For $N = 1$M: $wd = 0.01$, $lr = 0.001/0.0005$, batch size 192.
- For $N = 2$M: $wd = 0.01$, $lr = 0.0005/0.0003$, batch size 384.

Hyperparameters for MoE models. Mixture-of-Experts (MoE) training was conducted using the `tutel_moe` package [30], consistent with [8]. MoE training uses 32 experts with $topk = 1$ and $cap\_factor = 2$. Due to the larger learning rates required for MoE-based pretraining, we use the following hyperparameters:

- For $N = 50$K: $wd = 0.01$, $lr = 0.005/0.002/0.001$, batch size 12.
- For $N = 100$K: $wd = 0.01$, $lr = 0.005/0.002/0.001$, batch size 24.
- For $N = 200$K: $wd = 0.01$, $lr = 0.005/0.002/0.001$, batch size 48.
- For $N = 500$K: $wd = 0.01$, $lr = 0.002/0.001$, batch size 96.
- For $N = 1$M: $wd = 0.01$, $lr = 0.002/0.001$, batch size 192.
- For $N = 2$M: $wd = 0.01$, $lr = 0.001/0.0005$, batch size 384.

A.4 Details on Task Mano: Knowledge Manipulation

The synthetic pretraining task Mano evaluates a model's ability to manipulate stored knowledge mentally without relying on explicit intermediate cues (e.g., Chain-of-Thought reasoning). Unlike memorization tasks, Mano requires multi-step internal computation, testing the model's capacity for hierarchical manipulation.

Task format and setup. The dataset is defined by a maximum length L, with each instance consisting of arithmetic expressions of ℓ operations, where ℓ is sampled uniformly from $[1, L]$. Expressions are presented in prefix (pre-order) notation to eliminate ambiguities in parentheses and operator precedence. For example, a length-$\ell = 3$ instance is:

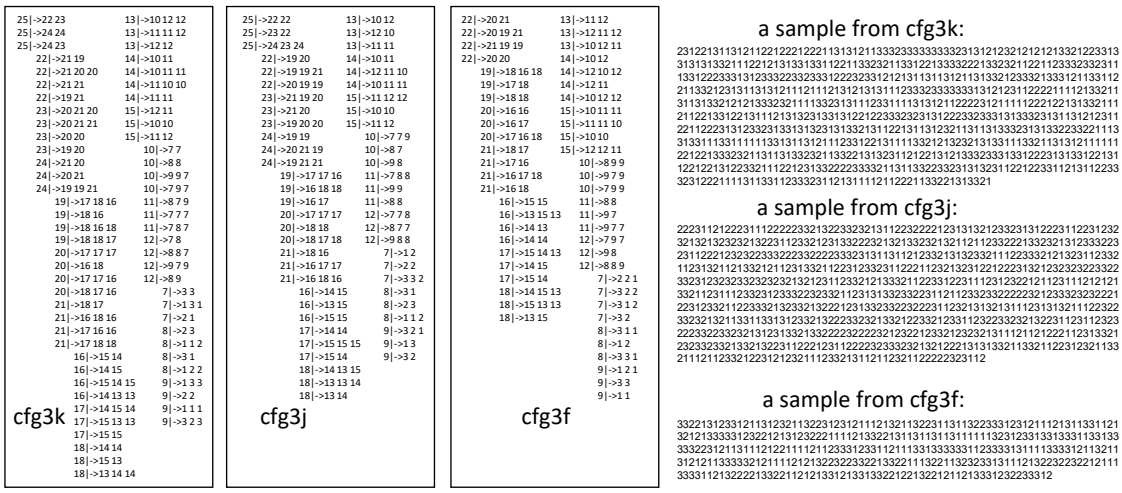


Figure 17: Task LANO: **our constructed dataset** cfg3k,cfg3j against the cfg3f dataset from [6].

```
<bos> <len_3> + * a b - c d <ans> ans
```

This corresponds to the expression $((a \times b) + (c - d)) \bmod 23$, where operands a, b, c, and d are integers sampled uniformly from $[0, 22]$. The task involves three operations (+, -, *), each represented as distinct tokens, with all computations performed modulo 23.

The factual base consists of three 23×23 arithmetic tables (addition, subtraction, and multiplication), which models learn implicitly during pretraining. Operands are encoded as single tokens from $[0, 22]$, while special tokens (`<bos>`, `<ans>`, and `<query_`ℓ`>` for $\ell \in [L]$) structure the sequence.

Expressions are generated recursively: the first operator is sampled uniformly from the three available options, and its operands are split into sub-lengths ℓ' and $\ell - 1 - \ell'$ (with ℓ' chosen uniformly), recursively generating sub-expressions.

Why modular arithmetic? Modular arithmetic (mod 23) ensures manageable knowledge size while introducing sufficient diversity in intermediate and final results. Similarly, limiting operations to addition, subtraction, and multiplication simplifies task design while retaining depth, enabling models to focus on hierarchical manipulation instead of memorizing surface-level patterns.

Training protocol. Models are pretrained on three datasets corresponding to difficulty levels $L = 16$, $L = 13$, and $L = 10$. The cross-entropy loss is applied over all tokens (problem description and answer), without label masking, since hierarchical manipulation requires attention across the full sequence. Instances are generated online, concatenated, and left-aligned into context windows of length 1024.

Models are trained from scratch using fixed random seeds for consistency across architectures. Training lasts 110k, 95k, and 80k steps for $L = 16$, $L = 13$, and $L = 10$, respectively. Hyperparameters include a batch size of 64, AdamW optimizer ($\beta = 0.9, 0.98$ and $\varepsilon = 10^{-6}$), weight decay of 0.1, learning rate warmup for 1000 steps, and cosine decay to 10% of the initial learning rate. Results are reported based on eight training runs with learning rates $\{0.0001, 0.0002, 0.0003, 0.0005\}$ and two seeds.

Evaluation protocol. During evaluation, expressions are sampled from the same distribution used for training, with ℓ fixed at L (maximum difficulty). Accuracy is computed over all problem instances within 1024-token context windows, including non-first instances. Since outputs are single tokens representing exact modular arithmetic results, partial correctness is not applied.

A.5 Details on Task Lano: Hierarchical Language Structure

The synthetic pretraining task LANO evaluates a language model's ability to perform structural reasoning, specifically long-range structural planning that requires dynamic programming to resolve ambiguity. Unlike in-context reasoning tasks (e.g., DEPO, BREVO) or knowledge reasoning tasks (e.g., MANO), LANO challenges models to learn hierarchical structures governed by probabilistic context-free rules and process sequences that cannot be resolved locally.

Task format and setup. Sentences are generated probabilistically using context-free rules. The cfg3f dataset [8] starts with the root non-terminal (NT) symbol 22, which uniformly expands into one of four rules:

$$22 \mapsto 20\ 21, \quad 22 \mapsto 20\ 19\ 21, \quad 22 \mapsto 21\ 19\ 19, \quad 22 \mapsto 20\ 20.$$

Each rule is chosen with probability 1/4, ensuring uniform randomness. Rules are applied recursively and probabilistically to NT symbols (e.g., 19, 20, 21), replacing all NT symbols with terminal (T) symbols 1, 2, or 3. The process generates sentences composed entirely of terminal symbols based on probabilistic expansions.

Pretraining involves predicting next tokens in CFG-generated sequences without access to the underlying rules, requiring models to learn structural reasoning implicitly. During evaluation, models are prompted with a single <bos> token and tasked to generate CFG-compliant sentences using temperature 1. Accuracy is assigned only for fully valid sentences, with no partial credit applied.

Parsing difficulty and ambiguity. Parsing CFG-generated sequences is uniquely challenging because resolving derivation chains requires global reasoning. For example, parsing "221213133" requires resolving structural ambiguity between terminal symbols that cannot be inferred from local patterns alone. Instead, parsing requires an $O(n^3)$ dynamic programming algorithm to globally reconstruct relationships across the sequence, even when CFG rules (from Figure 17) are explicitly available. During pretraining, models face additional difficulty as they must learn these relationships without direct access to the probabilistic rules.

Building upon cfg3f as a baseline, we introduce two extended datasets in this paper:

- cfg3k: Retains the probabilistic framework of cfg3f but increases depth by one level, doubling sequence length and increasing parsing complexity by eight times due to the cubic nature of dynamic programming ($O(n^3)$).
- cfg3j: Extends cfg3f by one level but reduces the number of rules, creating shorter sequences with intermediate difficulty between cfg3f and cfg3k.

Both datasets use the same probabilistic generation process and are detailed in Figure 17.

Training details. Pretraining uses cross-entropy loss computed over all tokens without label masking. Sentences are generated online, concatenated, and aligned into context windows. For cfg3f, we use a context length of 512 as in [8], while longer datasets cfg3j and cfg3k require extended context lengths of 1536.

Models are trained from scratch using fixed seeds for consistency across architectures. Training uses a batch size of 96, AdamW optimizer ($\beta = 0.9, 0.98$ and $\varepsilon = 10^{-6}$), weight decay of 0.1, no learning rate warmup, and linear decay to 0. Pretraining lasts 100k steps, and results are reported from four training runs using learning rates $\{0.0002, 0.0003, 0.0005, 0.001\}$.

Evaluation details. During evaluation, models generate sentences from a <bos> prompt using temperature 1 and beam width 1.[32] Generated sentences are validated using an $O(n^3m)$ dynamic

[32]This is crucial to ensure that the model is generating the genuine probabilistic distribution of sentences; if using temperature 0 for instance, the generation is always a fixed string, and accuracy would be either 0 or 100% forever.

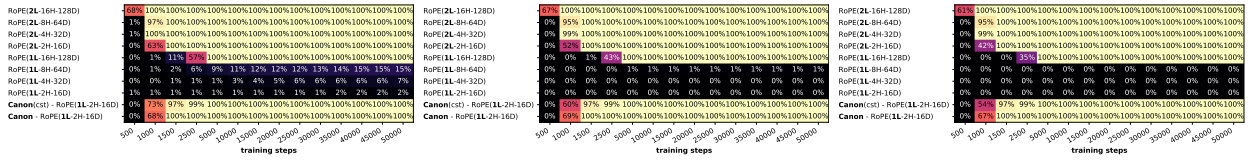
programming parser (n: sequence length, m: CFG rules) to confirm compliance. An alternative evaluation computes KL divergence between the model's next-token prediction distribution and the ground-truth CFG predictions. Both methods yield consistent architecture comparisons.

B Details on Other + Real-Life Experiments

This section provides a brief description of additional tasks used in the paper.

Full Copy. In Figure 5, we evaluated the performance of models with 1 or 2 layers on a trivial pretraining task. This task involves choosing $N = 500$ and generating a sequence starting with `<bos>`, followed by a random permutation of N tokens between 1 and N, then appending `<query>` and an identical copy of the sequence. The task uses label masking, where the loss is computed only on the N answer tokens. Models are pretrained with a context length of 1024, a total batch size of 32, AdamW optimizer ($\beta = 0.9, 0.98$ and $\varepsilon = 10^{-6}$), weight decay of 0.03, learning rate warmup for the first 1000 steps, and cosine decay to 10%. Training duration is set to 50k steps, and the best results are reported across learning rates $\{0.0005, 0.001, 0.002, 0.005\}$.

For this task, we also assessed the models' ability to correctly copy the first $t = 1, 2, 4$ tokens within the sequence. As shown in Figure 18, these results are nearly identical to those in Figure 5.



(a) Evaluated with $t = 1$ (b) Evaluated with $t = 2$ (c) Evaluated with $t = 4$

Figure 18: A trivial experiment for copying 500 tokens, evaluated only on correctly copying the first t tokens.

Task 1-hop-L and 2-hop-L. In the real-life experiment (Section 8), we evaluated models' performance on extremely simple 1-hop and 2-hop information retrieval tasks.

For the 1-HOP-L task, we prepared five random birth year statements of the form "[name] was born in the year of [year]," where names are generated as random combinations of first, middle, and last names, and years are sampled uniformly from 1950 to 2009. The five sentences were embedded into random Wikipedia documents of length L tokens, with each statement inserted between sentences at up to five randomly chosen positions. Finally, the model was prompted with "`\n\n`Answer me: name was born in the year of" to test its ability to retrieve the birth year. This setup closely replicates the needle-in-a-haystack task [29], but we intentionally made the task more "natural English" by using birth years (commonly found in pretraining datasets like Wikipedia) instead of abstract multi-digit numbers.

For the 2-HOP-L task, three random birth year statements were prepared in the same format as above. This was followed by three equivalence statements of the form "[name1] was born in the same year as [name2]," where random names were generated such that the equivalences formed a bijection between the two sets of three random names. To simplify the task, we did not shuffle the ordering of the statements; the three equivalence statements always followed the three original ones. These six sentences were then embedded into random Wikipedia documents of length L tokens, inserted at up to six different positions between sentences, respecting their original order. At the end, the model was prompted with "`\n\n`Answer me: name was born in the year of" to test its ability to infer and retrieve the correct birth year. To further assist the model, an instructional

statement was added at the beginning of the context. [33] This design represents arguably the simplest possible and most natural 2-hop in-context reasoning task, yet even with $L = 0$, models largely failed to perform, as demonstrated in Figure 16.

Babilong. For the Babilong experiments, we found the default few-shot prompts (qa1–qa5) slightly suboptimal and replaced them with improved ones, which are released in our GitHub repo [2].

SlimPajama and FineWeb-edu 100B. The SlimPajama dataset is taken from HuggingFace (`cerebras/SlimPajama-627B`), using the first 100M samples (more than 100B tokens). FineWeb-Edu [42] is obtained from `HuggingFaceFW/fineweb-edu`, using its predefined 100B split. Both datasets provide sufficient scale for our 1.3B-model pretraining experiments.

Following standard practice, all data are tokenized in order, concatenated into a continuous text stream, and sampled into random 4096-token windows for pretraining across architectures. We train with total batch size 48 using AdamW (β_1=0.9, β_2=0.98, ϵ=10^{-6}, weight decay 0.03). Llama and GPT models use learning rates $\{0.001, 0.002\}$, while linear models (Mamba, GLA, GDN) use $\{0.0005, 0.001, 0.002\}$ for stronger baselines. Each model is trained for 510,000 steps, processing $4096 \times 48 \times 510{,}000 \approx 100.2$B tokens per run. For each evaluation task, we report the *best accuracy* across the tested learning rates.

To ensure fairness, all architectures share the same random seed, guaranteeing identical data order and content—even if runs are interrupted and resumed. This setup minimizes variability from data differences and isolates architectural effects. For Llama(RoPE), we additionally test eight random seeds to measure variance, shown in Figure 16 and detailed in Appendix E.1, including both joint (data + model init) and model-init-only random seed variations. Architecture specifications appear in Appendix C.1.

Beyond 100B-1.3B. We find that academic-scale pretraining (100B tokens, 1.3B models) is too noisy to reveal subtle architectural gaps (e.g., Llama vs. Llama+Canon). Larger-scale experiments (1–8B models pretrained on 1–2T tokens) are therefore reported in our follow-up work [2].

C Details on Architectures Used

Transformer Models (Llama/GPT). In this paper, "Llama(RoPE)" refers to the Huggingface (HF) implementation LlamaForCausalLM, which employs rotary embeddings across all hidden dimensions and utilizes gated MLP layers. We did not enable group-query attention, as this study focuses on smaller-scale models. The intermediate size is set to $\frac{8d}{3}$, ensuring that each MLP layer contains $8d^2$ trainable parameters, consistent with standard MLP layers. "Llama(NoPE)" refers to the same architecture with rotary embedding completely disabled. "Llama(RoPE)♩" refers to the version where rotary embeddings are applied to only a quarter of the dimensions. The variants ♩, ♩♩, and ♩♩♩represent differing fractions of dimensionality on which RoPE is enabled, as described in the main paper.

For direct comparisons, "GPT2(RoPE)" refers to the Llama architecture with gated MLP layers replaced by standard MLP layers. The intermediate size in these models is set to $4d$, ensuring that each MLP layer contains $8d^2$ trainable parameters.[34]

We denote "GPT2(RoPE,R2)" as the GPT2(RoPE) model with its `silu` activation replaced by ReLU^2, following the design proposed in Primer [59]. Similarly, "Llama(RoPE,R2)" refers to

[33] "You will be asked questions about people's birth years, and the birth year descriptions are hidden in some random text. Some people's birth years are directly given, while others are given in the form that 'name1' was born in the same year as 'name2'. "

[34] The original GPT2 architecture differs from Llama in other minor ways, such as using GeLU activation and slightly different initialization. We do not investigate these small architectural differences in this paper.

Llama(RoPE) with ReLU^2 in place of `silu`.

ALiBi and H-ALiBi. For ALiBi [45], we follow the original recommendation of using a geometric sequence $2^{-8/n}$ for an n-head Transformer, which determines how each head is biased toward local context. For H-Alibi [31], we use their proposed strategy of restricting the h-th head to attend only to the nearest h tokens, and applied to half of the heads. (We briefly tested applying this to one-third of the heads instead, but observed slightly worse performance.)

Mamba Models. For "Mamba2," we use the HF implementation Mamba2ForCausalLM, with recommended configuration parameters (2 means expansion factor):

`ssm_state_size=64`, `num_heads=16`, and `head_dim=hidden_size * 2 / num_heads`.

This setup ensures each Mamba layer has $6d^2 + o(d^2)$ trainable parameters. The recurrent state size (per layer) is therefore $2d \times$ `ssm_state_size` $= 128d$ plus conv1d. We briefly tested `num_heads=8` but observed worse results, so did not include it. The model initialization follows the HF default (which uses PyTorch default init as opposed to a fixed 0.02 std init).[35]

For "Mamba2(mlp)," we use the same HF implementation but alternate between Mamba SSM layers and gated MLP layers. The intermediate size for gated MLP is $2d$, ensuring each MLP layer contains $6d^2$ trainable parameters. This ensures that ℓ-layer d-dimensional Llama(RoPE) and Mamba2(mlp), as well as 2ℓ-layer d-dimensional Mamba2, have comparable parameter counts.

Mamba1. We briefly tested Mamba1 and found it consistently outperformed by Mamba2 in our pretraining playground, so we excluded it from main results. Notably, removing its `conv1d` layer also degrades Mamba1 to GLA-level performance.

Mimetic initialization. Following [66], we enabled $A \approx 1$ (via c=8), $\Delta \approx 1$ (via b_Δ=0.54), $W_C^\top W_B \approx I$, and conv1d $\approx I$. We also tested c=4 and c=2 but observed no improvement.

GLA Models. For Gated Linear Attention (GLA) [72], we use the official `fla-org` implementation [71].[36] We use 4 linear attention heads (their default configuration; also suggested by their first author). With $d = 512$ or 768, this corresponds to `headdim` $= 128$ or 192, thus the recurrent state size (per layer) is $\frac{d}{2} \times$ `headdim` $= 64d$ or $96d$ — *both smaller than the Mamba2 models we tested.* We briefly tested 8 attention heads but found that these consistently degraded performance. Each linear attention layer contains about $4d^2$ trainable parameters; the (gated) MLP has an intermediate size of $\frac{8d}{3}$, contributing roughly $8d^2$ parameters, matching Llama.

The default GLA implementation has disabled `conv1d` (the functionality was not part of the original publication [72]), although their codebase supports `conv1d`, which we explicitly tested in this paper. They used 0.02 as initializer std for such `conv1d` layers with SiLU activation.

For GLA(elu) experiments in the ablation studies, we replaced the default feature map with $elu(x) + 1$, and conducted evaluations with and without `conv1d` and Canon layers.

GDN Models. For Gated DeltaNet (GDN) [73], we use the official `fla-org` implementation [71].[37] We use 4 or 6 heads for $d = 512$ or $d = 768$, respectively (as suggested by their first author). This

[35] We briefly tested the 0.02 init and did not observe significant difference.

[36] We use the default `expand_k` = 0.5 and `expand_v` = 1. From March to May 2025, the repo authors updated `initializer_range` to 0.006 (from the previously popular 0.02), which we found to negatively affect performance. We reverted it to 0.02; the authors also restored this value on May 3, 2025. We further disabled `rescale_prenorm_residual` for fair comparison. This option, inherited from GPT-2 [48], scales down the output projection (e.g., `o_proj`) initialization by $1/\sqrt{N}$, where N is the number of residual layers. The default HF implementations of Llama and Mamba2 both have this disabled, whereas the `fla-org` implementations enable it by default. We find that disabling this slightly improves model performance, and after communicating with Yang and Zhang [71], they also disabled it on June 24, 2025. Some of these were introduced after V2.0 of this paper, leading to small diffs in experimental results compared to V1.1.

[37] Similar to GLA (see Footnote 36), we adopt `initializer_range`=0.02 and disable `rescale_prenorm_residual`. Note their default `expand_k` = 0.75 and `expand_v` = 1.5.

corresponds to key/value headdim of $(96, 192)$, giving a recurrent state size (per layer) of $144d$, comparable to Mamba2. Each GDN layer contains about $6d^2$ trainable parameters, so we set the (gated) MLP intermediate size to yield another $6d^2$ parameters, matching Llama per layer block.

Weight tying, tokenizer. Unless otherwise stated (i.e., in Task CAPO), we do not tie weights between the embedding and output layers in any of the architectures (e.g., Llama, Mamba, GLA, GDN). Additionally, no tokenizers are used during pretraining except for Task CAPO.

Task Capo. The knowledge-capacity task pretrains on synthetic biographies following [8]. For consistency, we use `GPT2Tokenizer` and tie embedding/output weights to minimize capacity loss in small models (though the effect is minor).

Since CAPO measures bit-per-parameter knowledge capacity, both model and data scales are increased to assess scaling behavior. Following [8], we adopt the ℓ-h **notation** for model size, where Llama(ℓ-h) has ℓ layers, hidden size $64h$, and h heads, and extend this convention to GLA, Mamba2, and GDN for comparability.[38]

GPT2 experiments in Figure 11 use the original GPT2 architecture augmented with RoPE, as in [8]. Mixture-of-Experts (MoE) experiments employ `tutel` [30] with 32 standard MLP experts ($topk$=1, cap_factor=2).

C.1 Real-Life Experiments

For pretraining experiments on SlimPajama and FineWeb-Edu, we use all the architectures listed above alongside the Llama2 tokenizer (with vocab size 32,000) [65]. Weight tying is disabled to maintain consistency with prior works (e.g., [10, 73] and references therein).

The architectural configurations used in the real-life experiments are summarized below. They follow the setups described in Section C, except that we increase both width and depth to yield approximately 1.35B parameters per model:

- **Llama (RoPE/NoPE):** 24 layers, 32 heads, hidden size $d = 2048$.
- **GLA:** 24 layers, 4 heads, hidden size $d = 2048$.
- **Mamba2:** 48 layers, 16 heads, hidden size $d = 2048$.
- **Mamba2(mlp):** 24 layers, 16 heads, hidden size $d = 2048$.
- **GDN:** 24 layers, 12 heads, hidden size $d = 2048$.

For linear models (excluding `conv1d`), the per-layer recurrent state sizes are $256d$ for GLA, $128d$ for Mamba2 (with twice the layers), and $192d$ for GDN. These are of the same order of magnitude, while remaining close to the original authors' recommended settings. Each model contains roughly $12d^2$ trainable parameters per layer (except Mamba2, which has $6d^2$ per layer but twice as many layers), ensuring a fair comparison across architectures.

C.2 Canon Implementations

Canon layers (i.e., type A,B,C,D) in this paper are implemented using PyTorch's `nn.Conv1D` with kernel size 4, zero padding, and default initialization (i.e., `kaiming_uniform_` with $a = \sqrt{5}$). Unlike in GLA/GDN and in most linear models, this choice of the "default initialization" makes their weights initialized at $O(1)$ instead of 0.02. Based on our testing, this, combined with Canon's residual link, usually gives a **very stable performance improvement, without ever hurting**.

[38] GLA: ℓ layers, hidden size $64h$, 4 fixed attention heads. Mamba2: 2ℓ layers, hidden size $64h$ (ssm state size 64 and num heads 16). Mamba2(mlp): ℓ layers, hidden size $64h$ (ssm state size 64 and num heads 16). GDN: ℓ layers, hidden size $64d$, and $\max\{4, 64d/128\}$ heads. This ensures comparable parameter counts across architectures.

We use `causal_conv1d` [23] for its fast CUDA implementation. Canon layers are applied after layer normalization (if present, e.g., Canon-A/C) and before any non-linearity (if present, e.g., Canon-B/D).

We refer to the original conv1d implementations inside GLA/GDN/Mamba2 as Canon-b, and we leave its configuration identical to what was proposed by the original authors. In particular:

- conv1d in GLA/GDN has 0.02 initialization, with activation, without residual;
- conv1d in Mamba2 has $O(1)$ initialization, with activation, without residual.

We refer to `cst-Canon` as the constant, untrained version of Canon(res), where the convolution weights are fixed to PyTorch's default initialization.

Our implementation of Canon-ABCD for Llama, as well as Canon-AbCD for GLA, GDN and Mamba2, have been open-sourced on GitHub [2] (up-to-date links at `physics.allen-zhu.com`).

D Extensions of Figures 8 and 15

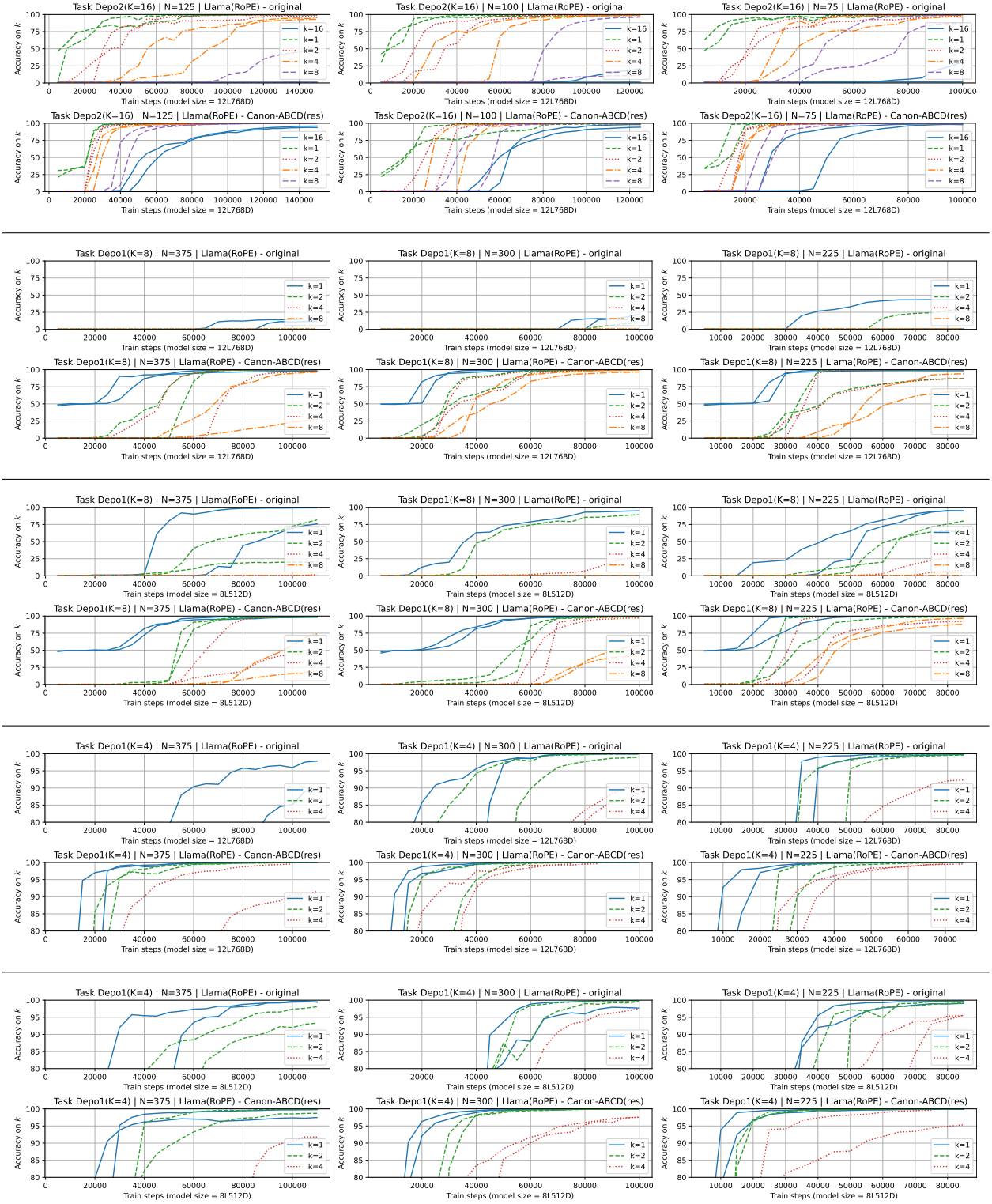


Figure 19: **This is an extension of Figure 8**: Training curves for 12L768 and 8L512D RoPE models, with and without Canon layers, on Task DEPO2($K = 16$), DEPO1($K = 8$), DEPO1($K = 4$), evaluated at varied depths and maximum graph size $n = N$, shown in two best learning rates.

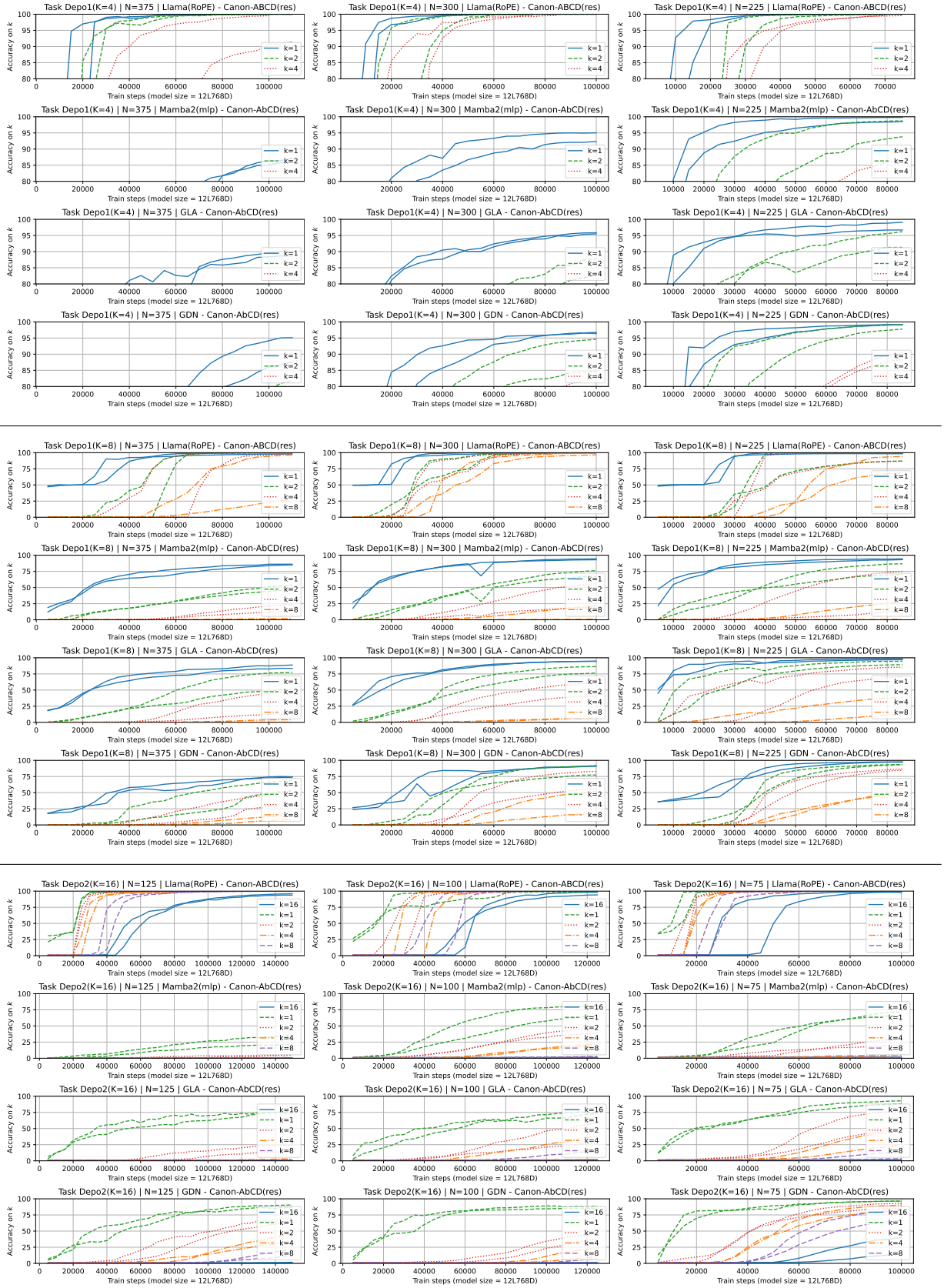


Figure 20: **This is an extension of Figure 15**: Training curves for 12L768D architectures on Task DEPO1(K=4 or 8), DEPO2(K=16), evaluated at varied k and maximum $n = N$; two best learning rates for each k.

E More Real-Life Experiments

E.1 Insufficiencies of Real-Life Pretraining at Academic Scale

As shown earlier in Figure 1, real-life pretrained models (FineWeb-Edu or SlimPajama) display large performance variance across random seeds. Here, we expand those results in Figure 21, including full experiments over eight seeds. Following feedback from an anonymous NeurIPS 2025 reviewer, we further test a controlled setup where data order is fixed and only model initialization varies—yet substantial benchmark variance remains.

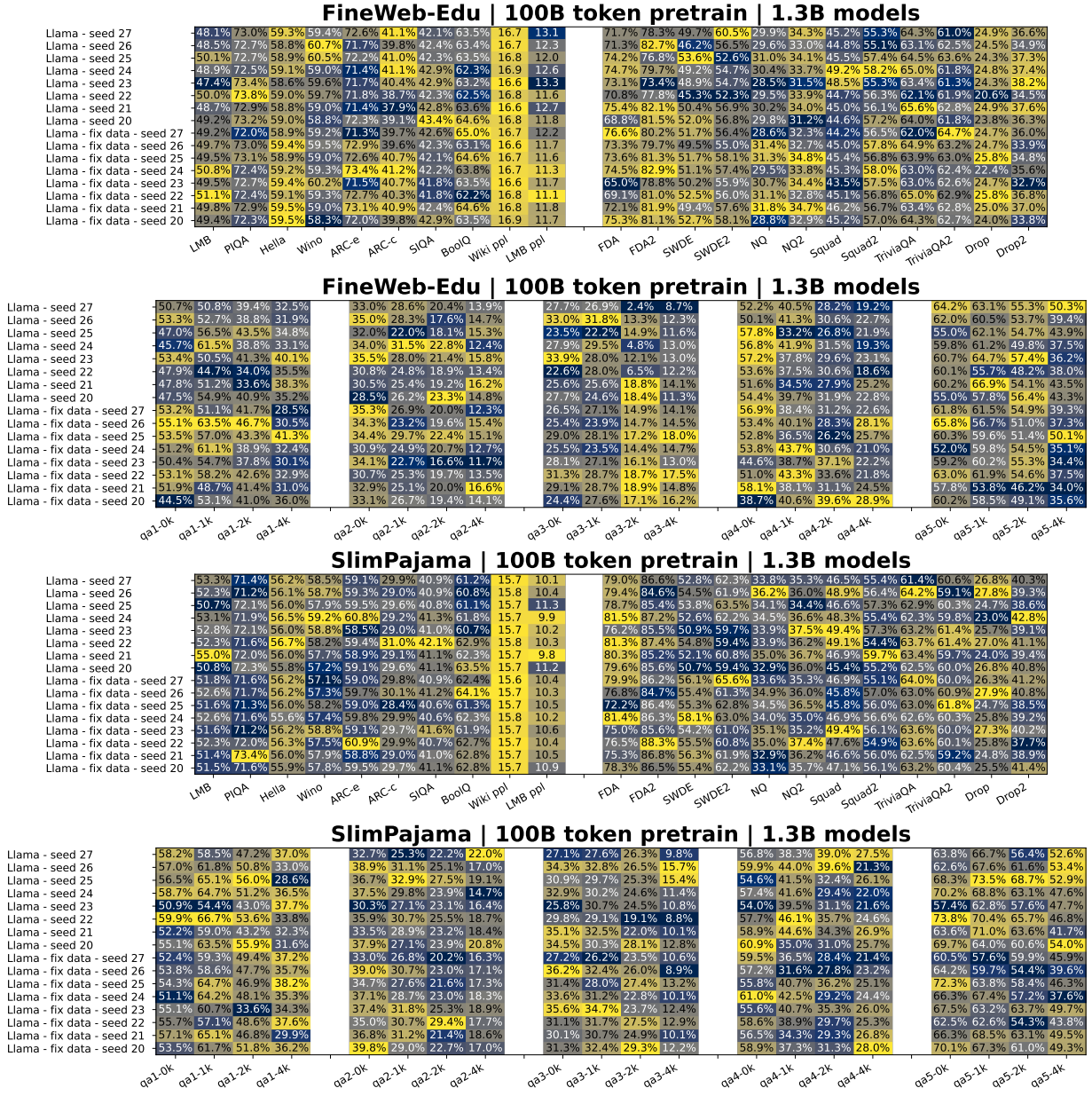


Figure 21: **Extended results to Figure 1** showing strong variance in benchmark accuracies for academic-scale real-life pretraining (1.3B models trained for 100B tokens).
Observations: Accuracy varies greatly across random seeds—both when changing data and initialization, and even when fixing data but varying initialization. HellaSwag [79] and wiki-ppl are relatively stable, though perplexity alone is an unreliable indicator of model capability.

E.2 Complete Real-Life Experiments

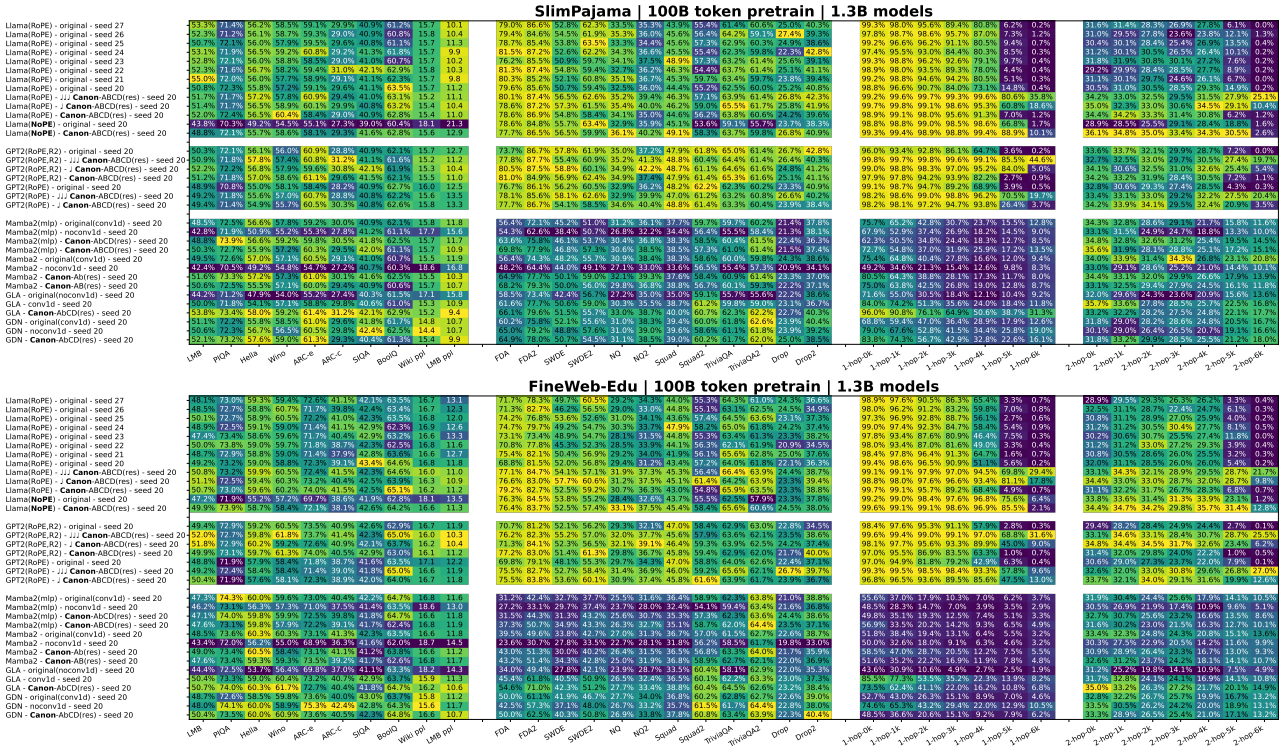


Figure 22: This is identical to Figure 16 but **additionally includes GPT2(RoPE)** models—identical to Llama(RoPE) but using standard MLPs—**and GPT2(RoPE,R2)**, which uses ReLU^2 activation [59]. **Key conclusions remain unchanged**: reducing RoPE improves length generalization, and many architectural differences (e.g., standard vs. gated MLP, `SiLU` vs. `ReLU`^{2}) are **buried in noise**.

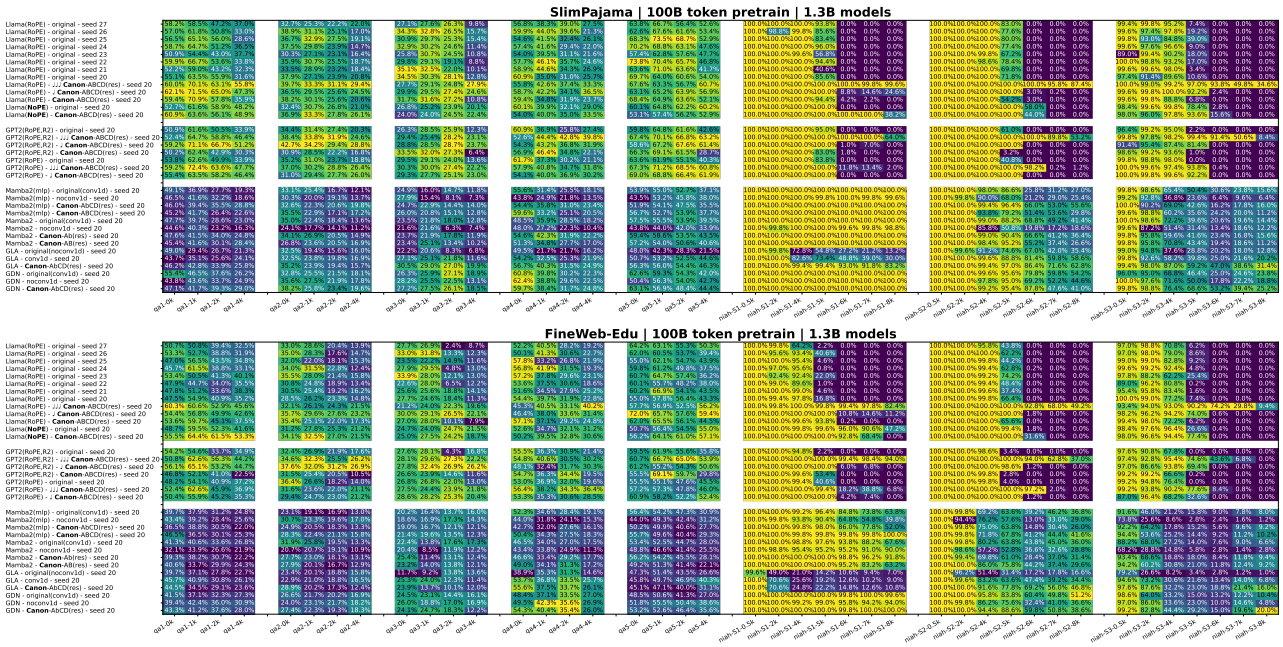


Figure 23: Results on the **Babilong + S-NIAH** dataset evaluating multi-hop reasoning across varied junk context lengths. Most architectural comparisons are statistically insignificant. **Key findings include:**

1. Linear models consistently underperform Transformers, even in short contexts without junk.
2. Models with reduced RoPE (NoPE, RoPE$\blacklozenge$) achieve notable improvements in long-context accuracy.
3. S-NIAH is *too easy*: linear models appear accurate but fail at short-context 1-hop retrieval (Figure 22).

F More Synthetic Experiments

We present missing figures that were intentionally omitted from the main body of the paper for the sake of clarity and conciseness.

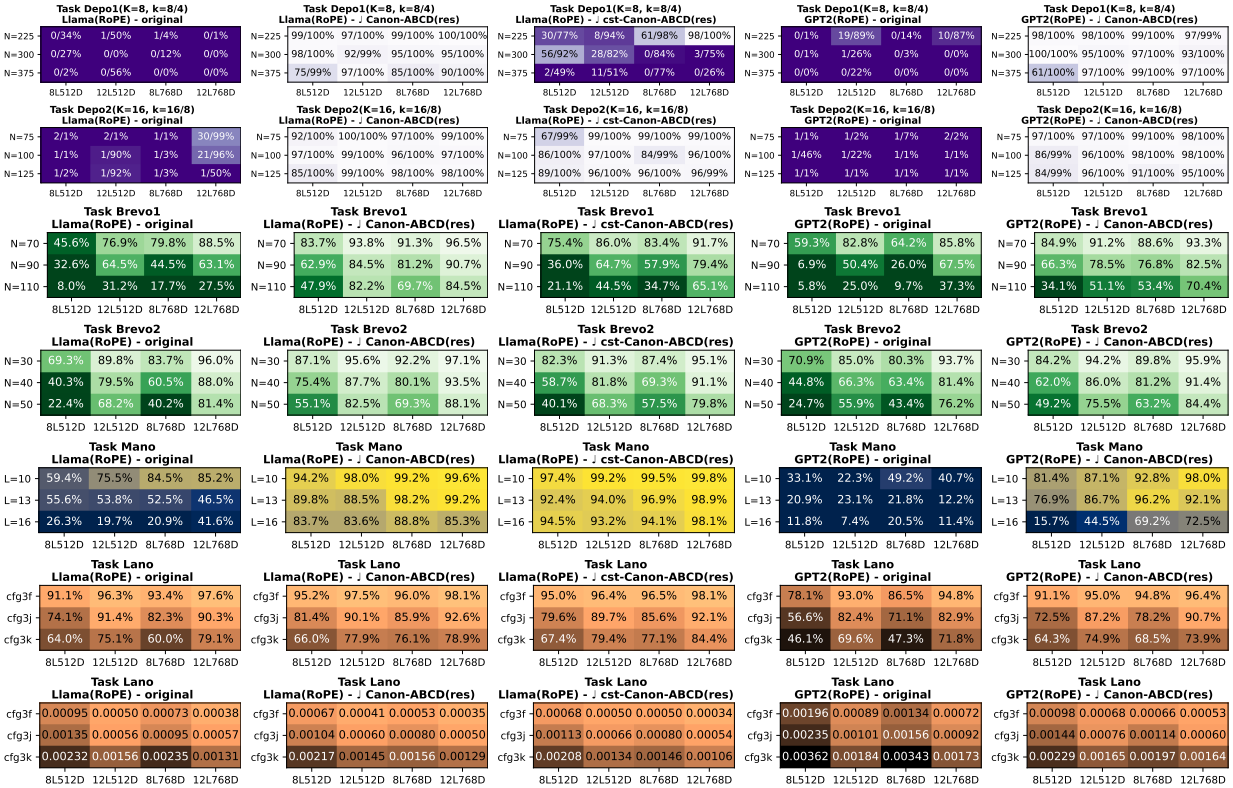


Figure 24: **Columns 1,2,3:** Constant Canon implementation (random, *non-trained* average of the past 3 tokens, denoted **cst-Canon**) already achieves strong performance, clearly outperforming vanilla Llama. **Columns 2,4,5:** Canon layers also perform strongly on GPT2 models (with standard MLP). Our playground reveals standard MLP is slightly weaker than gated MLP, especially in knowledge manipulation (cf. Result 5).

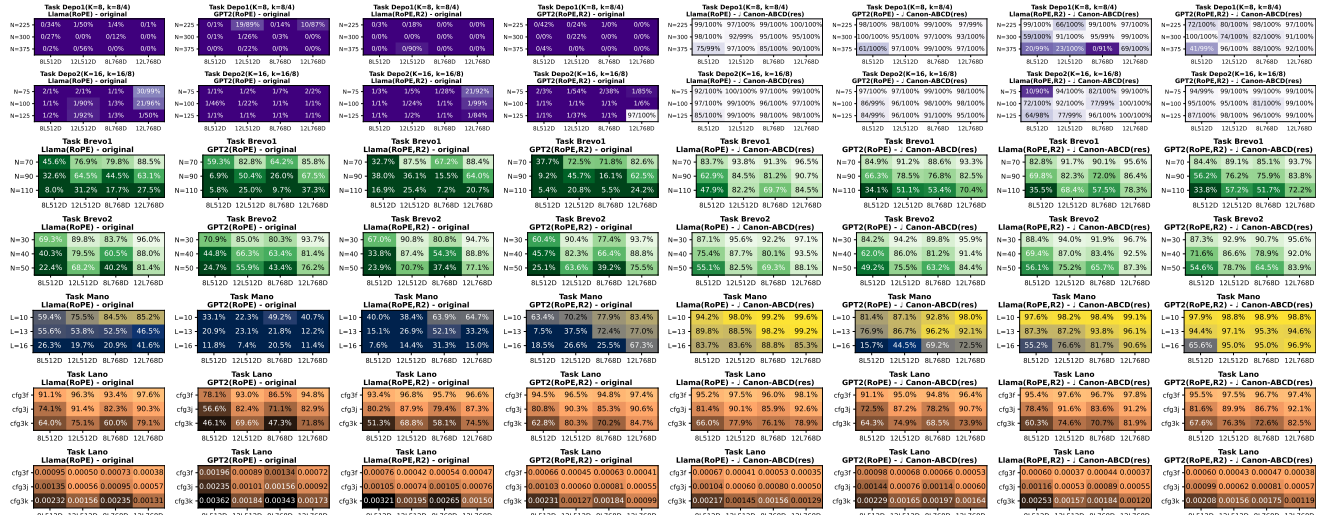


Figure 25: **Effect of ReLU² activation on standard vs. gated MLP. Columns 1→2, 5→6**: gated MLP outperforms standard MLP with `silu`. **Columns 2→4, 6→8**: adding ReLU² to standard MLP yields slight gains. **Columns 1→3, 5→7**: adding ReLU² to gated MLP hurts performance.

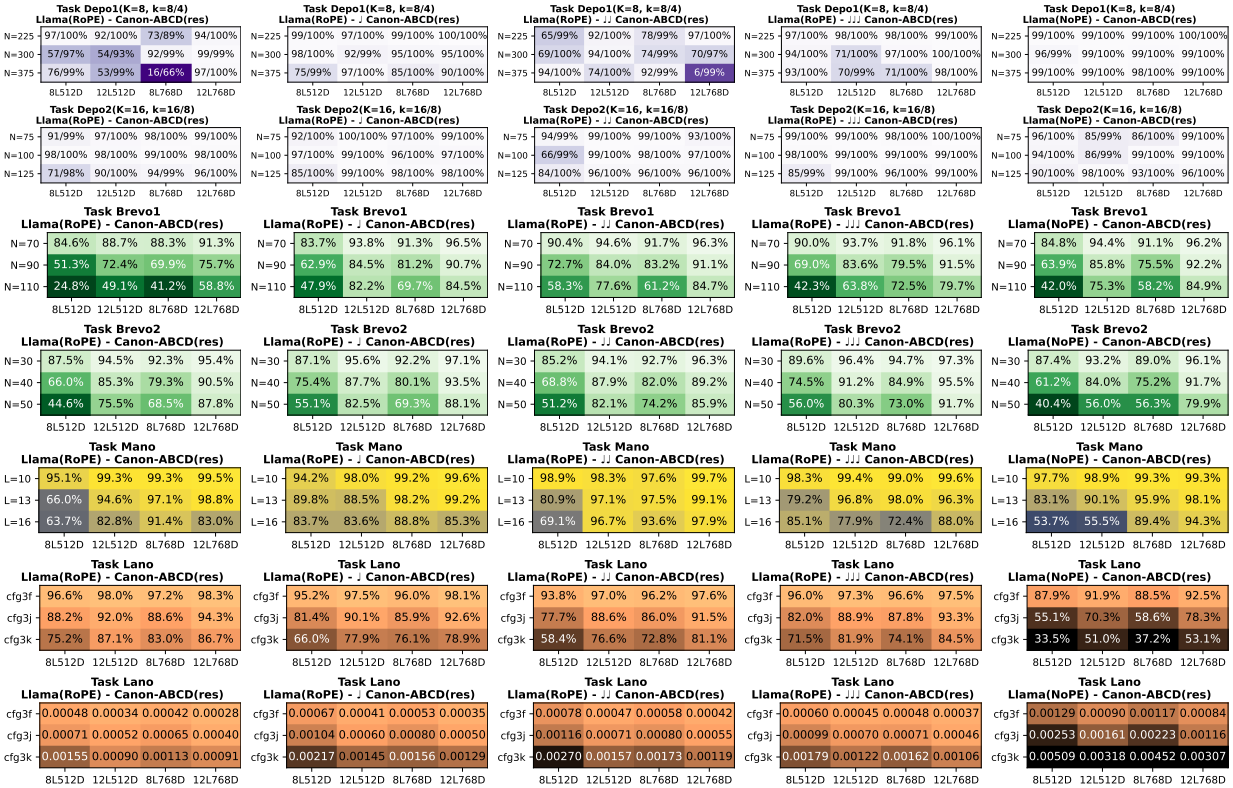


Figure 26: Transformer+Canon with **varying RoPE configurations**. From left to right: (1) RoPE; (2) RoPE♩: half of heads each with half RoPE dimensions; (3) RoPE♫: a quarter of heads with full RoPE dimensions; (4) RoPE♬: all heads each with quarter RoPE dimensions; (5) NoPE.

Conclusion: Canon layers eliminate the need for extensive RoPE usage, and reducing RoPE usage to 1/4 is even preferable, outperforming both full RoPE and NoPE setups. Among these reduced RoPE variants, RoPE♩ achieves slightly better overall performance.

G Complete Ablation Studies

This section presents full ablation results, including KL-divergence evaluations for Task Mano. These details were omitted from the main text for clarity but are included here for completeness and for readers seeking deeper experimental insight.

G.1 Llama(RoPE) family

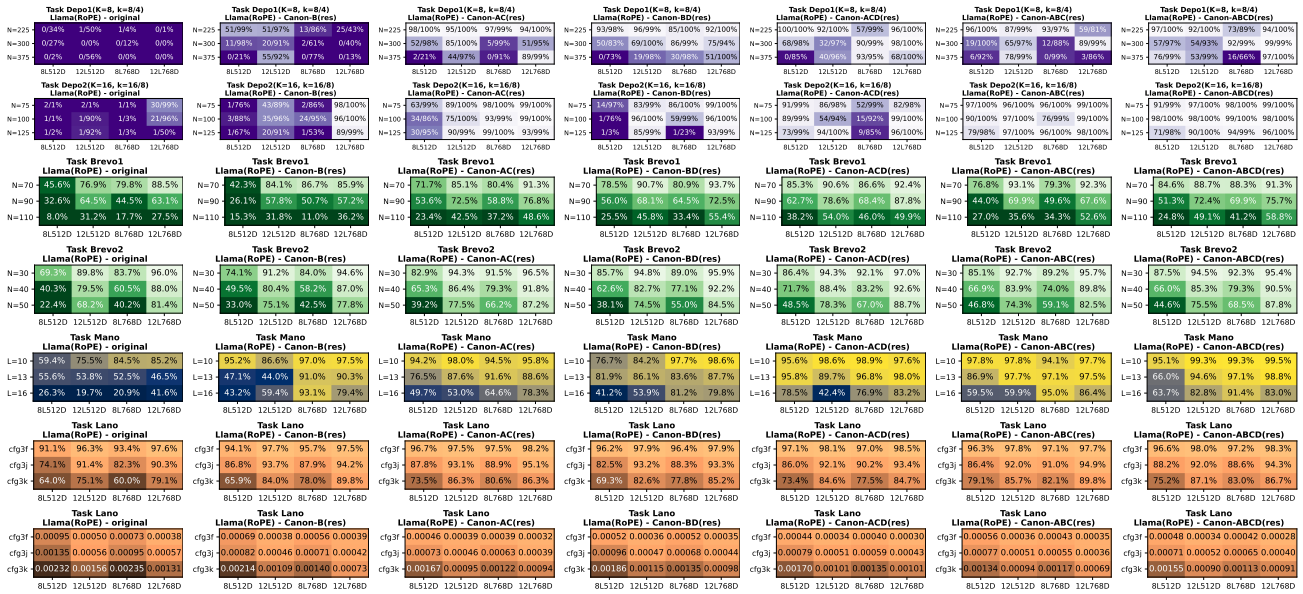


Figure 27: **Llama(RoPE) family**: (from left to right) original, Canon-B, -AC, -BD, -ACD, -ABC, -ABCD. This figure complements Figure 10 and gives more technical details.

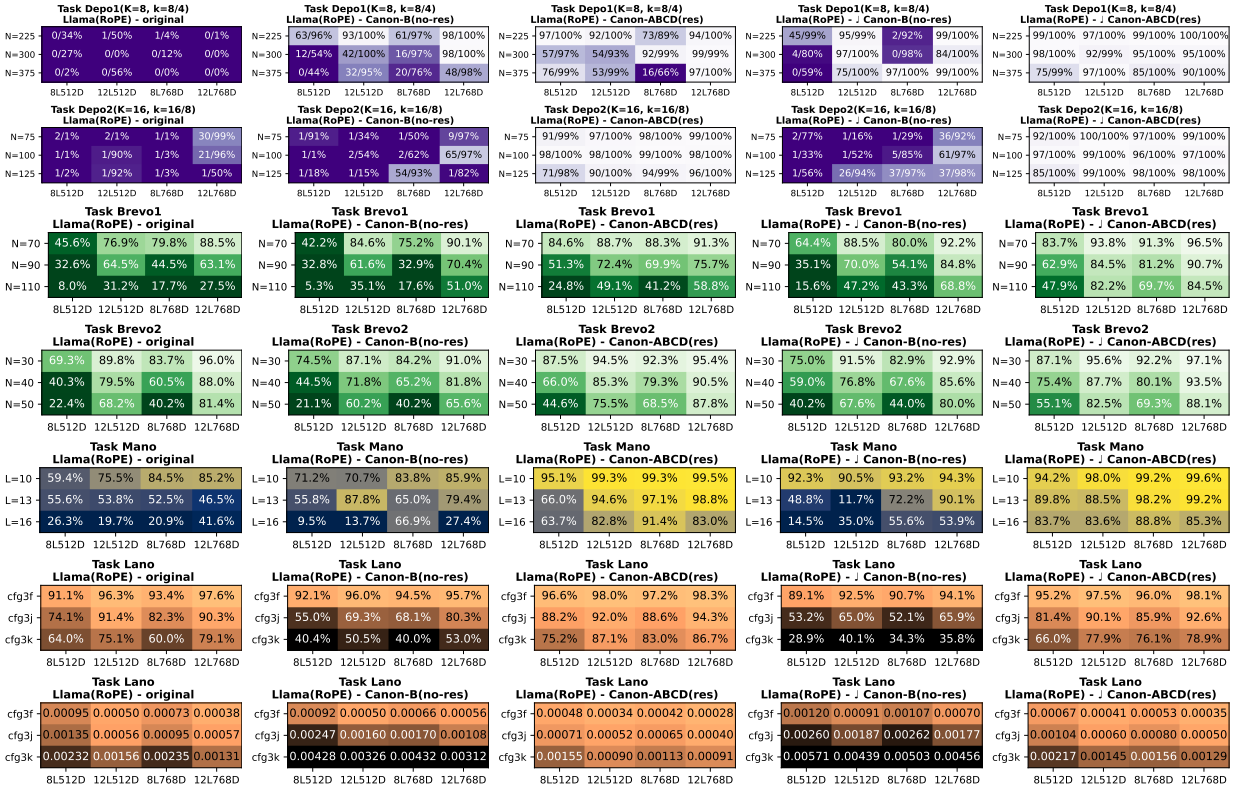


Figure 28: **Llama(RoPE) family**: (left to right) original, Canon-B(no-res), Canon-ABCD(res), ♩Canon-B(no-res), ♩Canon-ABCD(res). **This figure** complements Figure 10 and **directly compares to Primer [59] (i.e., Canon-B(no-res))**, showing its inefficiency and highlighting: (1) Canon layers are *not tied* to Attention; (2) Canon(res) at multiple points is safe and more effective.

G.2 Llama(NoPE) family

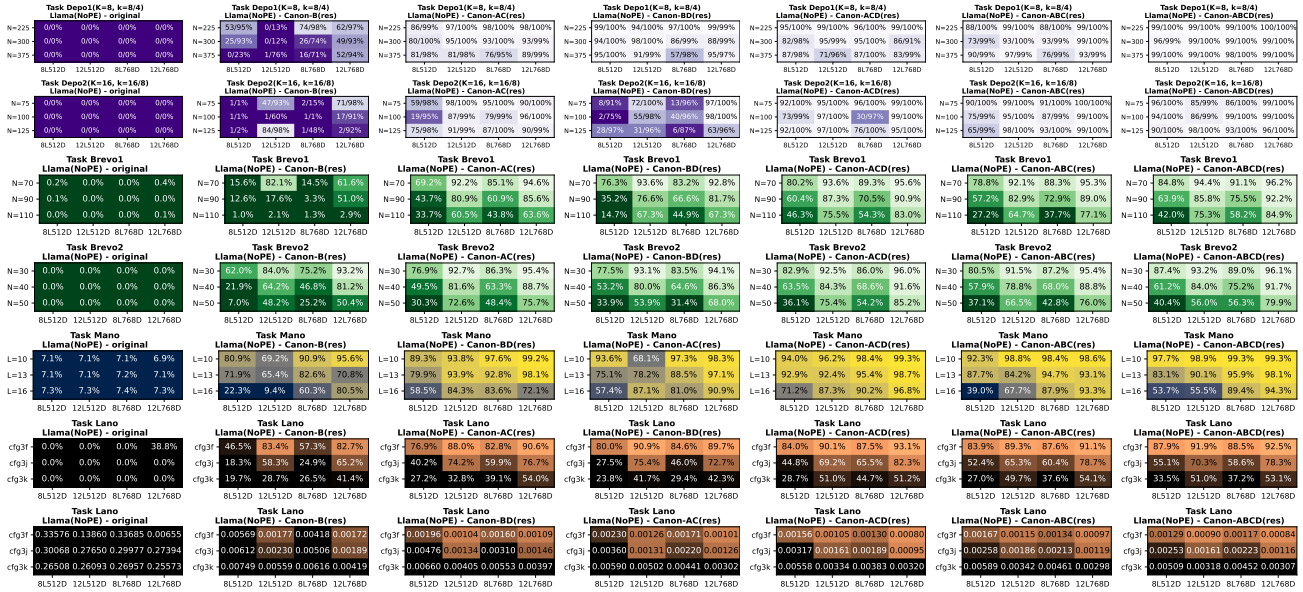


Figure 29: **Llama(NoPE) family**: (from left to right) original, Canon-B, -AC, -BD, -ACD, -ABC, -ABCD. This figure complements Figure 10 and gives more technical details.

G.3 Mamba2 family

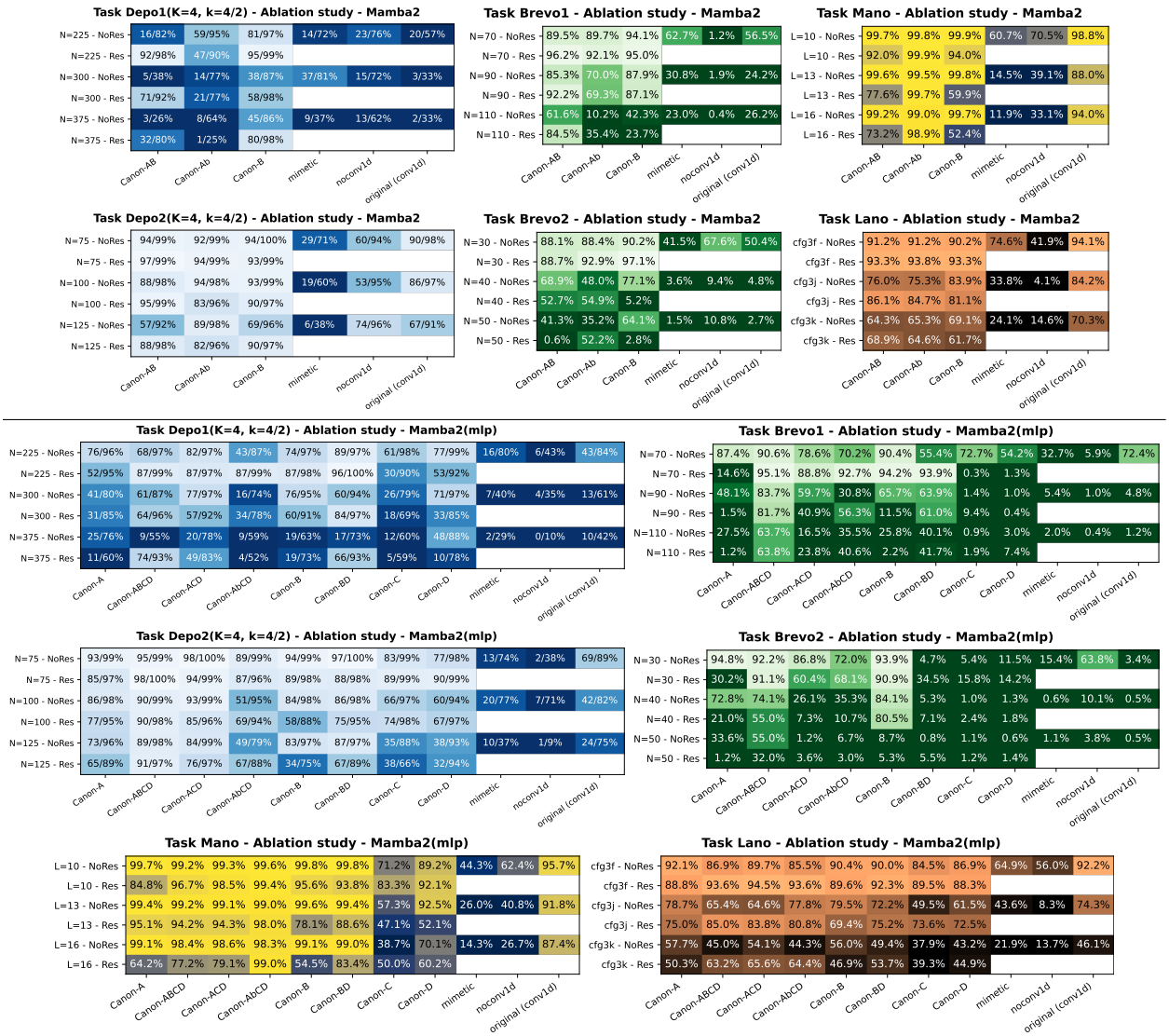


Figure 30: **Ablation study of Mamba2 models of 12L768D size** with Canon layers, Canon residuals, original non-linear `conv1d`, mimetic initialization. Full ablation studies (with additional model sizes, such as the effectiveness of Canon-ACD) are in Figure 31-32.

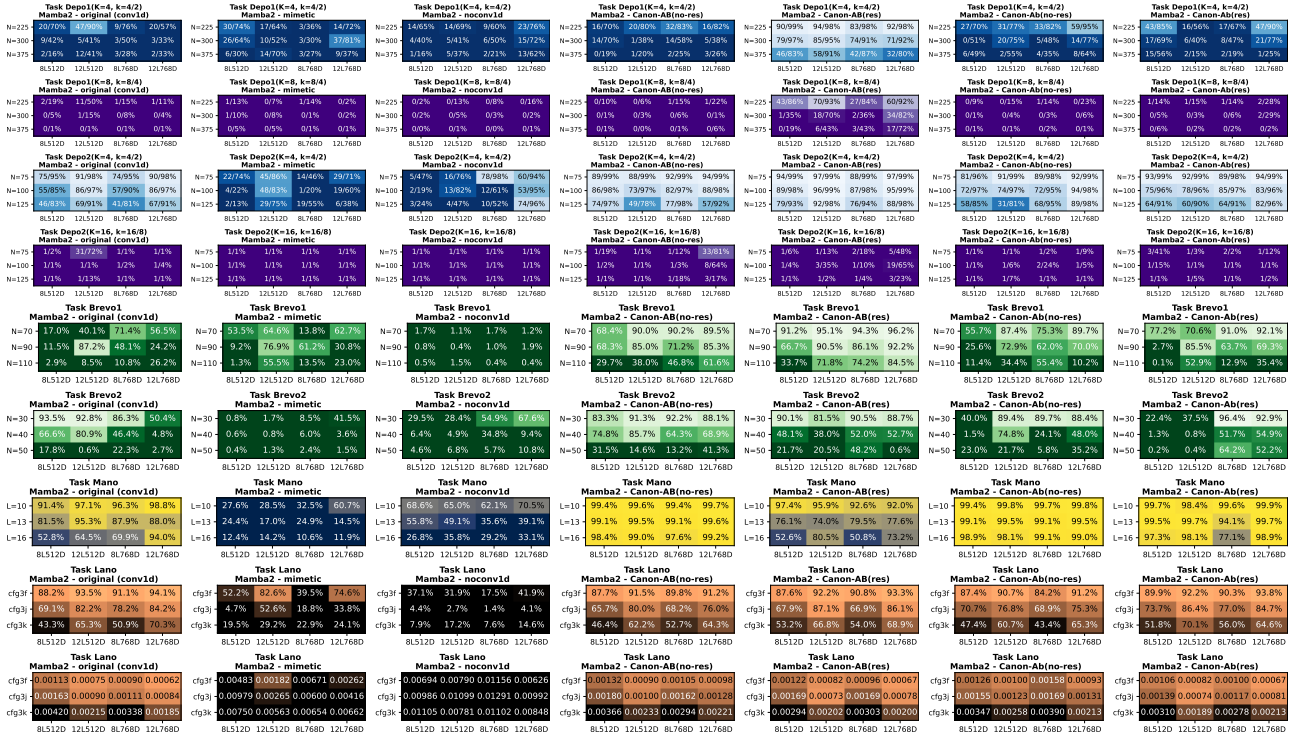


Figure 31: **Mamba2 variants** (left to right): original (conv1d), mimetic (w/ conv1d), no conv1d, Canon-AB(no-res), Canon-AB(res), Canon-Ab(no-res), Canon-Ab(res).

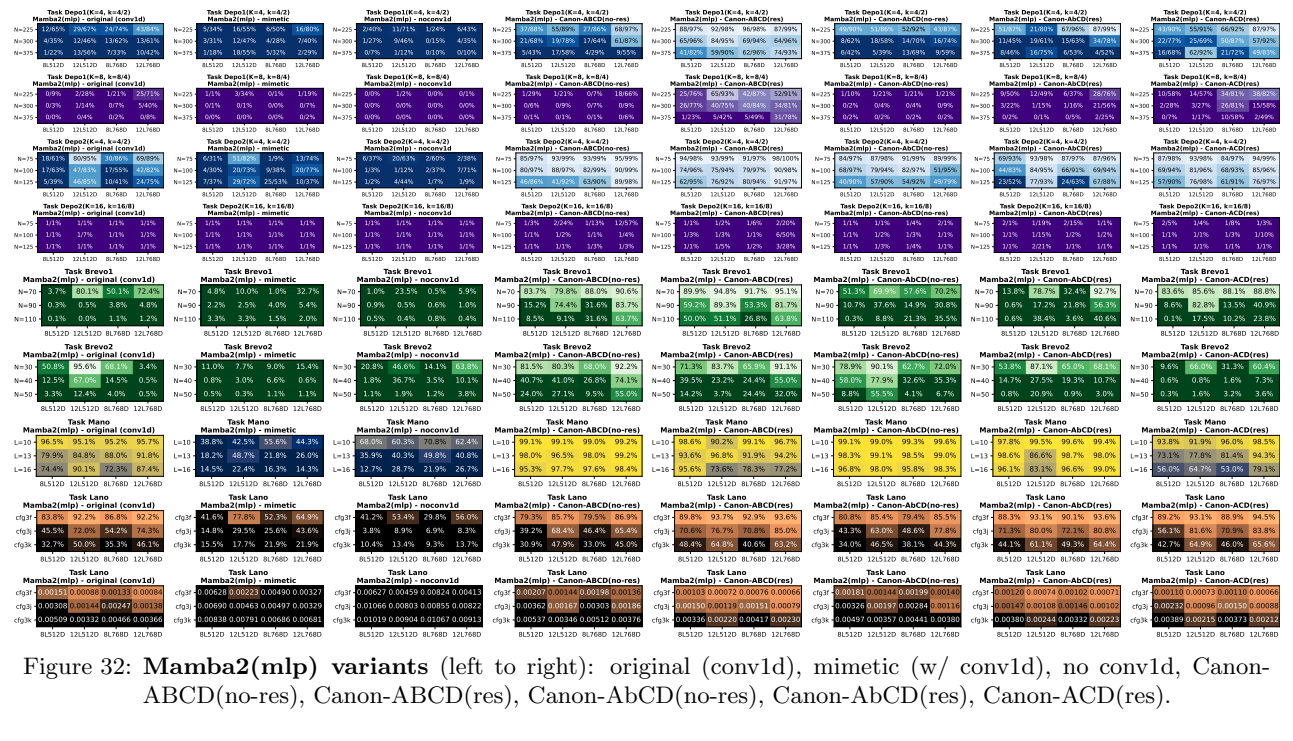


Figure 32: **Mamba2(mlp) variants** (left to right): original (conv1d), mimetic (w/ conv1d), no conv1d, Canon-ABCD(no-res), Canon-ABCD(res), Canon-AbCD(no-res), Canon-AbCD(res), Canon-ACD(res).

G.4 GLA family

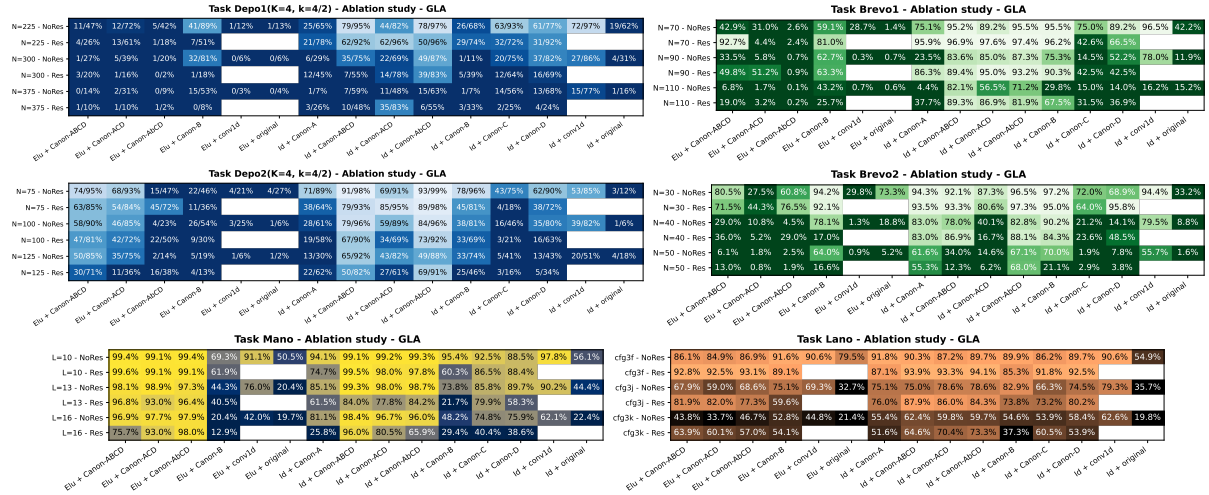


Figure 33: **Ablation study on 12L768D GLA** with Canon/conv1d layers, residual vs. non-residual, identity feature map vs non-linear ($\phi(x) = \text{elu}(x) + 1$) feature map.

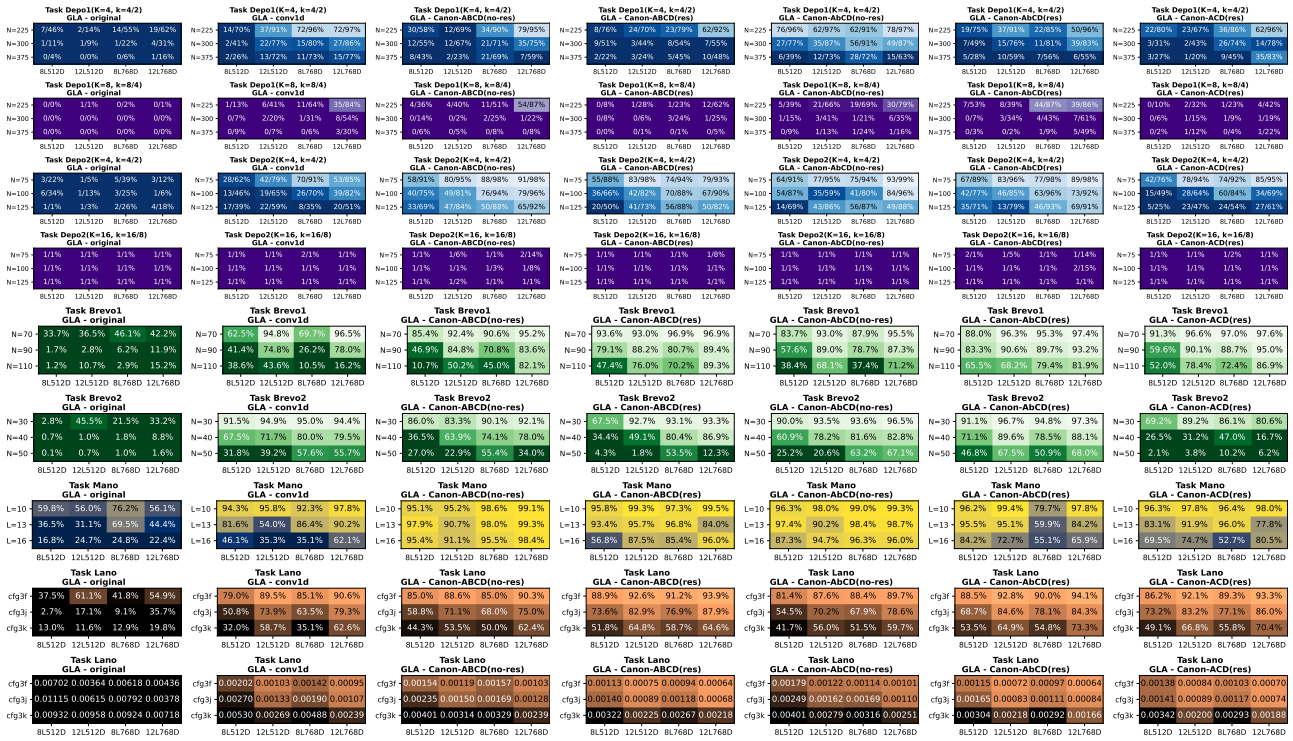


Figure 34: **GLA variants** (left to right): original, original + conv1d, original + Canon-ABCD(no-res), Canon-ABCD(res), Canon-AbCD(no-res), Canon-AbCD(res), Canon-ACD(res).

G.5 GDN family

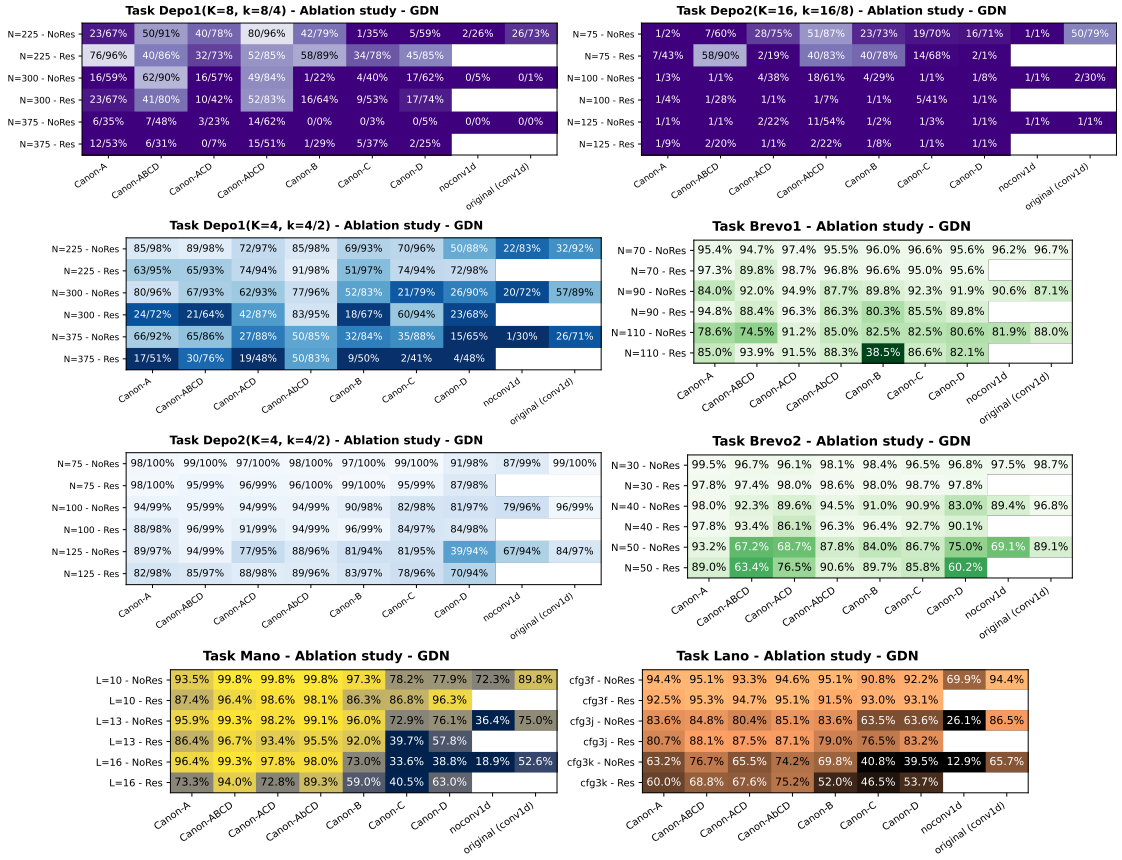


Figure 35: **Ablation study on 12L768D GDN** with Canon/conv1d layers, residual vs. non-residual.

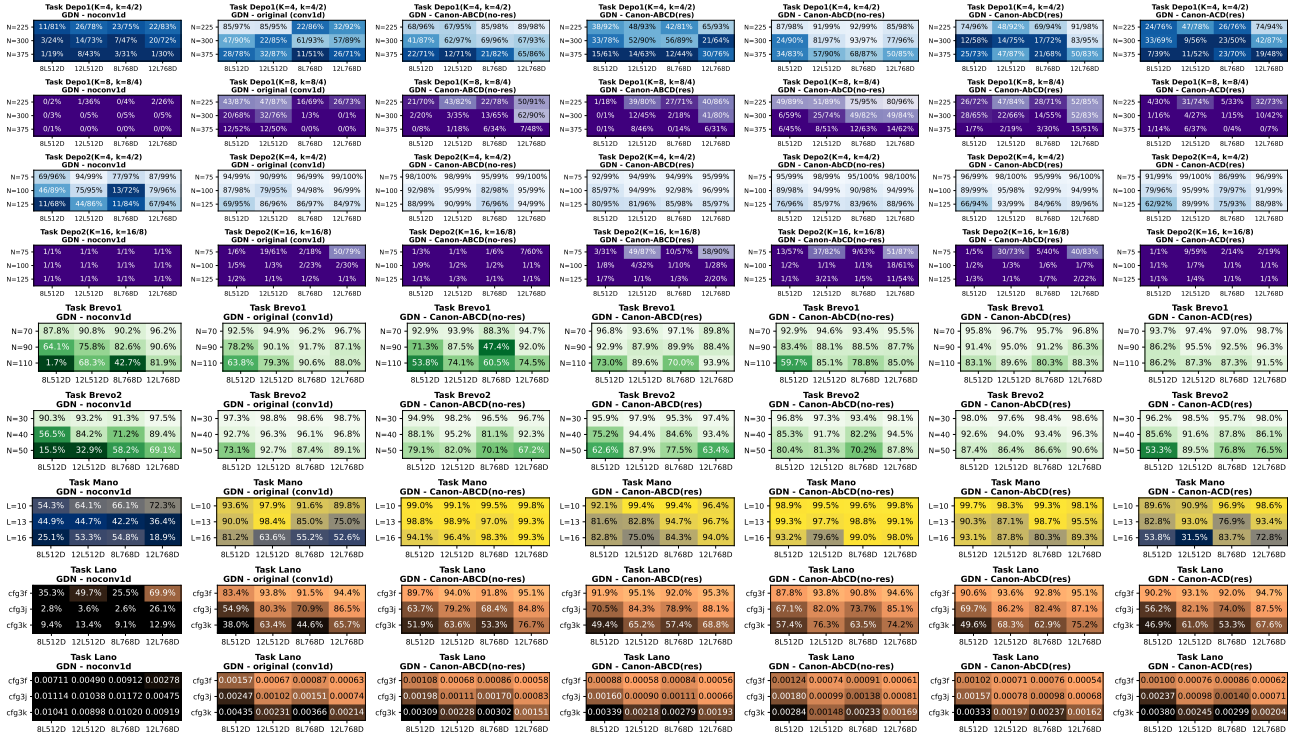


Figure 36: **GDN variants** (left to right): no conv1d, original (w/ conv1d), Canon-ABCD(no-res), Canon-ABCD(res), Canon-AbCD(no-res), Canon-AbCD(res), Canon-ACD(res).

References

[1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.

[2] Zeyuan Allen-Zhu. Physics of Language Models: Part 4.2, Canon Layers at Scale where Synthetic Pretraining Resonates in Reality, 2025. URL `https://physics.allen-zhu.com/part-4-architecture-design/part-4-2`. Code released at `https://github.com/facebookresearch/PhysicsLM4`.

[3] Zeyuan Allen-Zhu and Yuanzhi Li. Can SGD Learn Recurrent Neural Networks with Provable Generalization? In *NeurIPS*, 2019. Full version available at `http://arxiv.org/abs/1902.01028`.

[4] Zeyuan Allen-Zhu and Yuanzhi Li. Backward Feature Correction: How Deep Learning Performs Deep (Hierarchical) Learning. In *Conference on Learning Theory*, COLT '23, 2023. Full version available at `http://arxiv.org/abs/2001.04413`.

[5] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 3.1, Knowledge Storage and Extraction. In *Proceedings of the 41st International Conference on Machine Learning*, ICML 2024, 2024. Full version available at `http://arxiv.org/abs/2309.14316`.

[6] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 1, Learning Hierarchical Language Structures. *Transactions on Machine Learning Research*, 2025. Full version available at `http://arxiv.org/abs/2305.13673`.

[7] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 3.2, Knowledge Manipulation. In *Proceedings of the 13th International Conference on Learning Representations*, ICLR 2025, 2025. Full version available at `http://arxiv.org/abs/2309.14402`.

[8] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 3.3, Knowledge Capacity Scaling Laws. In *Proceedings of the 13th International Conference on Learning Representations*, ICLR 2025, 2025. Full version available at `http://arxiv.org/abs/2404.05405`.

[9] Simran Arora, Aman Timalsina, Aaryan Singhal, Benjamin Spector, Sabri Eyuboglu, Xinyi Zhao, Ashish Rao, Atri Rudra, and Christopher Ré. Just read twice: closing the recall gap for recurrent language models. *arXiv preprint arXiv:2407.05483*, 2024.

[10] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

[11] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.

[12] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

[13] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of the ACL Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022. URL `https://arxiv.org/abs/2204.06745`.

[14] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[15] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

[16] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[17] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2924–2936, 2019. doi: 10.18653/v1/N19-1300.

[18] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[19] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024. URL `https://arxiv.org/abs/2405.21060`.

[20] Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

[21] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246. URL `https://aclanthology.org/N19-1246/`.

[22] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.

[23] Daniel Y Fu, Tri Dao, Khaled Kamal Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052*, 2022. URL `https://arxiv.org/abs/2212.14052`.

[24] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024. URL https://zenodo.org/records/12608602.

[25] Olga Golovneva, Tianlu Wang, Jason Weston, and Sainbayar Sukhbaatar. Multi-token attention. *arXiv preprint arXiv:2504.00927*, 2025.

[26] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/abs/2312.00752.

[27] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

[28] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.

[29] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[30] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, Joe Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. Tutel: Adaptive mixture-of-experts at scale. *CoRR*, abs/2206.03382, June 2022. URL https://arxiv.org/pdf/2206.03382.pdf.

[31] Samy Jelassi, David Brandfonbrener, Sham M Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying. *arXiv preprint arXiv:2402.01032*, 2024.

[32] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730. URL https://doi.org/10.1145/3571730.

[33] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[34] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, 2017.

[35] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[36] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[37] Yury Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems*, 37:106519–106554, 2024.

[38] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

[39] Nayoung Lee, Ziyang Cai, Avi Schwarzschild, Kangwook Lee, and Dimitris Papailiopoulos. Self-improving transformers overcome easy-to-hard and length generalization challenges. *arXiv preprint arXiv:2502.01612*, 2025. URL https://arxiv.org/abs/2502.01612.

[40] OpenAI. Gpt-4 technical report, 2023.

[41] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, 2016. doi: 10.18653/v1/P16-1144.

[42] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.17557.

[43] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[44] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022. URL https://arxiv.org/abs/2201.02177.

[45] Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

[46] Zhen Qin, Songlin Yang, and Yiran Zhong. Hierarchically gated recurrent neural network for sequence modeling. *Advances in Neural Information Processing Systems*, 36:33202–33221, 2023.

[47] QwenTeam. Qwen3-Next: Towards Ultimate Training & Inference Efficiency, Sep 2025. URL https://qwen.ai/blog?id=4074cca80393150c248e508aa62983f9cb7d27cd&from=research.latest-advancements-list.

[48] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[49] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-2124. URL https://aclanthology.org/P18-2124/.

[50] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

[51] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

[52] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Socialiqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4463–4473, 2019. doi: 10.18653/v1/D19-1454.

[53] Eshika Saxena, Alberto Alfarano, Emily Wenger, and Kristin Lauter. Teaching transformers modular arithmetic at scale. *arXiv preprint arXiv:2410.03569*, 2024. URL https://arxiv.org/abs/2410.03569.

[54] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[55] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[56] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[57] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2016.

[58] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. Simplified state space layers for

sequence modeling. *arXiv preprint arXiv:2208.04933*, 2022.

[59] DR So, W Manke, H Liu, Z Dai, N Shazeer, and QV Le. Primer: Searching for efficient transformers for language modeling. arxiv 2021. *arXiv preprint arXiv:2109.08668*, 2021.

[60] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. `https://www.cerebras.net/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama`, June 2023. URL `https://huggingface.co/datasets/cerebras/SlimPajama-627B`.

[61] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021.

[62] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[63] Falcon-LLM Team. Falcon-h1: A family of hybrid-head language models redefining efficiency and performance, May 2025. URL `https://falcon-lm.github.io/blog/falcon-h1`.

[64] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[65] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[66] Asher Trockman, Hrayr Harutyunyan, J Zico Kolter, Sanjiv Kumar, and Srinadh Bhojanapalli. Mimetic initialization helps state space models learn to recall. *arXiv preprint arXiv:2410.11135*, 2024.

[67] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[68] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[69] Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart Van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015.

[70] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 22–31, 2021.

[71] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024. URL `https://github.com/fla-org/flash-linear-attention`.

[72] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

[73] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.

[74] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.

[75] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of Language Models: Part 2.1, Grade-School Math and the Hidden Reasoning Process. In *Proceedings of the 13th International Conference on Learning Representations*, ICLR 2025, 2025. Full version available at `https://arxiv.org/abs/2407.20311`.

[76] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of Language Models: Part 2.2, How to Learn From Mistakes on Grade-School Math Problems. In *Proceedings of the 13th International Conference on Learning Representations*, ICLR 2025, 2025. Full version available at `http://arxiv.org/abs/2408.16293`.

[77] Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1. *arXiv preprint arXiv:2407.06023*, 2024.

[78] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

[79] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019. doi: 10.18653/v1/P19-1472.

[80] Yu Zhang, Songlin Yang, Rui-Jie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, et al. Gated slot attention for efficient linear-time sequence modeling. *Advances in Neural Information Processing Systems*, 37:116870–116898, 2024.

[81] Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. Relu^2 wins: Discovering efficient activation functions for sparse llms. *arXiv preprint arXiv:2402.03804*, 2024.

[82] Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers can achieve length generalization but not robustly. *arXiv preprint arXiv:2402.09371*, 2024.